

## 目 录

## 图目录

## 表目录

## 主要符号表

## 第一章 前截断章节

### 1.1 当前版本编译时间

**二零二六年二月二日 21:43**

## 第二章 绪论

### 2.1 研究背景与意义

### 2.2 研究目标

### 2.3 研究所面临的挑战

### 2.4 研究内容

### 2.5 论文主要工作与创新点

### 2.6 论文组织结构

## 第三章 相关技术及相关研究综述

### 3.1 路线搜索和推荐

### 3.2 本章小结

## 第四章 动态路网中的交通拥堵缓解：面向行程请求流的持续最优路线组合

### 4.1 引言

### 4.2 研究背景及应用场景

随着基于位置服务的应用的兴起，路径规划服务已经成为我们生活中不可或缺的一部分。路径规划和行程推荐在近几年引起了学者们的广泛研究 [?????]。这些研究的研究目标是基于当前的交通状态为单行程制定最优的路线。值得关注的是，在路径规划服务使用越来越频繁的情况下，大量的用户很有可能会在极短的时间间隔内密集地发布行程请求，特别是在通勤时间等高峰时期，从而形成了持续的行程请求流。在这种新的场景下，实现针对行程请求流的路径规划这一需求变得更加迫切。已经存在的相关研究旨在为行程请求中的单个行程依次制定个人最优路线 [??]。然而，在为行程请求流规划路线时，仅仅基于当前的交通状态以个人最优为最终目标规划个人最优的路线，可能会导致交通拥堵。更合理的路线规划应该考虑到之前已经规划的行程会对未来的交通状态产生影响，因为它们会增加路网中各路段的车流量。

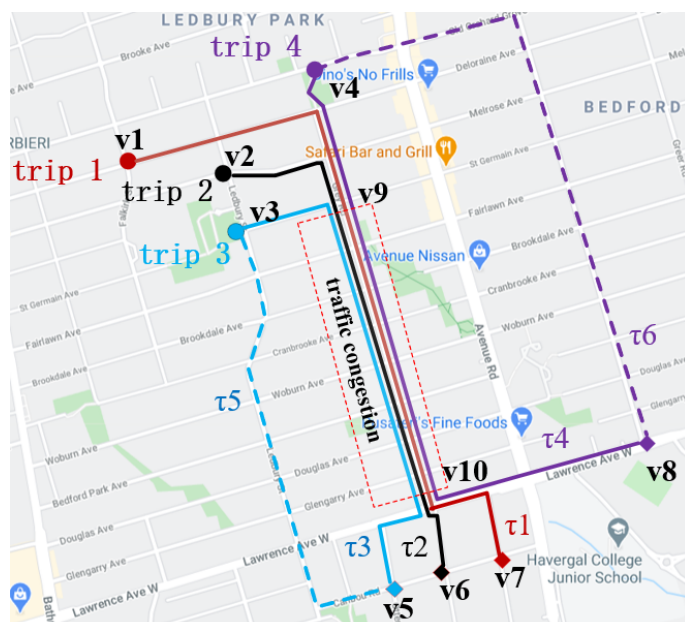


图 4-1 行程请求和路线

以图 1 为例，图中的每个行程包含一个起点和终点，分别用圆圈和钻石形状标记。顶点  $v_1 - v_4$  是行程 1-4 对应的起点，而顶点  $v_5 - v_8$  是行程 1-4 对应的终点。



行程请求 1–3 分别发布于 8:00:01, 8:00:01, 8:00:05。它们的最优路线分别是  $\tau_1, \tau_2$  和  $\tau_3$ ，在图中我们用实线标记。这三条路线预计分别会在 [8:02:00, 8:05:00], [8:01:40, 8:04:40], [8:01:30, 8:04:30] 行驶在路段  $e(v_9, v_{10})$ 。在 [8:02:00, 8:04:30]，它们都会都会行驶在路段  $e(v_9, v_{10})$ ，而这也许会导致交通拥堵，从而延路段  $e(v_9, v_{10})$  所有正在行驶的其他车辆的通行时间。行程 4 发布于 8:00:20，基于在 8:00:20 这一时刻的交通状态我们为它制定的最优路线为  $\tau_4$ 。很明显，该规划没有意识到路段  $e(v_9, v_{10})$  由于之前规划的路线  $\{\tau_1, \tau_2, \tau_3\}$ ，在未来可能会产生潜在的交通拥堵。糟糕的是，我们为行程 4 制定了最优路线  $\tau_4$ ，该路线再一次经过了路段  $e(v_9, v_{10})$ ，也许会进一步加剧交通拥堵。综上，给定一组密集发布的行程请求流，为了缓解交通拥堵，我们需要把它们看作一个整体进行路线规划。如例 1，我们可以先把行程 1–3 看作一个整体，考虑到可能产生的交通拥堵。我们可以调整行程 3 的最优路线为  $\tau_5$ ，在图??中用虚线表示。对当前到达的行程 4，通过把行程 1–4 看作一个整体，同理我们可以调整其最优路线为  $\tau_6$ 。这些调整后的路线对单个请求来说也许是次优的路线，但路段  $e(v_9, v_{10})$  的交通拥堵却可以得到大大缓解，行程 1–4 总的通行时间也会大大减少。通过采用  $\{\tau_1, \tau_2, \tau_5, \tau_6\}$  这一路线组合，行程 1–4 的总的通行时间可能会是最小的，我们称这样的一个路线组合是最优的路线组合。

#### 4.2.1 研究内容与挑战

本文提出和研究了一种针对持续的行程请求流的路线规划问题：在一个动态路网上给定一组行程请求流，行程请求流中的行程请求数是不断增加的，我们需要找到一个路线组合  $\Pi = \{\pi_1, \pi_2, \dots\}$  使得所有在  $\Pi$  中的路线的总的通行时间最小。其中，每一条路线  $\pi_i \in \Pi$  是为每一个具体的行程请求  $q_i$  对应规划的。该问题具有十分广泛的应用场景，包括但不限于城市交通流量管理，实时的路线规划以及持续的拥堵预防上。针对该问题的精确算法具有指数级的时间复杂度，因此不能有效地运用到实际的应用场景中。为了解决这个问题，我们提出了一个自感知的批处理算法。对应的实验研究证明了该算法的准确性和高效性。

## 4.2.2 研究成果与贡献

## 4.3 问题描述

### 4.3.1 Dijkstra 算法

### 4.3.2 位置点间的距离计算

### 4.3.3 问题定义

### 4.3.4 路网及其构成

**定义 4.1 (动态路网)** 一个动态路网  $G=(V,E)$  是由顶点集  $V$  和边集  $E$  构成的, 其中  $E \subseteq V \times V$ , 顶点代表路段的连接点, 交叉点; 连边则代表具体的路段。每条边  $e(v_i, v_j) \in E$  连接着两个顶点  $v_i$  和  $v_j$ , 这里  $v_i, v_j \in V$ 。为了便于描述, 我们假定所有行程请求的起点和终点位置都已经被映射到离它们最近的顶点上。针对每个路段  $e$ , 我们用  $C_e$  来代表该路段的车容量, 用  $T_m(e)$  代表在路段  $e$  的最小通行时间 (e.g., 当路段  $e$  上没有其他车辆时的通行时间)。在我们的设定中, 每一个路段的权重 (通行时间) 是动态变化的。一个路段的最小通行时间是  $T_m(e)$ , 而实际的通行时间是和该路段上的动态车流量成比例的。

**定义 4.2 (通行时间函数)** 车辆在时刻  $t$  经过路段  $e$  的通行时间和当前时刻路段  $e$  上的车流量  $f(e, t)$  相关。参照已经存在的研究 [??], 基于当前时刻路段  $e$  上的车流量, 我们使用下式来计算路段  $e$  在时刻  $t$  的通行时间, 我们用  $T(e, t)$  表示。

$$T(e, t) = T_m(e) \times (1 + \alpha \times (\frac{f(e, t)}{C_e})^\beta) \quad (4-1)$$

其中,  $C_e$  代表路段  $e$  的车流量,  $\alpha$  和  $\beta$  是由路段的不同性质 (e.g., 限速, 路宽) 等决定的可变参数。我们的解决方案是和这些参数独立的, 如何为这些参数设置合适的值并不在我们的研究范围内。由式??我们可以知道, 随着车辆数目的动态变化, 每条路段的通行时间也会随之发生改变。

**定义 4.3 (路线及其通行时间)** 我们用一个有限的顶点序列  $\langle v_0, v_1, \dots, v_{|\pi|-1} \rangle$  来表示一条路线  $\pi$ 。假定路线  $\pi$  出发时间为  $t_0$ ,  $t_i$  是其经过的顶点  $v_i$  的预计到达时间 (i.e.,  $t_i = t_{i-1} + T(e(v_{i-1}, v_i), t_{i-1})$ )。结合式??, 我们用式  $ET(\pi, t_0)$  来计算一条路线  $\pi$  的预计通行时间, 即路线  $\pi$  中每个路段的预计通行时间之和, 我们用  $ET(\pi, t_0)$  来表示。

$$ET(\pi, t_0) = \sum_{i=0}^{|\pi|-2} T(e(v_i, v_{i+1}), t_i) \quad (4-2)$$

之后我们使用  $\pi.v_0$  和  $\pi.v_{|\pi|-1}$  来分别代表一条路线  $\pi$  对应的第一个顶点 (起点) 和最后一个顶点 (终点)。

**定义 4.4 (行程请求流)** 我们用  $q = (s, d, t)$  来代表一个行程请求, 其中  $s$  是该行程的起点位置,  $d$  是该行程的终点位置, 而  $t$  是该行程的出发时间。  $Q = \{q_1, q_2, \dots\}$  由多个行程请求组成, 代表一组行程请求流 (i.e.,  $\forall q_i, q_j \in Q, i > j: q_i.t \geq q_j.t$ )。需要注意的是,  $Q$  是以流数据的形式到达的行程请求集合并且  $Q$  是动态更新的。

**定义 4.5 (全局最优路径规划问题)** 在一个动态路网  $G = (V, E)$  上, 给定一组行程请求流  $Q = \{q_1, q_2, \dots\}$  和时间间隔  $T$ , 持续的最优路线组合问题 (CORC 问题) 旨在不断地处理最近到达的一批行程请求  $Q_n = \{q_k, q_{k+1}, \dots, q_{|Q|}\}$  ( $Q_n \subset Q \wedge q_{|Q|}.t - q_k.t = T$ ), 找到一个最优的路线组合  $\Pi = \{\pi_1, \pi_2, \dots\}$  使得: (1)  $|Q| = |\Pi|$ ; (2)  $(\forall \pi_i \in \Pi) (\pi_i.v_0 = q_i.s \wedge \pi_i.v_{|\pi_i|-1} = q_i.d)$ ; (3) 在  $\Pi$  中规划的所有路线的总的通行时间最小, 我们用  $TT(\Pi)$  来表示所有路线的总的通行时间 (参照公式??)。

$$TT(\Pi) = \sum_{i=1}^{|\Pi|} ET(\pi_i, q_i.t) \quad (4-3)$$

#### 4.4 精确遍历算法

针对 CORC 问题, 一个简单直接的办法就是对每一个行程请求, 搜索起点和终点间可能存在的所有路线, 然后对所有的路线组合进行遍历评估, 比较各个路线组合所有路线总的通行时间, 最终筛选出具有最小通行时间的路线组合。具体来说, 对于每一个行程请求  $q_i \in Q$ , 我们可以采取深度优先算法 (DFS) 来查找从起点  $s_i$  到终点  $d_i$  可能存在的所有路线。其中, 我们可以应用一些剪枝策略来加速深度优先搜索。随后, 对于每一个路线组合我们计算它所有路线总的通行时间。最终, 我们返回具有最小通行时间的路线组合作为返回结果。精确算法比较简单直接, 便于理解。当行程请求数量  $|Q|$  较小时, 精确算法可以在有限时间内得到最优解。但是, 当行程请求数量  $|Q|$  很大时, 精确算法显然大量的计算时间, 不能运用到实际的应用场景中。

#### 4.5 自感知的批处理算法

为了高效解决 CORC 问题, 我们提出了一个自感知的批处理算法 (SBP 算法), 该算法主要包含两大组成部分: (1) 初始路线搜索和 (2) 批优化处理。具体来说, 给

定一组行程请求流  $Q_n$ ，我们首先采用初始路线搜索生成一个高质量的初始路线组合  $\Pi_n$ ，该组合包含了为该行程请求流中的每个请求指定的初始路线。接下来，我们执行批优化处理对初始路线组合中的每个初始路线进行优化。最终，我们将优化后的路线组合  $\Pi_n$  加入到全局的路线组合  $\Pi$  中去。每当我们处理完一批行程请求后，我们返回  $\Pi$  作为实时的规划结果。我们的自感知思想合理考虑了新规划的路线会影响未来的交通状态这一事实，即规划的路线会增加一些路段的车流量从而影响该路段的实际通行时间。

#### 4.5.1 初始路线搜索

初始路线搜索的目标是为新一批到达的行程请求流中的每个行程请求规划一条高质量的个人路线。每个路段的动态车流量由两部分组成的，一部分是在我们规划系统中的请求车辆所造成的交通流量，即我们规划的路线在各路段上造成的车流量，我们称之为系统内的车流量，一部分是系统外的车流量即在各路段上没有使用我们规划系统的车辆数目，我们把系统外的车流量直接作为算法输入。

##### 4.5.1.1 路段标签

为了实现对系统内车流量的快速计算，我们在每个路段上维持一系列的路段标签  $L_e = \{l_1, l_2, \dots\}$ 。这些路段标签记录了经过该路段的路线在该路段的时间信息。每个路段标签  $l_i = \{t_a, t_b\}$  记录了一条特定的路线进入路段  $e$  的时间  $t_a$  以及离开该路段的时间  $t_b$ 。刚开始没有行程请求的时候，每个路段维持的路段标签是一个空集。之后每当我们处理完一批新的行程请求集合的时候，对应路段的路段标签会进行动态地更新。

##### 4.5.1.2 系统内的车流量计算

我们可以快速地计算在各个路段上由我们规划系统规划的路线所产生的车流量。由我们规划系统规划的路线在路段  $e$ ，时刻  $t$  所造成的车流量，也就是满足  $[l_i.t_a, l_i.t_b] \ni t$  的路段标签  $l_i$  的数量，其中  $l_i \in L_e$ 。在我们的实验中，我们应用了一个优先队列来加速计算。

##### 4.5.1.3 通行时间下界

我们用一个启发值  $H(v_i, v_j)$  来代表从一个顶点  $v_i$  到另一个顶点  $v_j$  的通行时间下界。给定最小的系统外的车流量作为静态的车流量输入，该下界是由 Dijkstra 算法 [?] 预计算的。在本文中，我们将非请求车辆的车流量直接作为算法输入。

#### 4.5.1.4 算法描述

算法 ?? 提出了初始路线搜索的伪代码。算法输入包含一个动态路网  $G = (V, E)$ ，一个行程请求  $q = (v_s, v_d, t)$ ，所有两点间的通行时间下界  $H(v_i, v_j)$ ，所有路段的路段标签集合  $\mathcal{L} = \{L_{e_1}, L_{e_2}, \dots\}$ 。算法输出为行程请求  $q$  制定的一条初始路线。每个顶点  $v \in V$  包含以下记录信息：从起点  $v_s$  到达该点的精确通行时间  $t_s$ ；从该点到达终点的通行时间下界  $t_d$ ；从起点到达该点的最小时刻  $et$  以及一个前驱节点记录  $pred$ 。首先，我们初始化每一个顶点  $v \in V$  的信息记录，设置路线  $\pi$  是一个空集 (line 1)。接着我们把起点  $v_s$  加入一个优先队列  $PQ$  中。该优先队列  $PQ$  内部对顶点对象按照该点的  $v.t_s + v.t_d$  的值从小到大排序，队头元素为具有最小  $v.t_s + v.t_d$  的顶点 (line 2)。在搜索过程中，每一次我们从  $PQ$  选出队头元素  $v$  并探索它的邻接顶点。具体来说，每一次我们选出的  $v$  都具有最小的  $v.t_s + v.t_d$ ，之所以这样选择，是基于这样的顶点扩张可能会产生最小的通行时间 (line 4)。如果选择的  $v$  是行程请求的终点  $v_d$ ，我们就利用相应节点的前驱节点记录生成一条路线  $\pi$ ，并将  $\pi$  作为结果返回 (lines 5–11)。如果选择的  $v$  不是行程请求的终点  $v_d$ ，对于每一个从顶点  $v$  出发的路段  $e$ ，我们计算当前时刻该路段的车流量 (line 13)。对于路段  $e(v, v')$  的结束点  $v'$ ，我们需要检查该点能否通过顶点  $v$  到达，并且所花的时间比从其他点到达该点更小 (line 14)。如果满足，我们将分别更新顶点  $v'$  对应的  $t_s$ ,  $et$  和  $pred$  记录 (lines 15–17)。接下来，如果顶点  $v'$  不在优先队列  $PQ$  中，我们将其加入。当优先队列  $PQ$  为空或者我们已经扩张到行程请求对应的终点时，初始路线搜索算法终止。

初始路线搜索：对于每一个新到达的行程请求（包含起点，终点，出发时刻），寻找一条在出发时刻出发，从起点能最快到达终点的路径（最短路径）。假设车辆通过特定路段的时间是和该路段的属性和车流量决定的，而车流量由两部分组成。一部分是非请求车辆车流量，即没有使用我们路线规划系统的车辆，直接作为输入数据，不是我们研究的范畴；一部分是请求车辆的车流量，即使用了我们规划系统的车辆，这些车辆按照规划的路线行驶而在经过的路段上增加的车流量。由于路网每个路段车流量是动态变化的，通行时间也会随之动态变化，我们就是要在这样一个动态变化的路网中寻找从出发时刻出发，起点到终点的最短路径。

初始路线搜索过程从起点开始，对与起点相连的邻接顶点做网络扩张，接着选择一个新的顶点继续扩张，直到扩张到终点。我们的扩张策略是，每次选择一个最可能最快到达终点的顶点作为下一个扩张顶点，什么是最可能最快到达终点的顶点呢？即（从起点到该点的精确耗时 + 从该点到终点的最小耗时）最小的顶点。从起点到该点的精确耗时是根据每个路段的实时车流量准确计算出来的，而从该

点到终点的最小耗时是预处理中计算的。在预处理中，我们取每个路段的最小车流量作为静态的车流量，利用 *dijkstra* 算法计算顶点两两之间的最短路径（即耗时最小的路径），并将顶点两两之间的最小耗时存储起来作为我们初始路线搜索的算法输入。综上，我们从起点开始，按照该扩张策略一直扩张，直到扩张到终点，则我们就找到了一条在出发时刻能最快到达终点的路径。

**批优化处理：**对于在较短时间间隔（比如 1s）内发出的行程请求集合（包含起点，终点，出发时刻），我们为这些请求分别调用初始路线搜索后，这些初始路线可能会导致潜在的拥堵，从而增加对应路段的通行时间，（比如说行程数量很多，而它们规划的路线都要经过同一路段，则可能造成拥堵）。因此这些最优路线的部分路段是需要优化的，我们将在同一时间间隔（比如 1s）内发出的行程请求作为一个整体，对我们之前为它们规划的初始路线集合进行批优化处理。对于一批初始路线集合，我们批优化策略如下：1）从该集合中选取其中一条路线，将该集合内的其他路线看作已经发布的路线（其实还没有发布），假定这些用户将会按照发布的路线行驶，对应道路上就会增加一定车流量，从而全局车辆（包括之前已经规划的车辆在每个路段的耗时也会变化）。我们需要计算这些路线在各个路段增加的车流量，更新路段上的车流量，更新所有行程在各路段的通行时间，这些就相当于更新了交通信息。2）对于这个选取的初始路线，我们根据 1）更新的交通信息，重新查找一条从起点到终点的最短路径。如果该最短路径不是原来的初始路线且经过计算该最短路径可以减少全局所有行程的耗时超过一定幅度，我们称新搜索到的这条最短路径是有效的，将原来的初始路线更新为新的最短路径。3）重复 1）2）过程直到对于每一条路线，都不存在新的有效最短路径。将该批次优化后的路线发布给用户。

#### 4.5.2 批优化处理

为了提高算法质量，我们提出了一个批优化处理方法对由算法??产生的初始路线进行优化。一个初始路线组合  $\Pi_n$  包含了为最近到达的行程请求  $Q_n$  制定的初始路线。不同于  $\Pi_n$  的是，全局的路线组合  $\Pi$  包含了所有在当前时间之前已经规划的路线。优化处理过程重新利用了初始路线组合  $\Pi_n$ ，并通过一种自感应的批处理交换策略对  $\Pi_n$  中的每一条初始路线进行优化。批优化处理的精髓在于：在每一个优化过程中，我们选择一条路线  $\pi_i \in \Pi_n$ 。我们将除了该路线的其他正在规划的初始路线作为已知的路线规划输入到交通流量信息中，重新预测未来的交通状态并尝试重新分配一条新的路线  $\pi'_i$  给行程  $q_i$ 。我们用  $\pi'$  对原始地初始路线  $\pi \in \Pi_n$  进行交换，该交换操作可以用  $\Pi_n.swap(\pi, \pi')$  表示。如果该交换操作能够使所有行程的通行时间之和减少至少  $1 + \epsilon$  的比例（e.g.,  $\frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n.swap(\pi, \pi')\})} > (1 + \epsilon)$ ），我

**算法 4-1: InitSearch( $G, q, \mathcal{L}$ )**


---

**输入:** 动态路网  $G = (V, E)$ ;  
 行程请求  $q = (v_s, v_d, t)$ ;  
 所有两点间的通行时间下界  $H(v_i, v_j)$ ;  
 规划路线的路段标签集合  $\mathcal{L}$ 。  
**输出:** 为请求  $q$  制定的初始化路线  $\pi$ 。

```

1 Init:  $\forall v \in V: v.t_s = \infty, v.t_d = H(v, v_d), v.et = t;$   

    $v_s.pred = null, v_s.t_s = 0; \pi = \emptyset;$ 
2  $PQ \leftarrow \{v_s\};$ 
3 while  $PQ \neq \emptyset$  do                                     /* 主搜索循环 */
4    $v \leftarrow PQ.pop();$ 
5   if  $v = v_d$  then                                       /* 到达目的节点 */
6     while  $v_d.pred \neq null$  do
7        $\pi \leftarrow \{v_d, \pi\}; v_d \leftarrow v_d.pred;$ 
8     return  $\pi;$ 
9   for each edge  $e(v, v') \in G$  do                         /* 遍历邻接路段 */
10    计算当前时刻  $v.et$  下路段  $e$  的通行时间  $T(e, v.et)$ ;
11    if  $v.t_s + T(e, v.et) \leq v'.t_s$  then
12       $v'.t_s \leftarrow v.t_s + T(e, v.et); v'.et \leftarrow t + v'.t_s; v'.pred \leftarrow v;$ 
13      if  $v' \notin PQ$  then
14         $PQ.push(v');$ 

```

---

们认为该操作是有效的并采用该操作去更新原有的初始路线。其中，我们采用一个极小的常数值  $\epsilon$  排除一些“无用”的操作，这里无效操作指通过该操作只能减少很小的通行时间。此外， $\epsilon$  的使用还保证了交换操作的次数是有限的。实际上，有效的交换操作的次数是和  $\epsilon$  成反比的。具体来说，最大的有效交换操作的次数是  $\log_{(1+\epsilon)} \frac{TT}{TT_m}$ 。其中  $TT$  是当前规划的初始路线组合的总的通行时间，而  $TT_m$  是理想的最小通行时间。当路线组合  $Q_n$  中不存在有效交换操作，不能再产生一个更好的规划结果，批优化处理算法终止。在本文中，我们也提出了一个预检查策略来进一步提高效率。

#### 4.5.2.1 交换操作

在优化过程中，除了考虑系统外车流量和系统内已经规划的路线产生的车流量外，我们进一步考虑了当前正在规划的初始路线可能对未来交通产生的潜在影响。一个新的路段标签集合  $\mathcal{L}' = \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$  是由原有的路段标签  $\mathcal{L}$  以及预估的正在规划的初始路线的时间信息联合组成的 (cf. 算法 ??)。把  $\mathcal{L}'$  作为已知的交通信息输入，我们再次执行算法??为行程  $q_i$  分配一条新的路线  $\pi'_i$ 。如果该交换操

作被确定是有效的，我们将使用新路线  $\pi'_i$  替换原有的路线  $\pi_i \in \Pi_n$ 。

#### 4.5.2.2 预检查策略

为了检查一个交换操作的有效性，我们必须更新所有路段的路段标签去计算所有行程总的通行时间的降低率，这是非常费时的。因此，我们提出了一个预检查策略进一步提高算法效率。具体来说，我们定义了一个所有行程总的通行时间的降低率上界  $UB$ ，计算如下。

$$UB(\pi) = \frac{TT(\{\Pi, \Pi_n\})}{TT(\{\Pi, \Pi_n \setminus \pi\})} \quad (4-4)$$

注意  $TT(\{\Pi, \Pi_n \setminus \pi\}) \neq TT(\{\Pi, \Pi_n\}) - ET(\pi, t)$ ，在这里  $TT(\{\Pi, \Pi_n \setminus \pi\})$  是将路线  $\pi$  剔除，假设该路线不存在下的所有路线的总的通行时间。我们不仅要剔除路线  $\pi$  本身的通行时间，也要消除路线  $\pi$  对其他行程的影响，因为路线  $\pi$  造成了额外的交通流量，会对其他路线的通行时间造成一定的影响。

---

#### 算法 4-2: BatchRefining( $G, \Pi, \Pi_n, \mathcal{L}, \epsilon$ )

---

**输入:** 动态路网  $G = (V, E)$ ;

全局路线组合  $\Pi$ ;

初始路线组合  $\Pi_n$ ;

路段标签集合  $\mathcal{L}$ ;

参数  $\epsilon$ 。

**输出:** 优化后的路线组合  $\Pi_n$ 。

1 **Init:**  $flag \leftarrow true$ ;

2 **while**  $flag$  **do**

/\* 批量迭代优化 \*/

3      $flag \leftarrow false$ ;

4     **for each** route  $\pi_i \in \Pi_n$  **do**

/\* 遍历当前路线集合 \*/

5         **if**  $UB(\pi_i) > (1 + \epsilon)$  **then**

6              $\mathcal{L}' \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n \setminus \pi_i)\}$ ;

7              $\pi' \leftarrow InitSearch(G, q_i, \mathcal{L}')$ ;

8             **if** 操作  $\Pi_n.swap(\pi_i, \pi')$  是有效的 **then**

/\* 可行替换 \*/

9                  $\Pi_n \leftarrow \Pi_n.swap(\pi_i, \pi')$ ;  $flag \leftarrow true$ ;

---

#### 4.5.2.3 算法描述

算法??提出了批优化处理算法的伪代码。初始地，我们使用一个标志  $flag$  来标记在上一次“while”循环中是否存在有效的操作，算法开始我们设置其为  $true$  (line 1)。在优化过程中，对于每一条初始路线  $\pi \in \Pi_n$ ，我们检查它能否被一条新的路线  $\pi'$  替换 (lines 4–13)。具体来说，我们首先更改标志  $flag$  为  $false$ ，假设在本轮循环中不存在有效的交换操作 (line 3)。对于每一条路线  $\pi \in \Pi_n$ ，我们通过计算提升



上界  $UB(\pi)$  来预检查该交换操作的有效性。如果该交换操作是可能有效的，我们生成一个新的路段标签  $\mathcal{L}'$  (line 6). 将  $\mathcal{L}'$  作为输入，我们执行算法 ?? 产生一条新的路线  $\pi'$  (line 7). 如果该交换操作最终被确定是有效的，我们将采用这次交换操作并设置标志 `flag` 为 `true` 来记录当前循环仍然具有有效的交换操作 (lines 8–11). 当一轮循环，对所有的  $\pi \in \Pi_n$  都没有检测到有效的交换操作，算法终止。也就是说已经不存在能够产生更好结果的有效操作存在。

### 4.5.3 CORC 问题的总体解决方案

通过整合上文提出的所有方法，我们在这里提出针对 CORC 问题的总体解决方案即 SBP 算法。SBP 算法执行流程如下：对于每一个新到达的行程请求  $q$ ，我们先执行初始路线搜索为其分配一条初始个人路线。接着我们执行算法 ?? 周期性的批优化最近规划的初始路线，这些初始路线是为最近一段时间发布的行程请求制定的。一旦有新的行程请求发布，SBP 算法将不断执行上述操作。

#### 4.5.3.1 优化间隔

在我们的设定中，批优化算法是以固定的时间间隔周期性执行的 (e.g., 2s)。我们用  $T$  表示每两次批优化处理之间的时间间隔。通过利用这样一个时间间隔  $T$ ，我们合理考虑了更多的正在规划的初始路线对未来交通的潜在影响，在这种情况下我们能更准确地预测未来的交通状态，从而通过一次交换操作我们也许可以减少更多的通行时间，整个优化过程中总的交换次数也会减少，算法效率也就得到了提高。

---

#### 算法 4-3: SBP( $G, Q, T, \epsilon$ )

---

**输入：** 动态路网  $G = (V, E)$ ;

行程请求流  $Q$ ;

优化时间间隔  $T$ ;

参数  $\epsilon$ 。

**输出：** 实时的最优路线组合  $\Pi$ 。

1 **Init:**  $\mathcal{L} \leftarrow \emptyset$ ;  $\Pi \leftarrow \emptyset$ ;  $\Pi_n \leftarrow \emptyset$ ;

2 **while true do**

/\* 系统持续运行 \*/

3     **for each** trip request  $q \in Q$  **do**

/\* 处理到达的行程请求 \*/

4          $\pi \leftarrow \text{InitSearch}(G, q, \mathcal{L})$ ;  $\Pi_n \leftarrow \{\Pi_n, \pi\}$ ;

5     **if** 当前时间 mod  $T = 0$  **then**

/\* 周期性批量优化 \*/

6          $\Pi_n \leftarrow \text{BatchRefining}(G, \Pi, \Pi_n, \mathcal{L}, \epsilon)$ ;  $\Pi \leftarrow \{\Pi, \Pi_n\}$ ;  $\mathcal{L} \leftarrow \{\mathcal{L}, \mathcal{L}(\Pi_n)\}$ ;

$\Pi_n \leftarrow \emptyset$ ;

7         **Output**  $\Pi$

// 实时输出结果

---

### 4.5.3.2 算法描述

算法 ?? 提出了针对 CORC problem 的总体解决方案，即 SBP 算法的伪代码。初始地，我们的规划系统中没有接收到任何行程请求，我们设置路段标签  $\mathcal{L}$  和全局的路线组合  $\Pi$  都是空集。全局的路线组合  $\Pi$  记录了所有已经规划完毕的行程的路线。我们也使用了一个路线组合  $\Pi_n$  来记录为新的一批行程  $Q_n$  规划的初始路线，初始的  $\Pi_n$  也是空集 (line 1)。首先我们执行算法 ?? 为每一个新到达的行程  $q \in Q$  请求分配一条初始化路线 (lines 3–6)。我们持续地为新到达的请求生成初始路线直到下一次批优化处理 (line 7)。在批优化处理过程中，我们执行算法 ?? 去生成一个优化后的路线组合  $\Pi_n$  (line 8)。接下来，我们将这些优化后的路线加入全局的路线组合  $\Pi$  中，并根据新规划的路线对所有路段的路段标签记录进行更新 (lines 9–10)。每一个优化处理完成后，由于所有的初始路线已经被优化了，我们设置  $\Pi_n$  为空并继续用于记录下一批到达的行程的初始路线 (line 11)。每当我们处理完一批新的行程请求流，我们输出全局最优的路线组合  $\Pi$  作为实时的规划结果 (line 12)。

## 4.6 实验研究

### 4.6.1 实验设定

#### 4.6.1.1 数据集

我们使用两个路网数据集用于我们的实验研究：San Joaquin County Road Network (TG 路网)<sup>①</sup>和 the New York Road Network (NY 路网)<sup>②</sup>。我们用邻接表进行路网图的存储。对于每个路段连边，我们基于该路段的长度分配一个数值为 20–100 间的车容量  $C_e$ 。每个路段的最小通行时间  $T_m(e)$  是随机生成的，位于 5–10 分钟之间。为了模拟非请求车辆，我们假设每个路段  $e$  上的非请求车辆的平均车流量  $\bar{x}$  为  $0.4 \times C_e$ ，实时的非请求车辆在平均车流量附近周期性地动态变化  $0.8\bar{x}$  到  $1.2\bar{x}$ 。给定最小的非请求车流量  $0.8\bar{x}$  作为所有路段的静态车流量，我们提前计算好所有顶点两两间的通行时间下界，该下界即算法 ?? 中的启发值  $H(v_i, v_j)$ 。在路网较为密集的区域，我们随机采样起点-终点对来模拟行程请求。为了模拟持续到达的行程请求流，给定第一个行程的出发时间  $q_1.t$ ，接下来的第  $i$  个行程的出发时间  $q_i.t$  在上一个行程请求的出发时间  $q_{i-1}.t$  的基础上小幅度增加或保持不变。行程请求到达的速率设定见表 ??。

① <https://www.cs.utah.edu/lifeifei/SpatialDataset.htm>

② <https://publish.illinois.edu/dbwork/open-data>

|                     | TG 路网                         | NY 路网                         |
|---------------------|-------------------------------|-------------------------------|
| 行程请求数 $ Q $         | 4,000-20,000   default 4,000  | 2,000-10,000   default 2,000  |
| 参数 $\epsilon$       | 0.0002-0.0010   default 0.001 | 0.0002-0.0010   default 0.001 |
| 优化间隔 $T$            | 1s-5s   default 2s            | 1s-5s   default 2s            |
| 行程到达速率              | 20/s-100/s   default 50/s     | 20/s-100/s   default 50/s     |
| $\alpha$ (cf. 式 ??) | 1-5   default 2               | 1-5   default 2               |
| $\beta$ (cf. 式 ??)  | 1-3   default 2               | 1-3   default 2               |

表 4-1 参数设定

#### 4.6.1.2 车流量的计算

在我们的实验设定中，在时刻  $t$  路段  $e$  实时的车流量  $f(e, t)$  是由式 ?? 计算的。

$$f(e, t) = f'(e, t) + f''(e, t) \quad (4-5)$$

其中， $f'(e, t)$  代表系统外的非请求车辆的车流量，而  $f''(e, t)$  代表系统内请求车辆的车流量。为了模拟非请求车辆，我们假设  $f'(e, t)$  是随时间周期性动态变化的 (e.g.,  $f'(e, t + T) = f'(e, t)$ ,  $T = 2h$ )。如何计算系统内请求车辆的车流量我们已经在算法??中给出。

#### 4.6.1.3 算法评估设定

我们主要评估了自感应批处理算法 (SBP 算法) 的算法表现。为了评估该算法的结果质量，我们额外实施了一个基于个人的搜索算法 (Ind 算法)。具体来说，给定一组行程请求流  $Q$ ，对于每一个行程请求  $q \in Q$ ，Ind 算法基于单个行程的出发时刻的交通状态生成一条最优的个人最优路线，这和一些已经存在的研究相符 [??]。由于精确算法十分耗时，在默认设定下需要至少一天的时间，因此我们不研究精确算法的表现。我们知道 SBP 算法由两部分组成：初始路线搜索和批优化处理。我们提出的初始路线搜索合理考虑了自感应的思想和网络的动态性，为了验证其优越性，我们分别又实施了 SBP\*SA 算法和 SBP\*DA 算法。其中，SBP\*SA 算法是初始路线搜索没有采用自感知思想的 SBP 算法，即它没有考虑由我们规划系统规划的路线随交通流量造成的额外影响。SBP\*DA 算法是没有考虑网络动态性的 SBP 算法，它在初始路线搜索过程中面向的是一个静态的交通状态快照即一个行程的出发时刻所对应的静态交通状态。我们使用 CPU 运行时间和所有行程总的通行时间  $TT(\Pi)$  作为算法效率和算法准确性的评估指标。除非特别申明，我们的实验结果是超过 20 次独立实验的平均值结果。所有的算法都是在 windows10 平台上运行的，采用 Java 编程，使用 Intel(R) Core(TM) i5-9300H Processor (2.40 GHz)

的处理器和 16GB 的内存。具体的默认参数设定见表??。

## 4.6.2 实验结果

### 4.6.2.1 行程请求数目对算法效果的影响

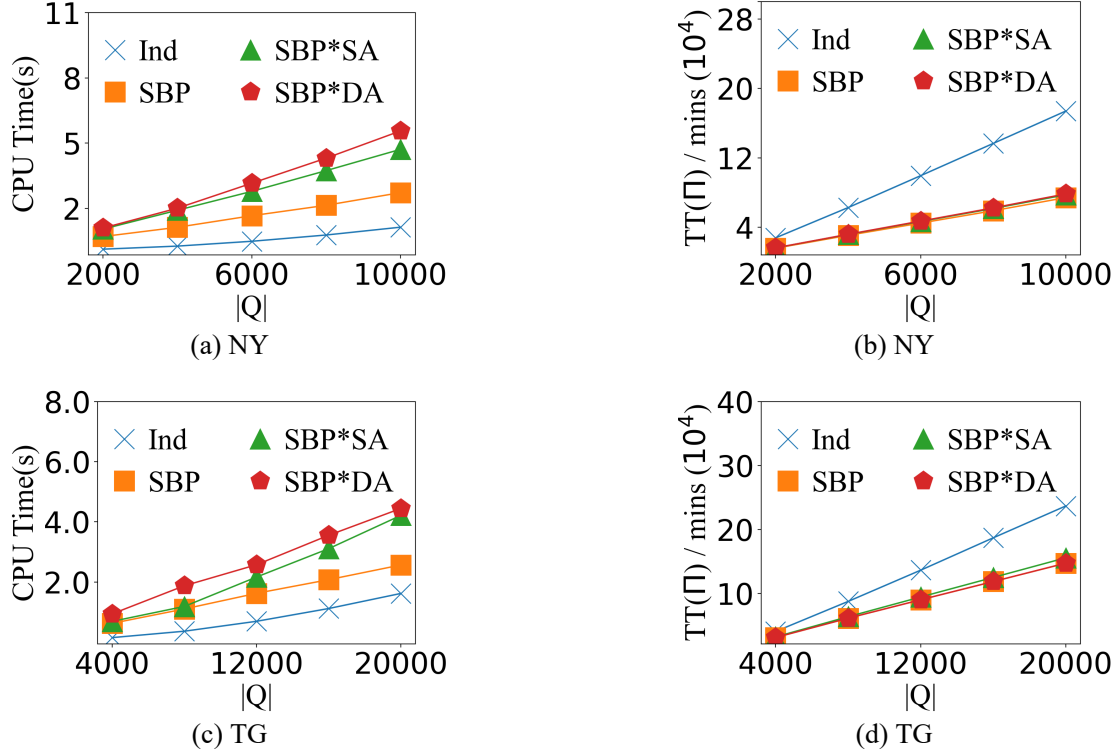


图 4-2 行程请求数目  $|Q|$  的影响

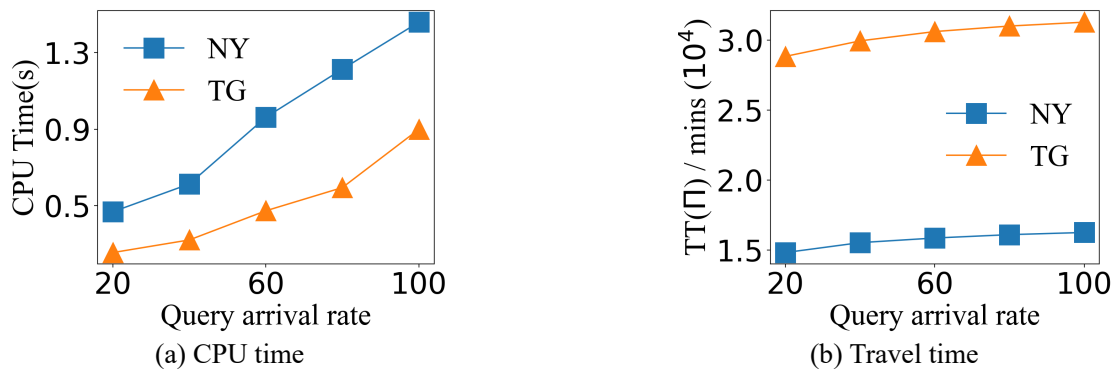


图 4-3 行程请求到达速率的影响

首先，在默认的参数设定下，我们调查了提出的算法在不同行程请求数目  $|Q|$  下的算法表现。直觉上，行程请求数目  $|Q|$  越大，所有行程路线的总的通行时间会增加。此外，一个更大的  $|Q|$  也会造成更大的计算消耗，CPU 运行时间也会增加。图??展示了我们提出的算法在 TG 路网和 NY 路网上的表现。意料之中的，随着行

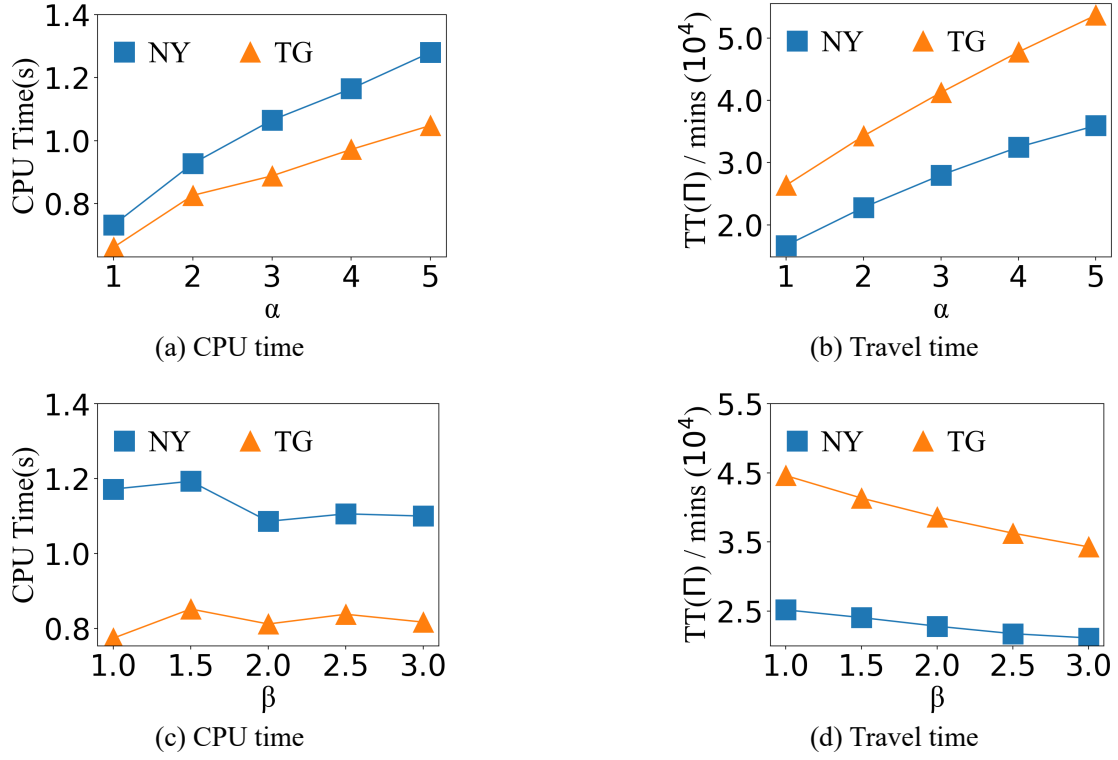


图 4-4 参数  $\alpha$  &  $\beta$  的影响

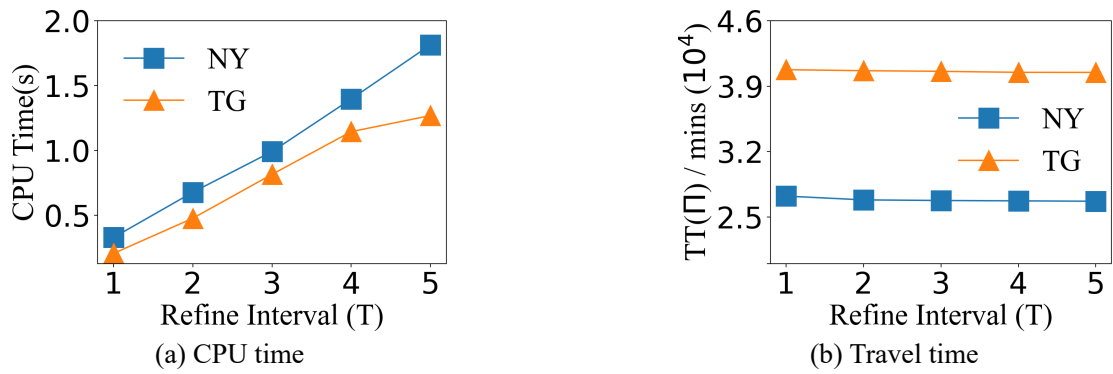


图 4-5 批优化时间间隔  $T$  的影响

程请求数  $|Q|$  的增加, 所有算法所需要的 CPU 时间都增加了。在这些算法中, 基于个人的 Ind 算法需要的 CPU 时间最小, 因为它是一次规划的, 没有经过我们所提出的批优化处理这一过程。我们也同时发现 Ind 算法产生的全局路线组合中所有路线总的通行时间  $TT(\Pi)$  比其他算法更大, 特别是在行程请求数目较大的情况下 (e.g., 当在 NY 路网中  $|Q| = 10,000$  和在 TG 路网中  $|Q| = 20,000$  时, 和 Ind 算法比较, 我们的 SBP 算法能够减少超过 40% 的通行时间)。

和 SBP 算法相比, SBP\*SA 和 SBP\*DA 表现较差。因为它们会在优化过程中进行较多次数的交换操作, 这是非常耗时的。具体来说, SBP\*SA 没有考虑到已经规划的路线也会对路段交通增加额外的车流量, 因此它在初始化路线搜索阶段进行的交通预测是不可靠的。对于 SBP\*DA 算法, 它以一个行程出发时刻的交通状态快照进行路径规划。因此, SBP\*SA 算法和 SBP\*DA 算法所指定的初始化路线往往是低质量的, 需要在批优化过程中采取大量的交换操作来对这些低质量的初始路线进行调整, 从而产生了更多的计算消耗去提高结果质量。从图??和图??我们也观察到这三种算法产生的全局路线组合的所有路线的总的通行时间是非常接近的, 而且我们的 SBP 算法总能产生较小的通行时间  $TT(\Pi)$ 。这些实验结果证实了通过采用自感知的思想和考虑到网络动态性, 我们的初始路线搜索能够有效地产生高质量的处理路线, 这样的初始路线可以减少后续优化处理过程中的交换操作次数。

#### 4.6.2.2 行程请求到达速率对算法效果的影响

图??展示了当行程请求到达速率发生改变时 SBP 算法的算法表现。行程请求到达速率越大, 说明单位时间会有更多的车流涌入到路网的各个路段上。在这种情况下, 每个路段车流量的增幅会更加明显。从而, 每个路段上会产生更多的额外车流量。在图??, 观察 CPU 时间和总的通行时间  $TT(\Pi)$ , 不管是在 NY 路网还是在 TG 路网上, 我们都可以发现它们有明显的增长趋势。 $TT(\Pi)$  增长是因为随着车流量增加, 每个路段的通行时间也相应增加了。在 TG 路网上, 所有的行程规划的路线的总的通行时间远远大于在 NY 路网上的, 这是因为在我们的默认设定中, TG 路网会有更多数目的行程请求产生。

#### 4.6.2.3 通行时间函数各参数对算法效果的影响

图??展示了当我们变化通行时间函数中的参数  $\alpha$  和  $\beta$  (cf. 式 ??) 时 SBP 算法的算法表现。直觉上,  $\alpha$  越大或者  $\beta$  越小, 各路段通行时间会增大, 从而可能会加剧交通拥堵。在图??中, 当  $\alpha$  从 1 变化到 5 时, CPU 时间和所有行程总的通行时间都有增加的趋势。相反的, 当  $\beta$  从 1 变化到 3 时, 总的通行时间具有下降的趋

势。值得注意的是，当我们变化  $\beta$  时，CPU 时间没有特别明显的变化趋势，这是因为 CPU 时间即我们所需要的计算消耗并不是和  $\beta$  的值直接相关的。

#### 4.6.2.4 优化时间间隔对算法表现的影响

图??展示了当优化时间间隔  $T$  变化时 SBP 算法的表现。 $T$  越大代表着每个请求的等待时间增加了，但是规划的结果会更趋于最优，这是因为有更多的正在规划的路线被考虑到了对未来的交通状态影响中去了。在这种情况下，我们通过交换操作大大减少所有行程总的通行时间  $TT(\Pi)$ 。在图??，CPU 时间随着  $T$  的增加而增加，这是因为我们在一次优化过程中需要处理更多的行程请求。在图??中，我们可以发现  $TT(\Pi)$  有一个轻微的减小趋势。这些结果说明了一个合适的优化时间间隔能够通过重新利用已经规划的初始路线来更准确地未来的交通状态，从而避免交通拥堵。

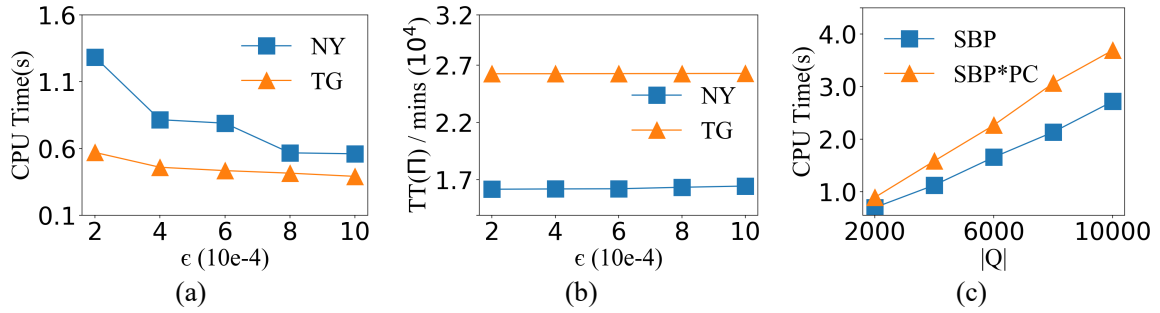


图 4-6 参数  $\epsilon$  和预检查策略的影响 ?? CPU 运行时间；?? 通行时间；?? 预先检查策略的影响评估

#### 4.6.2.5 参数 $\epsilon$ 和预检查策略对算法表现的影响

我们研究了当参数  $\epsilon$  从 0.0002 变化到 0.001 时的 SBP 算法表现。我们知道在优化处理过程中，总的有效交换操作的次数是和参数  $\epsilon$  的值成反比的 (cf. 算法 ??)。  $\epsilon$  越大，意味着有效操作的门槛越高，有效操作的次数越少，因此 CPU 时间也会随之减少。然而，一个更大的  $\epsilon$  也会产生具有更大通行时间的全局路线组合  $\Pi$ 。在图??中，当参数  $\epsilon$  从 0.0002 变化到 0.001 时，CPU 时间有一个减少的趋势。相反的，所有行程总的通行时间有一个轻微的增加的趋势。当  $\epsilon$  从 0.0002 变化到 0.0004 时，所需要的 CPU 时间降幅最大 (e.g., 在 NY 路网和 TG 路网中，需要的 CPU 时间减少幅度分别超过了 36% 和 19%)。我们也发现对于  $\epsilon$  值的选取，0.001 是非常合适的，从图中我们可以看出当  $\epsilon$  为 0.001 时，算法的综合表现是最好的。以上实验结果说明通过利用一个合适的参数  $\epsilon$ ，SBP 算法的表现能够得到较大的提升。我们用“SBP\*PC”来表示没有应用预检查策略的 SBP 算法，在图??中，我们观察到 SBP

算法所需要的 CPU 运行时间总是比 SBP\*PC 算法更少，这个实验结果说明了我们提出的预检查策略是能够进一步提升算法效率的。

## 4.7 本章小结

我们提出和调查了一个新颖的路线规划问题（CORC 问题），该问题旨在为持续的形成请求搜寻最佳的路线组合。我们提出了一个 SBP 算法来高效解决 CORC 问题。同时，一些修剪枝技术的提出也大大提升了算法效率。对应的实验研究证实了我们提出的算法能够有效地解决该问题，也证实了我们提出的剪枝技术有助于避免不必要的计算。



## 第五章 联合移动模式挖掘

### 5.1 引言

从轨迹中检测共同行为（co-movement）模式的问题已经被数据科学领域广泛研究。现有方法主要基于“过滤-精炼”（filter-and-refine）框架、索引技术、位置点聚类以及搜索算法。尽管这些技术已被广泛应用，但在处理具有复杂群体定义的模式时，往往需要巨大的计算开销，使其难以在大规模轨迹数据上产生及时的结果。在现实应用场景中，尤其是在疫情预警与防控中，实时且高质量的结果尤为重要。

基于此，我们对共同行为模式的定义进行了简化与放宽，在不失一般性的前提下支持快速检测。具体而言，我们通过**伴随群体**（companion group）来定义共同行为模式，该群体由至少  $m$  个在连续  $k$  个时间段内共同移动的对象组成。如果某个对象能够与其他对象形成多个伴随群体，则该对象归属于成员数量最大的伴随群体。给定一组移动对象轨迹，精确算法通过组合满足条件的共同行为对来发现伴随群体，但该过程计算开销较大。为实现高效发现，我们提出了一种快速的、具备社区感知能力的近似算法。基于两个真实世界数据集的大量实验验证了所提出方法的有效性与高效性。

#### 5.1.1 研究背景与意义

随着位置追踪设备（如车载导航系统和智能手机）以及 GPS 使能服务（如 Google Maps、Uber 和 Grab）的持续普及，轨迹数据的规模正呈现爆炸式增长。作为轨迹数据分析中的一项基础功能，共同行为模式（co-movement pattern）挖掘问题已经受到数据科学领域的广泛关注。一般而言，共同行为模式表示一组在一段时间内共同移动的对象。该类模式的典型表示形式包括 flock [? ?]、convoy [? ?]、swarm [? ?]、group [? ?]、platoon [? ?] 以及 gathering [? ?] 等。

高效且准确地从移动对象轨迹中发现共同行为模式在诸多实际应用中具有重要意义，包括拼车推荐、交通拥堵缓解（即通过及早发现道路上的高密度群体来规避潜在拥堵）、COVID-19 等传染病的暴露与风险分析 [? ?]、移动行为分析以及异常轨迹检测 [? ?] 等。

现有研究大多基于移动对象随时间演化的轨迹设计搜索算法。然而，这类算法通常具有较高的计算代价，尤其是在共同行为模式（如 flock、swarm、group）不稳定的情况下（即对象可能在时间上暂时离开群体）。为了挖掘目标共同行为模式，

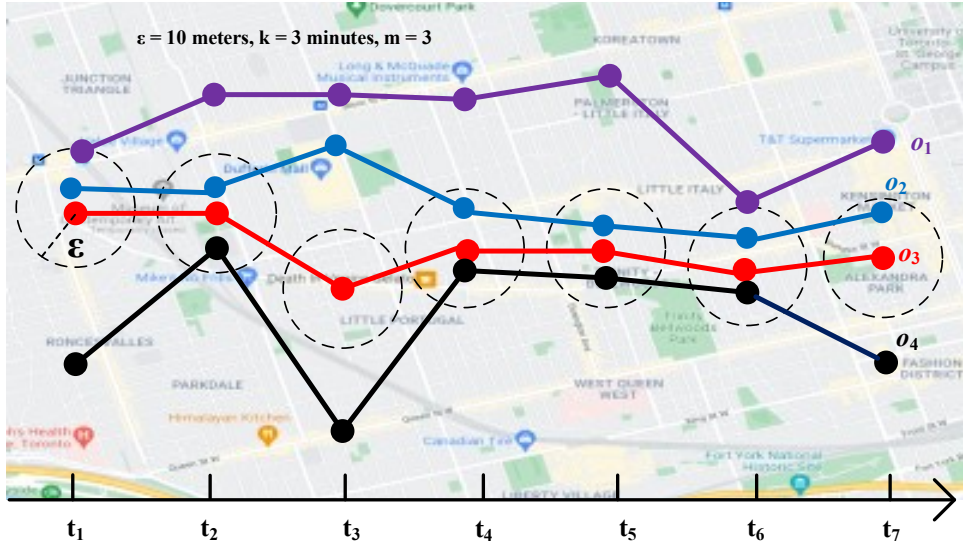


图 5-1 伴随群体示例

现有方法往往需要在每一个时间快照上计算对象聚类，这在实时应用场景中具有极高的计算开销。此外，其基于“过滤-精炼”的处理框架往往会产生大量候选结果，从而进一步加重计算负担。随着轨迹数量的增加，这些方法的性能会显著下降。

在现实应用中，尤其是在疫情预警与防控场景下，及时获得最新的共同行为结果至关重要。因此，共同行为挖掘任务需要以连续的方式执行。这对算法在效率和效果两方面都提出了更高要求。然而，现有的群体模式定义及其相关方法在同时兼顾有效性与高效性方面仍然存在明显不足。

为了解决上述问题，本文提出了一种新的共同行为模式概念——*伴随群体* (companion group)，其定义相对更加宽松。具体而言，一个伴随群体由至少  $m$  个移动对象组成，这些对象在至少  $k$  个时间段内共同移动（例如，其两两距离不超过预定义的距离阈值  $\epsilon$ ），且允许对象在时间上暂时离开其伴随群体。基于伴随群体，我们进一步定义了*伴随群体发现* (Companion Group Discovery, CGD) 问题。给定一组移动对象及其轨迹，CGD 问题旨在在用户可交互时间内发现所有伴随群体。在我们的设定中，若某一对象能够与其他对象形成多个伴随群体，则该对象归属于成员规模最大的伴随群体。CGD 问题可广泛应用于交通监测与管理、疫情防控以及暴力事件预防等实际场景。

下面通过图 ?? 给出一个示例说明。设  $o_1, o_2, o_3, o_4$  为四个移动对象，且所有对象每分钟上报一次位置信息。图 ?? 展示了它们在时间区间  $[t_1, t_7]$  内的轨迹。设距离阈值  $\epsilon = 10$  米，共同行为持续时间阈值  $k = 3$  分钟，群体规模阈值  $m = 3$ 。可以观察到，对象  $o_2$  与  $o_3$  在时间集合  $T = \{t_1, t_2, t_4, t_5, t_6, t_7\}$  内始终共同移动，且

$|T| = 6$  超过持续时间阈值  $k$ ，因此二者可构成一个共同行为对。同样， $o_3$  与  $o_4$  也可以构成一个共同行为对。此外，对象  $o_2, o_3, o_4$  在时间点  $t_2, t_4, t_5, t_6$  内的两两距离均不超过  $\epsilon$ ，因此我们将  $g = \{o_2, o_3, o_4\}$  视为一个伴随群体。需要注意的是，对象  $o_1$  仅在  $t_1$  和  $t_6$  与  $o_2$  短暂接近，无法与其他对象形成共同行为对或伴随群体。

针对 CGD 问题，我们首先设计了一种精确搜索算法，即 Exact Search with Exact Co-Movement Computation (ES-ECMC)，该算法能够获得精确结果，但计算代价较高。为进一步提升效率，我们提出了一种社区感知的近似搜索算法，即 Community-aware Search with Approximate Co-Movement Computation (CS-ACMC)，在兼顾结果质量与严格效率需求的前提下解决 CGD 问题。

CS-ACMC 算法主要包括两个阶段：近似共同行为对搜索与社区感知群体发现。在近似共同行为对搜索阶段，首先采用网格划分方法（如 [??]）将每个轨迹位置映射为对应的网格单元编号（即 **token**），从而将每条轨迹表示为一个 **token** 序列；随后，通过计算轨迹之间的公共 **token** 来生成近似共同行为对。在群体发现阶段，我们利用 SCAN 算法 [?] 对共同行为对进行高效聚类。本文算法的实现已开源发布<sup>①</sup>。

本文的主要贡献总结如下：

- 提出了伴随群体这一新的共同行为模式定义，该定义更加宽松，适用于需要实时结果更新的多种应用场景，如传染病（SARS、埃博拉、COVID-19 等）传播监测与防控。
- 设计了一种精确算法以及一种新颖的社区感知聚类模型，用于解决 CGD 问题。
- 基于两个真实世界数据集进行了大量实验评估，实验结果表明，所提出的社区感知算法能够在用户可交互时间内从大规模轨迹数据中发现伴随群体，体现了其在实际应用中的重要价值。

本文其余部分组织如下：第 ?? 节介绍问题定义与相关预备知识；第 ?? 节给出精确算法；第 ?? 节介绍社区感知的高效近似算法；第 ?? 节展示实验结果与分析；第 ?? 节回顾相关研究工作；第 ?? 节给出总结。

<sup>①</sup> [https://github.com/Like-China/gathering\\_mining.git](https://github.com/Like-China/gathering_mining.git)

## 5.1.2 研究内容与挑战

## 5.1.3 研究成果与贡献

## 5.2 相关工作

大规模轨迹数据的管理、挖掘与分析问题已受到数据科学领域的广泛关注。该方向的代表性研究主要包括轨迹搜索与推荐 [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]、位置搜索与推荐 [11, 12, 13, 14, 15]、路径规划与推荐 [16, 17, 18, 19, 20]，以及区域查询与交通模式挖掘等其他轨迹分析任务 [21, 22, 23, 24]。在上述研究中，共同行为模式搜索与轨迹聚类问题与本文所研究的群体模式发现问题关系最为密切。

**5.2.0.0.1 共同行为模式搜索** 现有研究已经提出并讨论了多种有意义的共同行为模式 [25, 26, 27, 28]，并在不同应用场景下得到了广泛应用。具体而言，flock [29] 是最早提出的共同行为模式，用于刻画在  $k$  个连续时间间隔内共同移动的  $m$  个对象组成的群体。后续研究 [30] 指出，flock 所采用的圆形约束难以准确刻画真实移动对象群体的形态，并提出了支持任意形状聚类的 convoy 模式。然而，flock 与 convoy 均对时间连续性提出了较为严格的约束。为支持对模式持续时间要求更加宽松的应用场景，研究者进一步提出了多种共同行为模式 [31, 32, 33]。

在这些研究中，Zheng 等人 [34] 提出了 gathering 模式，用于交通拥堵的早期发现，该模式允许成员在任意时间离开群体。Fan 等人 [35] 提出了更加通用的共同行为定义，并设计了并行框架用于模式发现。类似地，Naserian 等人 [36] 在轨迹数据流环境下提出了一种宽松的伴随群体发现方法。需要指出的是，Shein 等人 [37] 揭示了上述方法所需的高计算代价，并提出了一种增量式算法，但该方法在效率上仍然不足，难以支持实时发现需求。

值得注意的是，一些方法（如 [38, 39]）能够处理轨迹数据流，而另一些方法则仅适用于静态轨迹数据（如 [40, 41]）。与本文最为相关的研究之一是 Chen 等人 [42]，其提出了一种实时共同行为模式挖掘方法，定义了一种更加通用的共同行为模式，并通过  $L$ -连续性和  $G$ -连通性来刻画轨迹之间的连续性与连通性。然而，由于问题定义的不同，该方法无法直接应用于本文的问题设置：本文不要求时间上的连续性与连通性，也不要求轨迹在每个时间点均属于同一聚类；此外，当一个对象能够形成多个群体时，本文始终选择成员规模最大的群体。

为处理轨迹数据流，Fang 等人 [43] 提出了一个实时共同行为模式挖掘系统，用于在特定时空约束下实时发现共同行为对象；Chen 等人 [44] 进一步研究了分布式环境下的实时共同行为模式检测，并通过基于双层索引的范围连接操作来加速

聚类过程、减少不必要的验证。

尽管近年来已有大量研究尝试利用深度学习技术加速轨迹挖掘任务，包括轨迹聚类、异常轨迹检测 [?]、轨迹相似性计算 [?] 以及风险检测 [?] 等，但据我们所知，尚无基于深度学习的方法能够直接用于解决本文所研究的问题。

**5.2.0.0.2 轨迹聚类** 该方向的大多数研究 [??] 采用深度表示学习方法来加速轨迹相似性计算，其核心思想是将变长轨迹转换为固定长度的表示向量。Yao 等人 [?] 通过滑动窗口提取轨迹特征序列，并利用自编码器学习固定长度的深度表示；Boyle 等人 [?] 使用训练后自编码器最小隐藏层的输出作为轨迹表示；Fang 等人 [?] 提出了一种基于自训练的端到端聚类框架，无需人工特征提取。

除轨迹聚类外，通用的基于位置的群体搜索问题也得到了广泛研究 [?]，例如基于位置的群体查询处理 [?]、基于位置的连续查询处理 [?]、局部高频模式发现 [?]、局部事件探索 [?]、位置感知传播模式挖掘 [?] 以及邻域模式挖掘 [?] 等。

然而，上述针对轨迹聚类提出的解决方案均无法直接应用于本文的问题，因为处于同一聚类中的移动对象并不一定在时空上真实地共同移动。

## 5.3 问题定义

本节介绍本文所需的基础概念，并给出问题的形式化定义。给定一组移动对象，我们依次定义轨迹、协同行为对、伴随群体以及伴随群体发现问题。

**5.3.0.0.1 移动对象及其轨迹** 一个移动对象  $o$  由其轨迹  $\tau = \{s_1, \dots, s_n\}$  表示，其中轨迹是一个有限的、按时间顺序排列的离散时空点序列。该序列由位置追踪设备（如 GPS）从对象  $o$  的真实运动轨迹中采样获得。

为简化描述，我们使用  $s_i = ([x, y], t)$  表示二维空间中的一个离散时空点，其中  $[x, y]$  表示空间坐标， $t$  表示该位置被记录时对应的时间戳。对于一组移动对象  $O$ ，我们用  $\mathcal{T} = \{\tau_1, \dots, \tau_{|O|}\}$  表示其轨迹集合，其中  $\tau_i$  为对象  $o_i$  的轨迹。需要说明的是，本文对移动对象及轨迹的建模方式与已有研究保持一致 [??]。

**5.3.0.0.2 协同行为持续时间** 设  $o_i$  与  $o_j$  为两个移动对象。当  $o_i$  与  $o_j$  之间的空间距离小于给定的空间距离阈值  $\epsilon_s$ ，且时间距离小于给定的时间阈值  $\epsilon_t$  时，它们被认为处于协同行为状态。 $o_i$  与  $o_j$  的协同行为持续时间定义为满足上述条件的总行程时间长度。

**5.3.0.0.3 协同行为对** 设  $\tau_i$  与  $\tau_j$  分别为对象  $o_i$  与  $o_j$  的轨迹。若  $\tau_i$  与  $\tau_j$  之间的协同行为持续时间不少于阈值  $k$ （其中  $k$  为协同行为持续时间阈值），则称对象  $o_i$  与  $o_j$  构成一个协同行为对。

**定义 5.1（伴随群体）** 给定一组移动对象  $\mathcal{O} = \{o_1, o_2, \dots, o_{|\mathcal{O}|}\}$  以及最小群体规模阈值  $m$ ，若对象子集  $g \subseteq \mathcal{O}$  满足以下两个条件，则称  $g$  为一个伴随群体：(1)  $|g| \geq m$ ；(2) 对任意两个对象  $o_i, o_j \in g$ ， $o_i$  与  $o_j$  均能够构成协同行为对。

**5.3.0.0.4 伴随群体发现 (CGD) 问题** 给定一组移动对象  $\mathcal{O}$ ，伴随群体发现 (Companion Group Discovery, CGD) 问题旨在从  $\mathcal{O}$  中发现一组伴随群体  $\mathcal{G}$ ，并满足对任意两个群体  $g_i, g_j \in \mathcal{G}$ ，均有  $g_i \cap g_j = \emptyset$ 。

## 5.4 精确共同行为计算的精确搜索算法 (ES-ECMC)

本节提出一种用于发现伴随群体的精确解法：精确搜索并进行精确共同行为计算的算法 (Exact Search with Exact Co-Movement Computation, 简称 ES-ECMC)。

ES-ECMC 算法主要包含三个步骤。首先，从输入数据（即一组对象轨迹）中枚举并发现所有共同行为对；其次，基于这些共同行为对，以增量方式生成伴随群体候选；最后，从候选集合中选取成员规模最大的伴随群体作为最终结果。在介绍 ES-ECMC 算法之前，我们首先给出伴随群体候选的定义。

**5.4.0.0.1 伴随群体候选** 一个伴随群体候选  $g'$  是一个对象集合，满足其中任意两个对象都可以形成一个共同行为对。若存在至少一个对象  $o \notin g'$ ，使得  $g'' = \{g', o\}$  仍然是一个伴随群体候选，则称  $g'$  可以通过加入对象  $o$  进行扩展；否则，称  $g'$  不可扩展。

下面将详细介绍 ES-ECMC 算法。算法 ?? 给出了 ES-ECMC 的伪代码。算法输入包括一组轨迹  $\mathcal{T}$ 、空间距离阈值  $\epsilon_s$ 、时间距离阈值  $\epsilon_t$ 、群体规模阈值  $m$  以及共同行为持续时间阈值  $k$ ；输出为一组伴随群体  $G = \{g_1, g_2, \dots, g_{|G|}\}$ （其中  $g_i \subseteq \mathcal{T}$ ）。我们使用  $P$  和  $G_c$  分别存储所有共同行为对和伴随群体候选。

**步骤 1:**（枚举共同行为对）对于任意一对移动对象  $o_i, o_j \in \mathcal{O}$ ，我们计算其轨迹中任意两个采样位置  $s \in \tau_i$  与  $s' \in \tau_j$  之间的精确距离。若空间距离与时间距离分别不超过阈值  $\epsilon_s$  与  $\epsilon_t$ ，则将其共同行为持续时间  $|\tau_i \& \tau_j|$  加 1。具体而言，对于每一对对象  $\langle o_i, o_j \rangle$ ，当其累计的共同行为持续时间不少于  $k$  时，我们将其视为一个共同行为对<sup>②</sup>。当所有对象对均被评估后，该步骤结束（第 2–8 行）。

**步骤 2:**（生成伴随群体候选）在获得所有共同行为对之后，我们基于这些共

② 在上下文清晰的情况下，我们使用  $(o_i, o_j)$  或  $(\tau_i, \tau_j)$  表示一个共同行为对。

同行为对以增量方式生成伴随群体候选。具体而言，首先选择一个以轨迹  $\tau_i$  为起点的共同行为对，例如  $\langle \tau_i, \tau_j \rangle$ ，然后考察所有与其共享对象  $o_i$  的共同行为对  $\langle \tau_i, \tau_k \rangle$ 。初始群体记为  $g_i = (\tau_i, \tau_j)$ 。若  $\tau_k$  与  $g_i$  中的所有成员均构成共同行为对，则将  $\tau_k$  加入  $g_i$ 。当  $g_i$  仍可扩展时，重复上述扩展过程。若  $g_i$  的成员规模不少于  $m$ ，则将其加入候选集合  $G_c$ 。随后选择新的共同行为对作为初始群体并继续扩展。当所有共同行为对均被访问作为初始群体后，该步骤结束（第 9–19 行）。

**步骤 3:**（选择伴随群体）在获得所有候选之后，从中选取成员规模最大的伴随群体作为最终结果。具体而言，若某个对象属于多个候选群体，则选择其中成员规模最大的一个。该步骤在遍历完所有候选后结束（第 20 行）。

我们再次考虑图 ?? 中的示例。假设首先选择共同行为对  $\langle o_2, o_3 \rangle$  进行扩展。显然，对  $\langle o_2, o_4 \rangle$  与  $\langle o_2, o_3 \rangle$  共享对象  $o_2$ ，且对象  $o_2, o_3, o_4$  两两之间均可形成共同行为对。因此，我们将  $g' = \{o_2, o_3, o_4\}$  视为一个伴随群体候选。由于对象  $o_1$  无法与  $g'$  中的任一对象形成共同行为对， $g'$  不可进一步扩展。接着，由于  $|g'|$  不小于预设的群体规模阈值 3，我们将  $g'$  加入候选集合  $G_c$ 。最终，由于  $g'$  具有最大的成员规模，我们从  $G_c$  中选取  $g'$  并将其加入结果集合  $G$ 。

**定理 5.1** ES-ECMC 算法的时间复杂度为  $O(|\mathcal{T}|^2 \times |\tau|^2)$ ，其中  $|\mathcal{T}|$  表示移动对象（轨迹）的数量， $|\tau|$  表示单条轨迹的长度（即采样位置数）。

**证明:** ES-ECMC 在枚举所有共同行为对时，需要对每一对轨迹计算其位置之间的距离，其时间复杂度为  $O(|\mathcal{T}|^2 \times |\tau|^2)$ 。对于共享同一对象的共同行为对（例如  $\langle o_i, o_j \rangle$  与  $\langle o_i, o_k \rangle$ ），需要进一步判断其是否能够构成伴随群体候选，并在满足条件时持续扩展该候选。生成所有伴随群体候选在最少情况下需要  $O(|\mathcal{T}|)$  时间（例如不存在任何共享对象的共同行为对）；在最坏情况下，所有对象均可形成共享同一对象的共同行为对，此时生成候选的时间复杂度为  $O(|\mathcal{T}|^2)$ 。最后，从候选集合中生成最终的伴随群体需要  $O(|G_c|)$  时间，其中  $|G_c| < |\mathcal{T}|$ 。综上所述，ES-ECMC 算法的总体时间复杂度为  $O(|\mathcal{T}|^2 \times |\tau|^2)$ 。 ■

## 5.5 基于近似协同行为计算的社区感知搜索（CS-ACMC）

尽管 ES-ECMC 算法直观且结果精确，但其高昂的计算开销使其难以应用于实际场景中，尤其是在移动对象数量极大的情况下。其主要计算瓶颈来源于通过精确计算协同行为持续时间来枚举所有协同行为对。为了解决效率问题，我们提出了一种基于社区感知的搜索算法，结合近似协同行为计算，即 Community-aware Search with Approximate Co-Movement Computation（CS-ACMC）。

CS-ACMC 算法由两个阶段组成：近似协同行为对搜索，以及基于社区感知的

聚类以获取伴随群体。下面我们将依次介绍轨迹标记序列、近似协同行为对搜索方法以及社区感知聚类算法。

### 5.5.1 近似协同行为对搜索

在该阶段中，我们提出了一种高效的方法，用于计算两个对象轨迹之间协同行为持续时间的近似值。生成的近似协同行为对将作为后续社区感知聚类的输入，以高效地发现伴随群体。

**步骤 1:** (生成轨迹标记序列)

该方法受到群体 (flock) 圆形结构的启发 [?]。我们将底层空间划分为大小相等的网格单元。具体而言，将二维欧氏空间划分为空间等距的网格单元，同时将时间维度划分为等长的时间片。

因此，对象轨迹中的每一个数据点 (位置)  $s = ([x, y], t)$  都可以表示为一个标记 (token)，记为  $o(s) = f(x, y, t)$ ，其中  $f(\cdot)$  为一个单射函数。由此可以得到轨迹的标记序列：

$$O(\tau) = \{o(s_1), \dots, o(s_{|\tau|})\}.$$

图 ?? 展示了空间与时间维度的划分方式以及轨迹标记序列的生成过程。假设空间被划分为  $5 \times 4$  个网格单元，每个单元边长为 10 米，时间维度被划分为 3 个时间片，每个时间片长度为 10 秒。给定两条轨迹  $\tau_1$  和  $\tau_2$ ，其位置均落入单元  $\{([0, 0], 0), ([3, 1], 1), ([2, 1], 2)\}$ 。

基于该划分方式，我们定义单射函数

$$o(s) = f(x, y, t) = t \times (5 \times 4) + y \times 5 + x,$$

其中  $x \in \{0, 1, 2, 3, 4\}$ ， $y \in \{0, 1, 2, 3\}$ ， $t \in \{0, 1, 2\}$ 。例如，对于位置  $([3, 1], 1)$ ，其标记值为

$$1 \times (5 \times 4) + 1 \times 5 + 3 = 28.$$

如图 ?? 所示，尽管  $\tau_1$  与  $\tau_2$  在每个时间点的具体位置不同，但由于它们落入相同的空间与时间单元，因此可被表示为相同的标记序列  $\{0, 28, 47\}$ 。

**定理 5.2** 若将空间划分为边长为  $\epsilon$  的等大小网格单元，其中  $\epsilon$  等于协同行为距离阈值，则若两个对象  $o_i$  与  $o_j$  的轨迹标记序列共享  $p$  个相同标记，说明它们的协同行为持续时间至少为  $p$  个时间段。

**证明：**位置  $s$  的标记值由单射函数  $o(s) = f(x, y, t)$  计算得到。两个位置具有相同标记值意味着它们位于同一个空间网格单元中，因此二者之间的空间距离不超过  $\epsilon$ ，同时记录于同一时间片内。若两条轨迹标记序列共享一个标记，则表示对



**算法 5-1: ES-ECMC Algorithm**


---

**输入:** Trajectory set  $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{|\mathcal{O}|}\}$ ;  
 Spatial distance threshold  $\epsilon_s$ ;  
 Temporal distance threshold  $\epsilon_t$ ;  
 Minimum group size threshold  $m$ ;  
 Co-movement duration threshold  $k$ .

**输出:** A set of approximate companion groups  $G$ .

```

1  $P \leftarrow \emptyset$ ; // Set of valid trajectory pairs  $G_c \leftarrow \emptyset$ ; // Candidate groups  $G \leftarrow \emptyset$ ; // Final result
2 for  $i = 1$  to  $|\mathcal{T}|$  do                                     /* Generate valid trajectory pairs */
3   for  $j = i + 1$  to  $|\mathcal{T}|$  do
4     if  $|\tau_i \cap \tau_j| \geq k$  then
5        $P \leftarrow P \cup \{(\tau_i, \tau_j)\}$ ;

6 foreach  $(\tau_i, \tau_j) \in P$  do                                   /* Expand candidate companion groups */
7    $g_i \leftarrow \{\tau_i, \tau_j\}$ ;
8   foreach  $(\tau_k, \tau_p) \in P$  do
9     if  $\forall \tau_p \in g_i, (\tau_k, \tau_p) \in P$  then                 /* Pairwise-valid with all members in  $g_i$  */
10       $g_i \leftarrow g_i \cup \{\tau_k\}$ ;

11 if  $|g_i| \geq m$  then                                           /* Group size constraint */
12    $G_c \leftarrow G_c \cup \{g_i\}$ ;

13  $G \leftarrow \text{select\_max}(G_c)$ ;                                // Select maximal or optimal groups
14 return  $G$ ;
```

---

应对象在该时间段内保持不超过  $\epsilon$  的空间距离共同移动。故共享  $p$  个标记意味着至少存在  $p$  个协同行为时间段。 ■

步骤 2: (搜索近似协同行为对)

基于轨迹标记序列, 我们通过统计两个轨迹之间的公共标记数量来近似计算其协同行为持续时间。对于两个轨迹标记序列  $O(\tau_i)$  和  $O(\tau_j)$ , 若其公共标记数  $|O(\tau_i) \cap O(\tau_j)|$  不小于阈值  $k$ , 则将其记录为一个近似协同行为对。该过程在所有轨迹标记序列均被评估后终止。

**5.5.2 社区感知聚类**

对于轨迹集合  $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_{|\mathcal{O}|}\}$ , 通过第一阶段可获得轨迹标记序列集合

$$O(\mathcal{T}) = \{O(\tau_1), O(\tau_2), \dots, O(\tau_{|\mathcal{T}|})\},$$

以及一组近似协同行为对。在第二阶段中, 我们的目标是发现所有满足以下条件的子集  $\mathcal{T}' \subseteq \mathcal{T}$ : (1)  $|\mathcal{T}'| \geq m$ ; (2) 对任意  $\tau_i, \tau_j \in \mathcal{T}'$ , 均有  $|O(\tau_i) \cap O(\tau_j)| \geq k$ ,

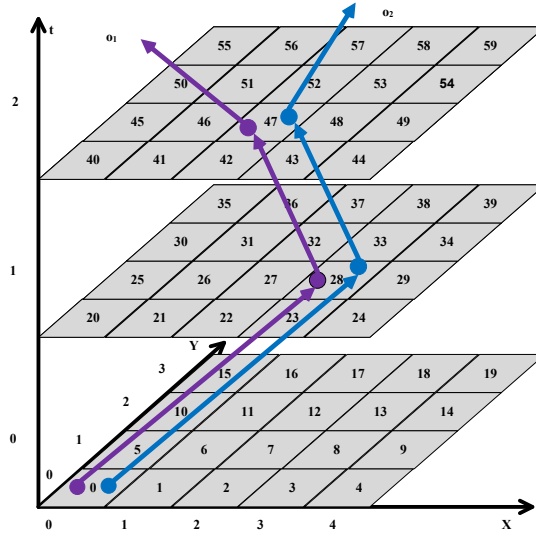


图 5-2 轨迹标记及标记序列示例

即伴随群体。为此，我们采用聚类技术以实现高效检测。

现有轨迹聚类方法主要包括基于划分的聚类 [??]、基于密度的聚类 [??] 以及基于深度学习的聚类 [???]。然而，这些方法难以刻画对象之间的内在连接关系，且基于密度的方法通常具有较高的计算复杂度。

我们的核心思想是将协同行为关系建模为网络或图结构，其中节点表示移动对象，边表示协同行为对。一个伴随群体由至少  $m$  个相互协同行为的对象组成。受同质性假设启发——即共享大量协同行为邻居的对象更可能属于同一群体——我们采用基于共享邻居的社区感知聚类方法。

SCAN 算法 [?] 是一种典型的社区感知聚类方法，其基于顶点结构与连通性进行聚类，时间复杂度为  $O(|P|)$ ，其中  $|P|$  为协同行为对的数量（即图中的边数）。因此，SCAN 能够高效地用于伴随群体检测。

**定理 5.3** CS-ACMC 算法的时间复杂度为  $O(|\mathcal{T}|^2 \times |\tau|)$ ，其中  $|\mathcal{T}|$  表示轨迹数量， $|\tau|$  表示单条轨迹的长度。

**证明：**CS-ACMC 算法需要对每一对轨迹计算其公共标记，时间复杂度为  $O(|\mathcal{T}|^2 \times |\tau|)$ 。在社区感知聚类阶段，SCAN 算法的时间复杂度为  $O(|P|)$ ，其中  $|P| \leq |\mathcal{T}|^2$ 。因此，总体时间复杂度为

$$O(|\mathcal{T}|^2 \times |\tau| + |P|) = O(|\mathcal{T}|^2 \times |\tau|).$$

■

表 5-1 Parameter Settings

|                 | Beijing                     | Porto                       |
|-----------------|-----------------------------|-----------------------------|
| Longitude       | [116.250, 116.550]          | [-8.735, -8.156]            |
| Latitude        | [39.830, 40.030]            | [40.953, 41.307]            |
| Length          | [20, 100]                   | [20, 100]                   |
| $\epsilon_s$    | 200 meters                  | 200 meters                  |
| $\epsilon_t$    | 200 seconds                 | 200 seconds                 |
| $ \mathcal{T} $ | 2K–10K<br>/default 6K       | 20K–100K<br>/default 20K    |
| $k$             | 2–6 (periods)<br>/default 4 | 2–6 (periods)<br>/default 4 |
| $m$             | 2–6 (members)<br>/default 4 | 2–6 (members)<br>/default 4 |
| $\mu$           | 0.42–0.50<br>/default 0.50  | 0.42–0.50<br>/default 0.50  |

## 5.6 实验评估

### 5.6.1 实验设置

**5.6.1.0.1 数据准备** 本文实验基于两个真实世界的出租车轨迹数据集。第一个数据集来自北京出租车轨迹数据 [20, 100]<sup>③</sup>，该数据集包含 10,357 辆出租车在中国北京一周内的轨迹记录。该数据集共包含约 1,500 万个位置点，轨迹总行程约为 900 万公里。第二个数据集<sup>④</sup>采集自葡萄牙波尔图市，时间跨度为 19 个月，包含超过 170 万条轨迹。

在北京数据集中，轨迹的平均采样时间间隔为 177 秒，对应的平均采样距离约为 623 米；而在波尔图数据集中，每辆出租车以 15 秒为间隔上报其位置信息。时间戳的取值范围被统一映射到 24 小时内。对于波尔图数据集，每条轨迹的出发时间被随机设置在 24 小时范围内。所有轨迹均被预先转换为轨迹标记序列（trajectory token sequence）。

生成标记序列及伴随群体的默认参数设置如表 ?? 所示。同时，我们对轨迹的经纬度范围以及轨迹长度进行了限制，具体设置亦列于表 ?? 中。

**5.6.1.0.2 对比算法** 本文对以下两种算法进行性能比较：Exact Search with Exact Co-Movement Computation（ES-ECMC）算法，以及 Community-aware Search with

③ <https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample/>

④ <https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i/data>

*Approximate Co-Movement Computation* (CS-ACMC) 算法。此外, 采用 R-tree 索引结构 [?] 以在早期阶段剪枝不满足条件的对象, 从而提升整体计算效率。

**5.6.1.0.3 评估指标** 为了评估 CS-ACMC 算法的有效性, 我们采用 ES-ECMC 算法中发现的协同行为对 (参见问题定义) 作为基准, 并使用 *协同行为对的平均命中率* 作为评估指标。

具体而言, 对于 ES-ECMC 算法发现的每一个精确伴随群体  $g$ , 若其中任意一个协同行为对  $\langle o_i, o_j \rangle$  (其中  $o_i \in g, o_j \in g$ ) 存在于 CS-ACMC 算法发现的任意一个伴随群体中, 则认为该协同行为对被命中, 命中计数加一; 否则认为其未被命中。为简化描述, 本文使用“命中率 (hit ratio)”表示该指标。该指标能够有效反映近似算法所得伴随群体的结果质量。

在效率评估方面, 我们采用平均 CPU 运行时间作为主要性能指标。需要注意的是, 在默认参数设置下, ES-ECMC 算法在北京数据集上使用单线程至少需要 1 天的运行时间, 因此在实验中我们为 ES-ECMC 启用了 40 个线程, 而 CS-ACMC 仅使用单线程。所有算法均使用 Python 实现, 并在 Ubuntu 平台上运行, 实验环境配置为: 2 颗 Intel(R) Xeon(R) Silver 4214R CPU (2.40GHz) 以及 128GB 内存。除非特别说明, 所有实验结果均为 20 次不同测试轨迹输入下的平均值。

## 5.6.2 性能评估

**5.6.2.0.1 轨迹数量的影响** 首先, 我们在默认参数设置下, 研究轨迹数量变化对算法性能的影响。具体而言, 在波尔图数据集中, 轨迹数量从 20,000 增加至 100,000; 在北京数据集中, 轨迹数量从 2,000 增加至 10,000。之所以采用该设置, 是因为北京数据集的轨迹密度高于波尔图, 因此北京数据集中任意两条轨迹更容易形成协同行为对。

随着轨迹数量的增加, 可以观察到波尔图数据集上的命中率呈现上升趋势 (见图 ??), 而北京数据集上的命中率变化趋势并不明显 (见图 ??)。值得注意的是, 波尔图数据集中捕获的协同行为对数量明显多于北京数据集, 从而带来更高的计算开销。从图 ?? 和 ?? 可以看出, 我们的模型在运行时间上至少比 ES-ECMC 快一个数量级。即便 ES-ECMC 采用了多线程计算, 当轨迹数量分别达到 10,000 (北京) 和 100,000 (波尔图) 时, 其运行时间仍超过 8,000 秒和 20,000 秒。这些结果表明, 我们的方法在保证较高有效性的同时, 也具备显著的效率优势。

**5.6.2.0.2 协同行为持续时间阈值的影响** 图 ?? 展示了当协同行为持续时间阈值  $k$  从 2 增加到 6 时算法的性能表现。直观上, 较大的  $k$  会减少协同行为对和伴随群

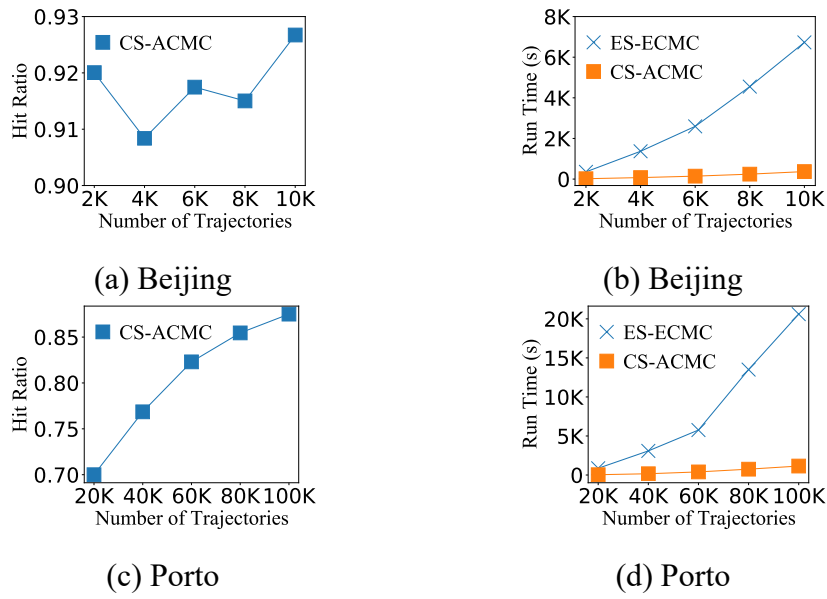


图 5-3 Effect of the number of trajectories

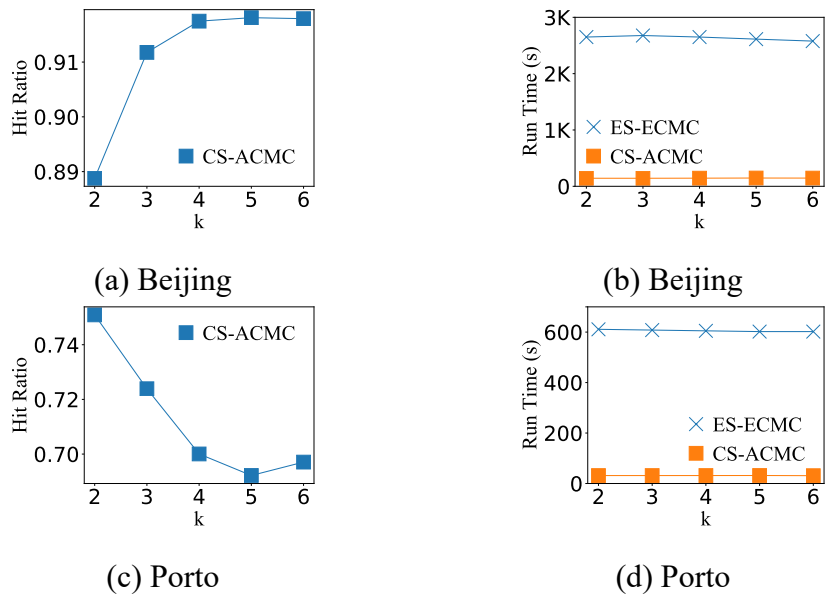
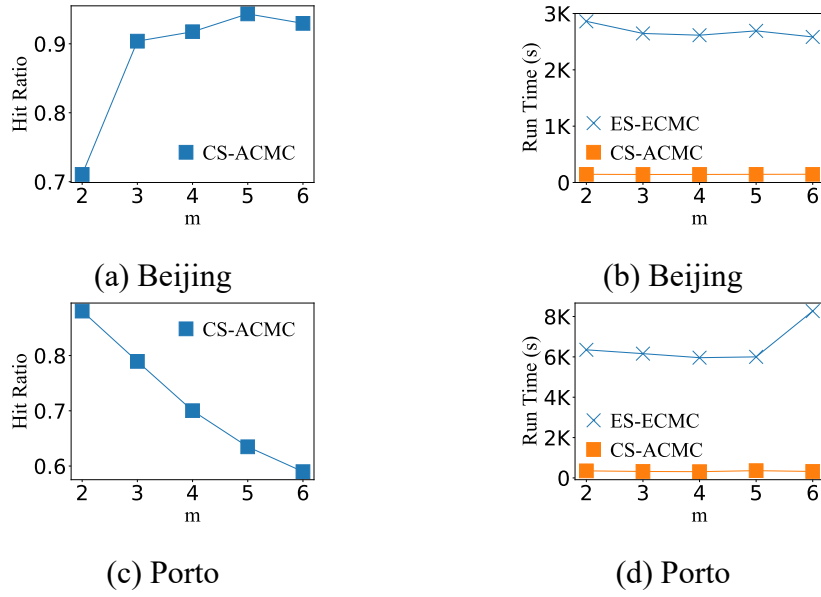


图 5-4 Effect of parameter  $k$


 图 5-5 Effect of parameter  $m$ 

体的数量，从而可能降低计算成本。然而，如图 ?? 和 ?? 所示，运行时间并未随着  $k$  的增大而呈现明显下降趋势。其原因在于，主要计算开销来自于协同行为对的枚举过程，而从协同行为对中发现伴随群体的成本相对较低。此外，当  $k$  增大时，北京数据集上的命中率有所提升，而波尔图数据集上的命中率则有所下降，这可能与两者不同的数据分布特性有关。在所有参数设置下，我们的方法在 CPU 运行时间方面均显著优于 ES-ECMC。

**5.6.2.0.3 群体规模阈值的影响** 图 ?? 展示了当群体规模阈值  $m$  从 2 变化到 6 时的算法性能。更大的群体规模阈值或协同行为持续时间阈值通常会导致伴随群体数量减少。随着  $m$  的增加，北京数据集上的命中率呈现上升趋势，而波尔图数据集上的命中率则有所下降，这同样可归因于数据分布差异。实验结果表明，在所有设置下，我们的方法在运行时间方面始终优于 ES-ECMC。

**5.6.2.0.4 聚类参数的影响** 图 ?? 展示了当结构相似度阈值  $\mu$  从 0.42 增加到 0.50 时的命中率和运行时间变化情况。直观而言，较小的  $\mu$  值意味着两个对象更容易被连接，从而更多对象可能被划分到同一群体中，最终生成的群体数量减少。如图 ?? 所示，在波尔图数据集中，随着  $\mu$  的增大，命中率呈现下降趋势，其中  $\mu = 0.42$  时取得最佳效果。相比之下，北京数据集上的命中率始终高于 0.9，且随  $\mu$  变化不明显，这可能是由于北京数据集具有更高的采样密度。即便 ES-ECMC 采用多线程实现，其运行时间仍显著高于 CS-ACMC，至少相差一个数量级。实验结果进一步验证了本文方法在效率和效果上的优越性。

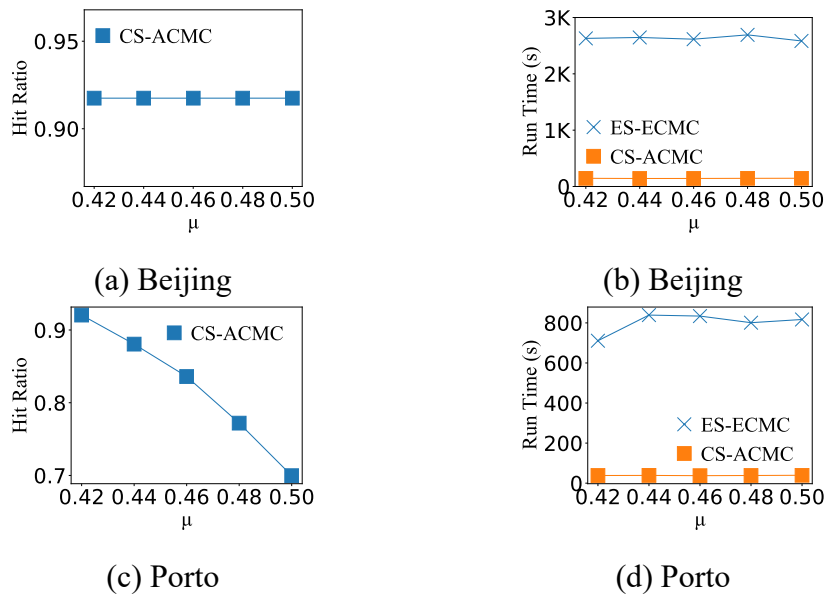


图 5-6 Effect of the structural similarity

## 5.7 本章小结

## 第六章 面向疾病传播控制：大规模移动对象上的连续暴露发现

### 6.1 引言

传染性疾病长期以来一直被视为重大的公共卫生问题。密切接触发现（close contact discovery）在防止疫情传播方面发挥着不可或缺的作用。在此背景下，我们研究**连续暴露搜索问题**：给定一组移动对象和一组移动查询对象，在一段时间内持续地发现所有直接或间接暴露于至少一个查询对象的对象。

该问题面向多种应用场景，包括但不限于疾病防控、疫情预警、信息传播分析以及共移动模式挖掘。

为了解决该问题，我们设计了一种带有优化策略的精确群组处理算法。此外，我们还提出了一种近似算法，在不产生漏检（false dismissal）的前提下显著提升了计算效率。大量实验结果验证了所提出算法在有效性和效率方面的优势。

#### 6.1.1 研究背景与意义

随着位置追踪设备（如车载导航系统和智能手机）以及 GPS 赋能服务（如 Google Maps）的持续普及，轨迹数据的规模正在迅速增长。在多种轨迹查询任务中 [?? ?]，接触搜索与接触追踪问题近年来受到了广泛关注 [?? ?]。一般而言，当两个移动对象在一段时间内保持较近的空间距离并相互影响时，即构成一次密切接触。对密切接触进行高效、准确的发现，在诸多实际场景中具有重要意义，包括但不限于疾病防控、疫情预警、信息传播分析以及共移动模式挖掘。在疫情期间，实时发现密切接触对象有助于限制人群传播并评估个体感染风险。

现有研究主要聚焦于针对单个查询对象（或查询轨迹）的接触追踪问题，通常基于查询对象与数据对象之间的空间邻近性进行处理（如 [?? ?]），也有部分研究基于轨迹片段之间的相似性计算开展分析（如 [?? ?]）。然而，在疾病暴发期间，大规模个体可能在短时间内被感染，因此，基于轨迹对海量移动对象进行连续接触搜索显得尤为重要。一个有效的接触搜索方法不仅需要能够同时处理大规模移动对象，还应保证对每个对象的实时检测能力。

据我们所知，现有研究通常将该问题建模为独立的查询处理问题、轨迹搜索问题 [?? ?]，或多查询搜索问题 [?? ?]。这些方法要么仅适用于少量查询，要么假设输入为静态轨迹集合，因而在实际场景中难以有效处理大规模移动对象及大量查询。此外，这些方法也难以对每个对象持续维护实时检测结果。



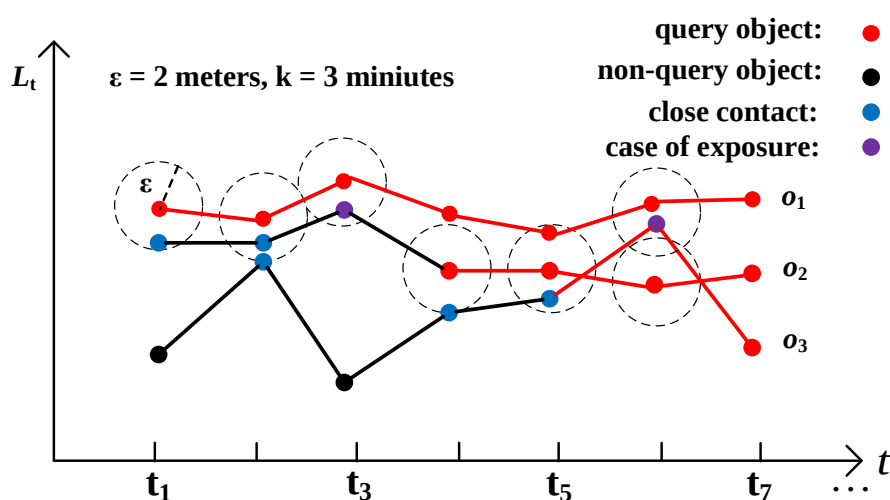


图 6-1 Example of cases of exposure

### 6.1.2 相关工作

与我们研究的问题相似的一个方向是协同行为模式挖掘 (co-movement pattern mining)。现有研究已经讨论了多种有趣的协同行为模式 [??]。其中, flock 模式 [?] 是最早提出的协同行为模式, 用于刻画在连续  $k$  个时间区间内共同移动的  $m$  个移动对象组成的群体。在相关工作中, 文献 [?] 从交通拥堵发现的角度提出了 gathering 模式。此外, 文献 [?] 以流式方式挖掘协同行为群体。与上述研究不同的是, 我们的 CES 问题不要求对群体中对象数量进行约束。需要注意的是, 在我们的定义下, 群体之外的对象也可能被视为感染对象。因此, 这些研究提出的解决方案均无法直接应用于我们的问题。

为了有效防止 SARS、埃博拉以及 COVID-19 等传染病的传播, 已有大量相关研究被提出 [??]。其中, 文献 [?] 首次提出了接触追踪 (contact tracing) 问题, 但该工作主要关注疾病传播的模拟, 无法支持实时查询。从另一个角度来看, 文献 [?] 提出了接触追踪查询 (Contact Tracing Query, CTQ), 用于识别与查询对象存在直接或间接接触的个体。文献 [?] 提出了广义轨迹接触搜索 (Trajectory Contact Search, TCS) 查询, 并设计了一种迭代算法进行求解。另一项相关工作 [?] 提出了一种用于追踪与患者接触的所有对象并搜索潜在感染者的技术, 但该方法基于轨迹片段, 且主要面向室内应用场景。据我们所知, 现有方法均无法直接用于解决本文提出的问题, 因为它们要么针对静态轨迹数据, 要么采用了不同的暴露案例定义。

### 6.1.3 研究成果与贡献

在此背景下，我们定义并研究了连续暴露搜索（Continuous Exposure Search, CES）问题：给定一组移动对象  $\mathcal{O}$ ，以及一个查询集合  $\mathcal{Q} \in \mathcal{O}$ ，用于表示已有的暴露病例，我们的目标是通过实时更新  $\mathcal{Q}$ ，持续检测新的暴露病例。在我们的设定中，新的暴露病例是指在距离阈值  $\epsilon$  内、并且在时间阈值  $k$  内与查询对象共同移动的对象。

如图 ?? 所示的示例中，设  $o_1, o_2, o_3$  为三个移动对象，初始查询集合为  $\mathcal{Q} = \{o_1\}$ 。距离阈值  $\epsilon$  设为 2 米，接触时长阈值  $k$  设为 3 分钟。我们使用红色、蓝色和紫色点分别表示已暴露（查询）对象的位置、密切接触对象的位置以及最新确认的暴露病例的位置。对象  $o_2$  在连续时间段  $[t_1, t_3]$  内与查询对象  $o_1$  保持密切接触，因此我们将  $o_2$  加入查询集合，即更新为  $\mathcal{Q} = \{o_1, o_2\}$ ，并在  $t_3$  时刻返回更新后的  $\mathcal{Q}$  作为实时结果。类似地，对象  $o_3$  在时间段  $[t_4, t_6]$  内与查询对象发生密切接触，因此在  $t_6$  时刻将查询集合更新为  $\mathcal{Q} = \{o_1, o_2, o_3\}$ 。

为 CES 问题设计一种既高效又有效的解决方案并非易事。直接方法难以应对大规模移动对象，同时也无法保证实时性。为此，我们提出了一种高效的精确实时搜索算法，并引入多种优化技术。此外，我们还提出了一种近似实时搜索算法，在不产生漏检的前提下进一步提升搜索效率。本文的主要贡献总结如下：

- 我们定义了一种新的 CES 问题，用于在大量移动对象中持续维护最新的暴露病例。该问题有助于监测和控制流行病的传播，例如 SARS、埃博拉和 COVID-19。
- 我们提出了两种算法来解决 CES 问题，分别是精确实时搜索算法 Exact Group Processing (EGP) 和近似实时搜索算法 Approximate Group Processing (AGP)。同时，我们设计了分组过滤和预检查策略以提升搜索效率。
- 我们在两个真实数据集上进行了大量实验来评估所提方法的有效性和效率。实验结果表明，该方法能够在大规模移动对象场景下实时检测暴露病例。

## 6.2 问题描述

我们以一组移动对象  $\mathcal{O} = \{o_1, \dots, o_n\}$  以及一个查询对象集合  $\mathcal{Q} (\mathcal{Q} \subseteq \mathcal{O})$  作为输入。在现实场景中，例如，移动对象可以是行人，而查询对象可以是已暴露于新冠病毒的人群。我们的目标是以实时的方式检测潜在的暴露案例。接下来，我们将在本节其余部分中定义一些重要概念。

**定义 6.1 (移动对象的位置流)** 移动对象  $o$  在时间  $t$  的位置表示为  $l_t^o = ([x, y], t)$ ，其中  $[x, y]$  表示空间坐标， $t$  表示时间戳。需要注意的是， $l_t^o$  是动态更新的。我们

使用  $L_t(\mathcal{O}) = \{l_t^o \mid o \in \mathcal{O}\}$  来表示所有移动对象在时间  $t$  的位置集合。

**定义 6.2（近距离接触）** 若对象  $o_i$  与对象  $o_j$  之间的距离小于给定的距离阈值  $\epsilon$ ，则称对象  $o_i$  与对象  $o_j$  发生了近距离接触。

**定义 6.3（暴露案例）** 给定持续时间阈值  $k$ ，若对象  $o$  ( $o \in \mathcal{O} \setminus \mathcal{Q}$ ) 与查询集合  $\mathcal{Q}$  中任意对象之间的连续近距离接触持续时间超过  $k$ ，则称对象  $o$  为一个暴露案例。

**定义 6.4（连续暴露搜索问题）** 给定一组移动对象  $\mathcal{O}$ 、一组已暴露（查询）对象  $\mathcal{Q}$ 、社交距离阈值  $\epsilon$  以及接触持续时间阈值  $k$ ，Continuous Exposure Search（CES）问题旨在通过实时更新  $\mathcal{Q}$ ，持续检测新的暴露案例。

### 6.3 精确遍历算法

一种用于回答 CES 问题的直接精确解法（ET）如下所示。在每一个时间戳，我们检查每个对象  $o_i \in \mathcal{O} \setminus \mathcal{Q}$  是否与每个查询对象  $o_j \in \mathcal{Q}$  发生近距离接触。具体而言，若对象  $o_i$  与查询对象  $o_j \in \mathcal{Q}$  之间的距离小于阈值  $\epsilon$ ，则将对象  $o_i$  记录为一个近距离接触对象。我们对所有对象不断到达的位置数据周期性地执行上述操作，并通过在每个时间点维护一个哈希集合来跟踪连续的近距离接触情况。

当某个对象未能持续保持近距离接触，即在其接触持续时间达到  $k$  之前发生中断时，我们将重置该对象的接触时长。若某个对象在连续时间段内被记录为近距离接触对象，且持续时间达到  $k$ ，则将其视为一个暴露案例。随后，我们通过将最新检测到的暴露案例加入查询集合  $\mathcal{Q}$  来更新该集合，并将更新后的  $\mathcal{Q}$  作为实时结果返回。最后，更新后的  $\mathcal{Q}$  将用于下一轮检测过程。

**6.3.0.0.1 时间复杂度。** 设  $|\mathcal{Q}|$  表示查询对象的数量， $|\mathcal{O}|$  表示非查询对象的数量。在每一个时间戳，都需要执行一轮 ET 算法。每一轮 ET 的时间复杂度为  $O(|\mathcal{Q}| \times |\mathcal{O}|)$ 。因此，该方法在处理大规模移动对象并维持实时检测结果时具有极高的计算开销，难以在实际场景中高效应用。

### 6.4 精确分组处理算法

为在高效回答 CES 问题的同时，保持对每个移动对象的精确且实时的检测结果，我们提出了一种 精确分组处理（Exact Group Processing，EGP）算法。EGP 算法不再计算每个对象与每个查询对象之间的两两距离，而是根据对象的当前位置分别对查询对象和非查询对象进行空间划分，形成查询组和非查询组。随后，算法通过两个阶段来发现暴露案例。在阶段一中，我们在组层面计算组与组之间的距离，并生成潜在的暴露候选；在阶段二中，我们再在对象层面对这些潜在暴露候

选进行精确验证。此外，我们还提出了若干预检查策略，以进一步提升 EGP 算法的执行效率。

**6.4.0.0.1 组距离。** 在 ET 算法中，需要计算每个非查询对象与每个查询对象之间的距离来发现近距离接触，这一过程极其耗时。为避免这种两两距离计算，已有大量距离度量被提出，用以衡量两组位置之间的空间距离，其中 Hausdorff 距离 [?] 是一种具有代表性的度量方法。然而，对于分别包含  $m$  和  $n$  个位置的两组对象，一次 Hausdorff 距离的计算需要  $O(m \times n)$  的时间复杂度，这在大规模位置数据场景下会带来极高的计算开销。

**6.4.0.0.2 分组划分。** 为了实现对近距离接触的快速发现，我们基于对象的当前位置将所有对象划分为若干组。由于同一组内的对象在空间上彼此接近，因此可以在早期阶段有效过滤掉距离较远的对象。具体而言，我们采用一个网格索引  $\mathcal{C}$  对底层空间进行划分，每个网格单元为边长为  $\frac{\sqrt{2}}{2}\epsilon$  的正方形，其中  $\epsilon$  表示近距离接触的距离阈值。落入同一单元格  $c \in \mathcal{C}$  的查询对象和非查询对象，分别构成一个查询组和一个非查询组，记为  $\mathcal{O}_c^q$  和  $\mathcal{O}_c$ 。需要注意的是，查询组和非查询组是分别存储和索引的。

图 ?? 展示了在给定一个位于单元格  $c_{ab}$ （以星号标记）的查询组时，如何识别可能存在暴露案例的对象分组。由图 ?? 可见，仅有落入红色单元格中的对象（即  $SC = \{c_{ij} \in \mathcal{C} \mid |i-a| \leq 2 \wedge |j-b| \leq 2 \wedge |i-a| + |j-b| < 4\}$ ）才有可能在距离阈值  $\epsilon$  内与查询对象发生近距离接触，因此这些对象被视为潜在的暴露案例。需要指出的是， $|SC|$  为一个常数，其值为 21。对于不在红色单元格中的对象分组（例如灰色单元格），由于其与  $c_{ab}$  中查询对象的距离超过  $\epsilon$ ，因此不可能发生近距离接触。

**6.4.0.0.3 预检查策略。** 为了发现非查询组  $\mathcal{O}_{c'}$  中所有与查询组  $\mathcal{O}_c^q$  发生近距离接触的对象，直接计算需要  $O(|\mathcal{O}_c^q| \times |\mathcal{O}_{c'}|)$  的时间复杂度，开销较大。为此，我们基于分组划分设计了若干预检查策略，以避免不必要的计算。具体而言，我们首先扫描  $\mathcal{O}_c^q$  和  $\mathcal{O}_{c'}$  中的所有位置点，分别得到其最小外包矩形  $MBR(\mathcal{O}_c^q)$  和  $MBR(\mathcal{O}_{c'})$ 。公式 ?? 定义了集合  $\{dist(l^{o_i}, l^{o_j}) \mid o_i \in \mathcal{O}_c^q, o_j \in \mathcal{O}_{c'}\}$  的上界  $UB$  和下界  $LB$ 。与 Hausdorff 距离相比，该预检查过程仅需要  $O(|\mathcal{O}_c^q| + |\mathcal{O}_{c'}|)$  的线性时间复杂度。当上界  $UB(\mathcal{O}_c^q, \mathcal{O}_{c'})$  小于  $\epsilon$  时，我们可直接将  $\mathcal{O}_{c'}$  中的所有对象视为近距离接触对象；相反，当下界  $LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) > \epsilon$  时，则可以安全地剪枝  $\mathcal{O}_{c'}$ 。

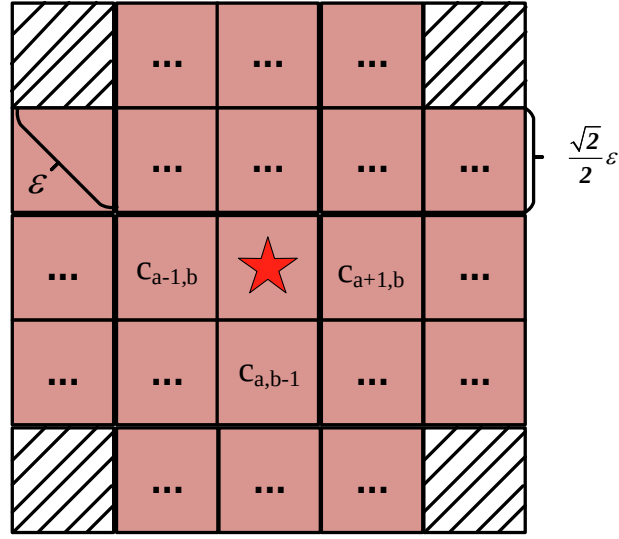


图 6-2 Potential cases of exposure

$$\begin{aligned}
 UB(\mathcal{O}_c^q, \mathcal{O}_{c'}) &= \max(dist(MBR(\mathcal{O}_c^q), MBR(\mathcal{O}_{c'}))) \\
 LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) &= \min(dist(MBR(\mathcal{O}_c^q), MBR(\mathcal{O}_{c'})))
 \end{aligned} \tag{6-1}$$

**6.4.0.0.4 算法细节。** 算法 ?? 给出了我们提出的精确群体处理 (Exact Group Processing, EGP) 算法的伪代码。其输入包括: 对象集合  $\mathcal{O}$ , 位置流  $L_t(\mathcal{O}) = \{l_t^o \mid o \in \mathcal{O}\}$  (表示在每个时间戳  $t$  时对象集合  $\mathcal{O}$  的位置), 查询对象集合, 社交距离阈值  $\epsilon$ , 以及接触时长阈值  $k$ 。输出为更新后的查询对象集合, 其中包含最新的暴露案例。

在初始化阶段, 所有非查询对象的接触时长  $cd(o)$  被设置为 0 (第 1 行)。在搜索过程中, 我们使用  $\Pi$  来存储至少覆盖一个查询对象的网格单元。同时, 用  $A$  表示当前发现的近距离接触对象集合。在算法开始时,  $\Pi$  和  $A$  均被初始化为空集 (第 2 行)。

该算法包含两个阶段: 1) 群体划分阶段 (第 3–11 行); 2) 基于群体的近距离接触发现阶段 (第 12–20 行)。在群体划分阶段, 对于每一个到达的位置  $l_t^o$  (对应对象  $o$ ), 首先确定覆盖该位置的网格单元  $c$ 。如果  $o$  是查询对象, 则将其存入查询群体  $\mathcal{O}_c^q$ , 并将单元  $c$  加入  $\Pi$ ; 否则, 将  $o$  存入非查询群体  $\mathcal{O}_c$  (第 3–11 行)。至此, 所有查询群体和非查询群体均已构建完成。

在近距离接触发现阶段, 我们针对每一个查询群体及其相邻、具有潜在暴露风险的非查询群体, 执行预检查策略 (第 12–20 行)。具体而言, 如果查询群体  $\mathcal{O}_c^q$  与非查询群体  $\mathcal{O}_{c'}$  之间距离的上界 (参见公式 ??) 不大于阈值  $\epsilon$ , 则将  $\mathcal{O}_{c'}$  中的所有对象  $o$  加入当前近距离接触集合  $A$ 。相反, 如果其下界超过  $\epsilon$ , 则可以安全地剪枝  $\mathcal{O}_{c'}$ , 因为该群体中的任何对象都不可能和查询群体中的对象形成近距离接触。

---

**算法 6-1: 精确群体处理 (EGP) 算法**


---

**输入:** 对象集合  $\mathcal{O}$ ;  
 位置流  $L_t(\mathcal{O}), L_{t+1}(\mathcal{O}), \dots$ ;  
 查询对象集合  $\mathcal{Q} \subseteq \mathcal{O}$ ;  
 社交距离阈值  $\epsilon$ ;  
 接触时长阈值  $k$ 。  
**输出:** 更新后的查询对象集合  $\mathcal{Q}$ 。

```

1  初始化:  $\forall o \in (\mathcal{O} - \mathcal{Q}): cd(o) \leftarrow 0$ ;
    $\Pi \leftarrow \emptyset; A \leftarrow \emptyset$ ;
2  for each location  $l_t^o \in L_t(\mathcal{O})$  of object  $o$  do                                /* 对象位置映射到网格 */
3      令  $c$  为覆盖位置  $l_t^o$  的网格单元;
4      if  $o \in \mathcal{Q}$  then
5           $\Pi \leftarrow \{\Pi, c\}; \mathcal{O}_c^q \leftarrow \{\mathcal{O}_c^q, o\}$ ;
6      else
7           $\mathcal{O}_c \leftarrow \{\mathcal{O}_c, o\}$ ;
8  for each query cell  $c \in \Pi$  do                                            /* 处理查询单元 */
9      for each non-query cell  $c' \in SC(c)$  do                                /* 检查相邻非查询单元 */
10         if  $UB(\mathcal{O}_c^q, \mathcal{O}_{c'}) \leq \epsilon$  then
11              $A \leftarrow \{A, \mathcal{O}_{c'}\}$ ;
12         else if  $LB(\mathcal{O}_c^q, \mathcal{O}_{c'}) \leq \epsilon$  then
13              $A \leftarrow \{A, ET(\mathcal{O}_c^q, \mathcal{O}_{c'})\}$ ;
14 更新  $cd(o)$ ;
15  $\mathcal{Q} \leftarrow \{\mathcal{Q} \cup o \mid cd(o) \geq k\}$ ;
16 return 更新后的  $\mathcal{Q}$ ;                                                    // 实时结果
    
```

---

在完成群体级别的发现之后, 我们进一步计算每个查询群体与其相邻、具有潜在暴露风险的非查询群体之间对象的成对距离 (第 17 行)。在每一轮处理结束时, 我们更新所有对象的接触时长 (第 21 行)。具体地, 对于所有不属于  $A$  的对象  $o \in \mathcal{O} \setminus A$ , 将其接触时长重置为 0, 表示在其接触时长达到阈值  $k$  之前发生了中断。

随后, 对集合  $A$  中的对象 (即最新发现的近距离接触对象) 更新其接触时长, 并据此更新查询集合  $\mathcal{Q}$ , 将接触时长满足  $cd(o) \geq k$  的对象作为最新的暴露案例加入其中 (第 22 行)。最后, 输出更新后的  $\mathcal{Q}$  作为实时结果 (第 23 行)。

**6.4.0.0.5 时间复杂度分析。** 构建所有查询群体和非查询群体的时间复杂度为  $O(|\mathcal{O}|)$ 。设  $p$  和  $q$  分别表示每个查询群体和每个非查询群体中的对象数量, 则发现近距离接触的时间复杂度为  $O(|\Pi| \times |SC| \times p \times q) = O(|\Pi| \times p \times q)$ , 其中  $|SC|$  为常数 (参见群体划分过程)。

在最坏情况下，一个网格单元可能覆盖所有查询对象，而另一个网格单元覆盖所有非查询对象（例如  $|\Pi| = 1, p = |Q|, q = |\mathcal{O}|$ ）。由于  $|SC|$  为常数，此时每一轮运行 EGP 的时间复杂度为  $O(|Q||\mathcal{O}|)$ 。需要指出的是，在大多数实际场景中， $p$  和  $q$  通常远小于  $|Q|$  和  $|\mathcal{O}|$ 。

## 6.5 近似分组处理算法

为进一步提升暴露案例发现的效率，我们提出了一种 近似分组处理 (Approximate Group Processing, AGP) 算法。本节将介绍所采用的近似策略及算法细节。

**6.5.0.0.1 首尾优先检查策略。** 在相邻的两个时间戳之间，可能存在大量对象彼此靠近地移动，但其中大多数对象并不能在较长时间内持续同行。若一个非查询对象在其接触时长尚未达到阈值  $k$  之前未能持续成为近距离接触对象，则该对象不可能成为暴露案例。基于这一观察，我们提出了首尾优先检查策略。与按时间顺序逐一发现近距离接触不同，我们首先计算  $A_t$ （即“尾部”）与  $A_{t-k}$ （即“头部”）的交集，以获得在时间戳  $t$  仍可能存在近距离接触的候选对象，其中  $A_t$  表示时刻  $t$  的近距离接触集合。对于时间区间  $[t-k+1, t]$ ，我们只需在该交集结果中进一步查找近距离接触，从而显著提升空间和时间效率。

**6.5.0.0.2 跳跃检查。** 给定接触时长阈值  $k$ ，若对象  $o$  在连续  $k$  个时间段  $[t_i, t_{i+k-1}]$  内是潜在的暴露案例，并且在时间戳  $t_i$  和  $t_{i+k-1}$  均与至少一个查询对象发生近距离接触，则我们将对象  $o$  视为一个近似暴露案例。基于近似暴露案例的定义，我们提出跳跃检查策略：不同于处理所有时间戳上的位置数据，我们仅在部分时间戳  $T = 0, m, 2m, \dots$  上处理位置数据，其中  $m$  为跳跃长度。需要注意的是，当  $m = 1$  时，需要在每一个时间戳上进行处理。若时间戳  $t \notin T$ ，我们假设所有对象均为近距离接触对象。跳跃检查的目标是在不产生漏检的前提下，进一步降低计算开销。

**6.5.0.0.3 算法细节。** 算法 ?? 给出了 AGP 算法的伪代码。与 EGP 相比，AGP 额外引入了跳跃长度  $m$  作为输入参数。算法的输出为更新后的查询对象集合，其中包含原有查询对象以及最新发现的近似暴露案例。我们使用  $cd'(o)$  表示对象  $o$  作为潜在暴露案例的持续时长，并在算法开始时对每个非查询对象初始化  $cd'(o)$ （第 1 行）。在算法中，仅当  $t \% m = 0$  时，才在该时间戳上检测潜在暴露案例（第 2 行）。我们使用  $\Pi$  来存储至少包含一个查询对象  $o \in Q$  的网格单元集合。我们分别用  $A$  和  $A'$  表示近距离接触集合和潜在暴露案例集合。对于对象  $o$  在时间  $t$  的位置

**算法 6-2: 近似分组处理算法 (AGP)**


---

**输入:** 对象集合  $\mathcal{O}$ ;  
 位置流  $L_t(\mathcal{O}), L_{t+1}(\mathcal{O}), \dots$ ;  
 查询对象集合  $\mathcal{Q} \subseteq \mathcal{O}$ ;  
 社交距离阈值  $\epsilon$ ;  
 接触持续时间阈值  $k$ ;  
 跳跃步长  $m$ 。

**输出:** 实时近似暴露结果 (更新后的查询集合  $\mathcal{Q}$ )。

```

1  初始化: 对所有  $o \in (\mathcal{O} \setminus \mathcal{Q})$ , 令  $cd'(o) \leftarrow 0$ ;
2  if  $t \bmod m = 0$  then                                     /* 仅在跳跃时间点进行处理 */
3       $\Pi \leftarrow \emptyset$ ;  $A' \leftarrow \emptyset$ ;
4      foreach 对象  $o$  在时刻  $t$  的位置  $l_t^o \in L_t(\mathcal{O})$  do           /* 分组阶段 */
5          令  $c$  为覆盖  $l_t^o$  的网格单元;
6          if  $o \in \mathcal{Q}$  then                                           /* 查询对象 */
7               $\Pi \leftarrow \Pi \cup \{c\}$ ;
8               $\mathcal{O}_c^q \leftarrow \mathcal{O}_c^q \cup \{o\}$ ;
9          else                                                         /* 非查询对象 */
10              $\mathcal{O}_c \leftarrow \mathcal{O}_c \cup \{o\}$ ;
11         foreach 查询单元  $c \in \Pi$  do                               /* 近邻单元扫描 */
12             foreach 非查询单元  $c' \in SC(c)$  do
13                  $A' \leftarrow A' \cup \mathcal{O}_{c'}$ ;
14             // 更新接触持续时间
15             更新所有对象的  $cd'(o)$ ;
16             // 头尾优先检查并更新查询集合
17              $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{o \mid o \in A_t \cap A_{t-k+1} \wedge cd'(o) \geq k\}$ ;
18 return 更新后的  $\mathcal{Q}$  作为实时近似结果;
    
```

---

$l_t^o \in L_t(\mathcal{O})$ , 我们执行与 EGP 相同的操作以获得查询分组  $\mathcal{O}_c^q$  以及非查询分组  $\mathcal{O}_c$  (第 4–12 行)。随后, 我们将位于  $SC(c)$  中的对象视为潜在暴露案例, 并相应地增加其接触时长 (第 13–17 行)。接着, 我们对不属于  $A'$  的对象  $o$  重置其接触时长 (第 18 行)。对于满足  $cd'(o) \geq k$  的对象, 我们进一步检查其在时间戳  $t$  和  $t - k + 1$  是否为近距离接触对象。若条件成立, 则将其视为近似暴露案例, 并将其加入查询集合  $\mathcal{Q}$  (第 19 行)。最后, 返回更新后的  $\mathcal{Q}$  作为实时检测结果。

**6.5.0.0.4 时间复杂度分析。** 分组划分过程需要  $O(|\mathcal{O}|)$  的时间复杂度, 主要差异体现在暴露案例的发现阶段。此外, 潜在暴露案例的发现需要  $O(|\Pi| \times |SC|) = O(|\Pi|)$  的时间。在最坏情况下, 任意两个查询对象位于不同的网格单元中, 即  $|\Pi| = |\mathcal{Q}|$ 。因此, AGP 算法每一轮的整体时间复杂度为  $O(|\mathcal{O}| + |\mathcal{Q}|)$ 。需要注意的是,  $|\Pi|$  的



最大值为  $|Q|$ 。

## 6.6 实验研究

### 6.6.1 实验设置

**6.6.1.0.1 数据准备。** 我们的实验基于两个真实世界的轨迹数据集开展。第一个数据集来源于北京出租车数据集 [?]，包含在北京一周内跟踪到的 10,357 辆出租车的轨迹，总数据点数量约为 1,500 万。第二个数据集采集自葡萄牙波尔图市 (PT)，时间跨度为 19 个月，包含超过 170 万条轨迹。在北京数据集中，平均采样时间间隔为 177 秒，对应的平均移动距离约为 623 米；而在波尔图数据集中，每辆出租车以 15 秒的间隔上报其位置信息。时间戳的取值范围被限制在 24 小时之内。我们限制经纬度范围，以保证采样位置具有足够高的空间密度。此外，我们过滤掉采样点数量较少的轨迹，具体而言，仅保留采样点数量不少于 10 的轨迹用于实验。为了能够在任意时间点计算对象（如出租车、行人）之间的距离，我们通过线性插值对所有轨迹进行时间对齐，使得每个对象以固定频率返回其实时位置，其中北京和波尔图数据集的频率分别为 10 秒和 5 秒。最终，北京和波尔图数据集中分别包含 296,364,033 和 23,767,470 个有效位置点。在不失一般性的前提下，我们从  $\mathcal{O}$  中随机选取部分对象  $Q$  作为初始查询对象。

**6.6.1.0.2 对比算法。** 我们对比评估了所提出的算法，包括精确遍历算法 (ET)、精确分组处理算法 (EGP) 以及近似算法 (AGP)。对于 AGP，默认的跳跃长度 (hop length) 设置为 2。效率评估和效果评估所采用的性能指标分别为 CPU 时间以及发现的（近似）暴露案例总数。为简化描述，我们使用  $|CE|$  表示（近似）暴露案例的数量。CPU 时间定义为每一轮算法运行的平均执行时间。此外，我们还评估了 AGP 的准确性，其计算方式为真实暴露案例数量与近似暴露案例数量之比。

**6.6.1.0.3 参数设置。** 默认参数设置如表 ?? 所示，该设置基于两个数据集的具体数据分布。所有算法均采用 Java 实现，并在配备 AMD Ryzen 5 处理器 (3.6GHz) 和 32GB 内存的 Windows 10 平台上进行评测。除非另有说明，实验结果均为在不同查询对象输入条件下进行 20 次独立实验后的平均值。

### 6.6.2 性能评估

**6.6.2.0.1 查询对象数量的影响。** 首先，我们在默认参数设置下研究查询对象数量  $|Q|$  对所提出算法性能的影响。如图 ?? 所示，在 Porto 数据集上，暴露案例数量呈现出显著的增长趋势，而在 BJ 数据集上该趋势并不明显。可以清楚地看到，ET 与

|            | 北京 (Beijing)                               | 波尔图 (Porto)                                    |
|------------|--------------------------------------------|------------------------------------------------|
| 经度         | [116.250, 116.550]                         | [-8.735, -8.156]                               |
| 纬度         | [39.830, 40.030]                           | [40.953, 41.307]                               |
| $\epsilon$ | {2, 4, 6, 8, 10}<br>2 米 (默认)               | {2, 4, 6, 8, 10}<br>2 米 (默认)                   |
| $k$        | {5, 10, 15, 20, 25}<br>15 (默认)             | {5, 7, 9, 11}<br>5 (默认)                        |
| $ Q $      | {2, 4, 6, 8, 10} $\times 10^2$<br>600 (默认) | {2, 4, 6, 8, 10} $\times 10^4$<br>100,000 (默认) |

表 6-1 参数设置

EGP 发现的暴露案例数量完全一致，这是因为它们都是精确算法。三种算法的结果总体上较为接近。如图 ?? 和图 ?? 所示，在 CPU 时间方面，组处理算法的性能略逊于近似算法。ET 在处理北京数据集时所需的 CPU 时间超过 1,000 ms，这表明 ET 无法用于实时查询。相比之下，EGP 和 AGP 至少将所需 CPU 时间降低了一个数量级，充分体现了我们方法的优势。

**6.6.2.0.2 距离阈值  $\epsilon$  的影响。** 直观上，较大的  $\epsilon$  值意味着两个对象更容易与查询对象发生近距离接触。图 ?? 展示了当距离阈值  $\epsilon$  从 2 米变化到 10 米时的性能表现。正如预期的那样，在 BJ 和 PT 数据集上，随着  $\epsilon$  的增大，所有算法发现的暴露案例数量均随之增加。从图 ?? 可以看出，在 BJ 数据集上，随着  $\epsilon$  的增大，ET 与 AGP 结果之间的差异逐渐缩小；而在 PT 数据集上，这种差异则随着  $\epsilon$  的增大而更加明显。这种现象可能归因于两种数据集在分布上的差异。值得注意的是，ET 算法的运行时间并没有呈现出明显的变化趋势，这是因为 ET 所需的计算开销在很大程度上依赖于查询对象的数量。

**6.6.2.0.3 接触持续时间  $k$  的影响** 我们研究了接触持续时间阈值  $k$  对算法性能的影响。较小的  $k$  表示对象更容易被判定为暴露案例，从而导致暴露案例数量增加，并使得在每个时间快照下需要更高的 CPU 计算开销。如图 ?? 和图 ?? 所示，当  $k$  从 10 增加到 40 时，所有算法在北京和波尔图数据集上的暴露案例数量均显著下降。此外，从图 ?? 和图 ?? 可以观察到 CPU 时间呈现出轻微的下降趋势，这是因为较大的  $k$  会减少需要进一步检查的候选对象数量。

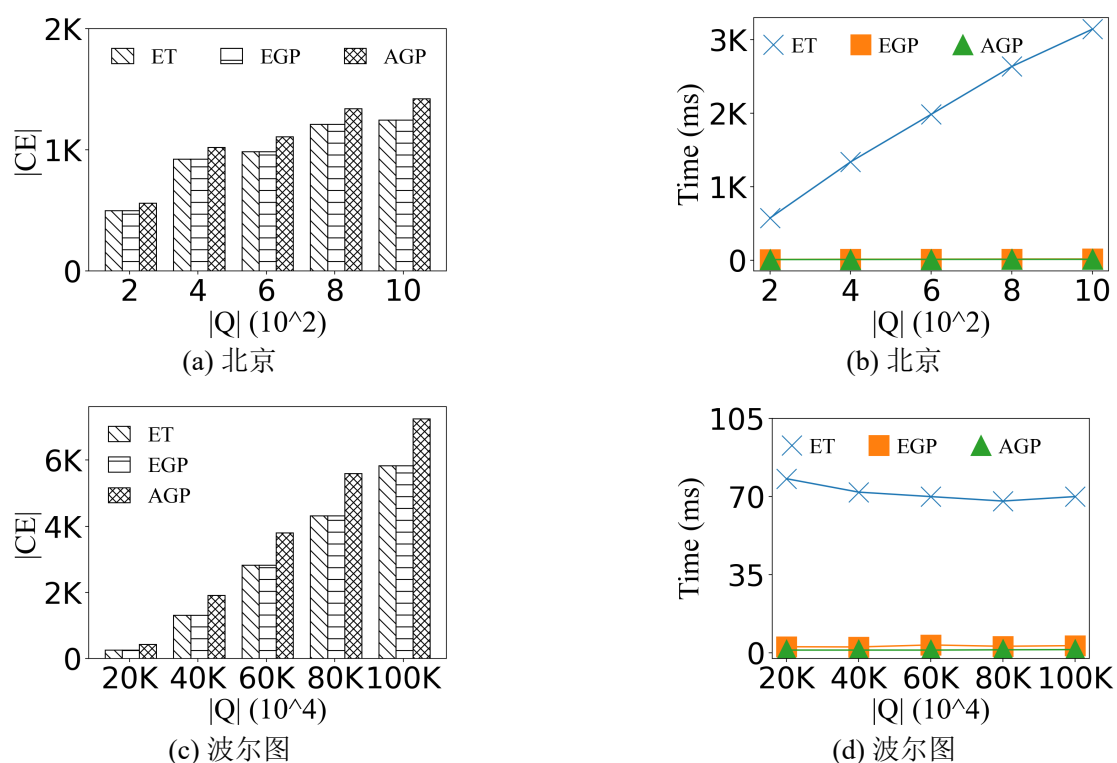


图 6-3 查询对象数量的影响

**6.6.2.0.4 近似算法的评估** 图 ?? 展示了在改变查询对象数量  $|Q|$  和社交距离阈值  $\epsilon$  时, AGP 算法的准确率表现。在北京数据集上, AGP 算法能够保持较高的准确率, 并且在  $|Q|$  和  $\epsilon$  增加时未表现出明显的性能退化。相比之下, 在波尔图数据集上, 随着这些参数的增大, 算法的准确率有所下降, 这可能与不同数据集之间的分布特性和移动模式差异有关。

**6.6.2.0.5 预检查策略的评估** 我们将不采用预检查策略的 EGP 方法记为  $EGP*PC$ 。如图 ?? 所示, 在改变查询对象数量的情况下, EGP 在北京和波尔图数据集上的 CPU 时间始终低于  $EGP*PC$ 。这一结果清楚地表明, 所提出的预检查策略能够有效减少不必要的计算开销, 从而提升整体执行效率。

## 6.7 本章小结

我们提出并研究了一种新的暴露搜索问题, 用于在轨迹位置流中发现暴露事件 (CES 问题)。针对该问题, 我们提出了两种高效算法, 并设计了相应的预检查策略以提升整体计算效率。大量实验结果表明, 所提出的方法在效率和有效性方面均表现优异, 且预检查策略能够有效避免不必要的计算开销。

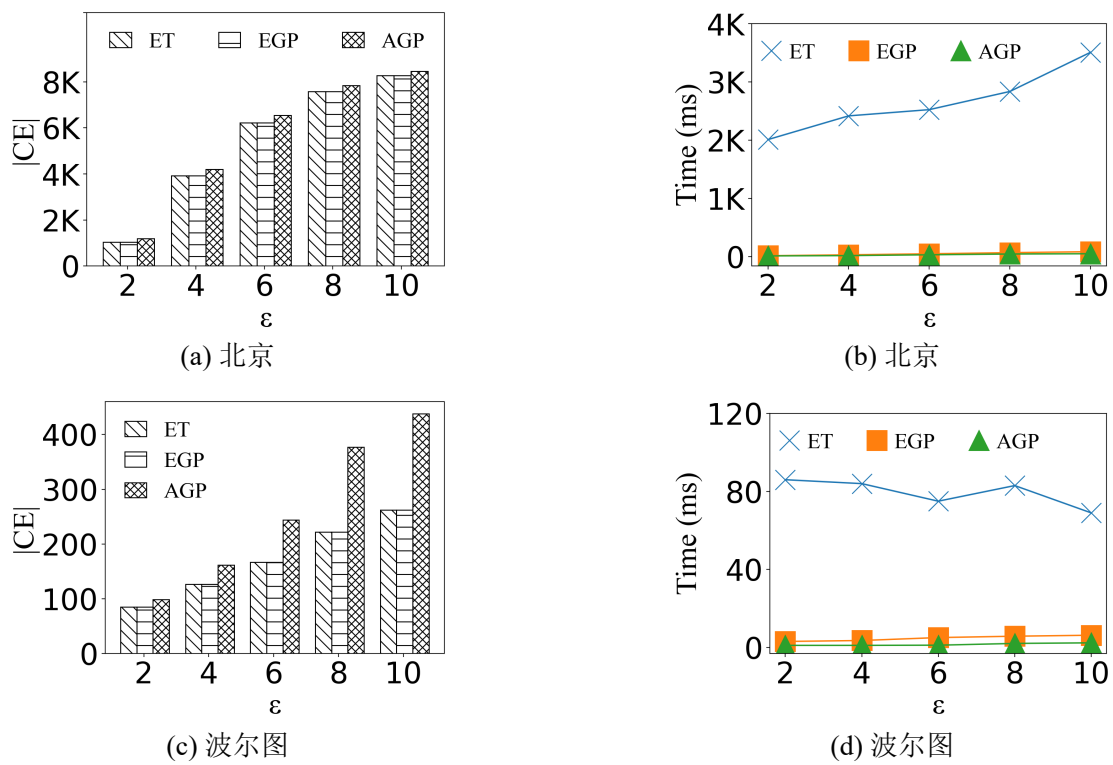


图 6-4 距离阈值  $\epsilon$  的影响

## 第七章 一种面向时空运动范围的轨迹相似性连接方法

### 7.1 引言

随着智能手机等 GPS 设备的普及，对移动对象的查询问题受到了广泛关注，相关研究涵盖了连接 (join)、范围查询、 $k$ NN 查询、相似性查询等多种类型。然而，由于采样频率不一致、位置采样可能存在误差，以及相邻采样点之间位置不可观测等因素，这类查询面临着诸多挑战。现有的相似性连接方法通常依赖于离散的位置采样点，难以刻画移动过程中的不确定性，从而可能遗漏具有实际意义的交互行为。

为了解决上述局限性，本文提出了一种新的方法——Intersection Similarity Join (IS-Join)，该方法并非基于位置邻近性，而是根据对象运动范围的重叠程度来识别对象对。我们将运动范围定义为对象在给定时间段内可能经过的空间区域，并进一步提出了一种交集相似性度量来量化不同对象运动范围之间的重叠程度。

为了高效处理 IS-Join 查询，我们设计了一种带有重划分策略的 Hybrid Ball-tree 索引结构，以实现可扩展的候选对象过滤。此外，我们还引入了预检查 (pre-checking) 和剪枝 (pruning) 技术，以进一步降低计算开销。基于两个真实轨迹数据集的大量实验结果表明，IS-Join 在性能上显著优于精心设计的对比方法，运行时间最多可降低至原来的  $1/3$  (即实现最高约  $3\times$  的加速)。该研究为城市出行分析、交通监测、野生动物追踪以及接触追踪等应用场景开辟了新的可能性。

#### 7.1.1 研究背景与意义

相似移动对象对的匹配问题，通常称为相似性连接 (similarity join)，是数据管理中的一项基础功能。该领域的现有研究工作 [1-4] 通常将移动对象建模为一系列带时间戳的位置采样点，并基于这些采样点之间的空间邻近性来度量对象间的相似性。

然而，在真实场景中，出于隐私保护、商业因素、数据缺失以及部署成本高昂等原因，移动对象往往只能以较低的采样频率进行观测 [5]，这使得相邻观测之间的位置信息不可获取。为了引入更高的灵活性，已有研究通过最小外接矩形 (minimum bounding rectangle, MBR) 来表示对象，并采用基于速度的表示模型来刻画其运动行为 [6-8]。另一类方法 [9] 则利用分段聚合近似 (piecewise aggregate approximation, PAA) [10] 将轨迹表示为符号序列，即将轨迹片段映射为来自预定义字母表的符号。

尽管上述方法在一定程度上提升了建模能力，现有表示方式仍然难以刻画采样点之间的运动过程，这一问题在稀疏采样轨迹场景下尤为突出。由此可能导致本应满足条件的结果对象对被错误地遗漏。因此，如何有效建模移动对象之间的运动交集关系，以及在海量移动对象数据下如何高效地发现所有交集，仍然是一个有待解决的开放性挑战。

在此背景下，我们提出了面向移动对象的交集相似性连接（Intersection Similarity Join, IS-Join）问题，通过引入运动范围来刻画对象在相邻观测样本之间可能访问的位置。具体而言，我们将对象的运动范围定义为其在某一时间点或给定时间区间内可能到达的空间区域。两个对象之间的交集相似性由其运动范围内特征的重叠程度来确定。

给定表 `MovingObject` 中的一组对象以及相似性阈值  $\theta$ ，我们以类似 SQL 的形式定义 IS-Join，如下所示<sup>①</sup>。

```
SELECT o.id, o'.id
FROM MovingObject AS o, MovingObject AS o'
WHERE o.id < o'.id AND IS(o, o', I) ≥ θ
```

每个对象  $o$  由一个  $id$  属性、两个位置坐标属性以及两个时间戳属性来表征，其中时间戳表示对象访问对应位置的时间。条件  $o.id < o'.id$  用于避免重复比较。函数  $IS(o, o', I)$  用于度量对象  $o$  与  $o'$  在时间区间  $I$  内基于各自运动范围中特征重叠程度的交集相似性。参数  $\theta$  为预先设定的相似性阈值。

IS-Join 问题面向多种应用场景，例如疾病防控、轨迹数据清洗、拼车推荐以及交通拥堵预测等 [22]。下面我们将讨论若干潜在的应用场景。

**场景 1 —— 城市交通监测。**特征集合可以包括道路路段、交叉路口以及交通信号灯等要素。交集相似性刻画了两辆车辆经历相似交通状况的可能性，有助于拥堵预测与交通优化。对于兴趣点（POI）和道路交叉口等点状特征，我们为每一个点分配一个唯一的 ID。为了将类似的方法应用于道路路段，我们将道路划分为等长度的子路段，并为每个子路段分配唯一的 ID。

**场景 2 —— 野生动物迁徙。**所定义的运动范围通过考虑栖息地、水源以及迁徙路线等特征，有助于识别共享的迁徙通道。此外，运动范围在节能的数据采集与准确的交互检测之间提供了一种折中方案 [23]。

**场景 3 —— 接触追踪。**所定义的运动范围能够对潜在的接触事件进行估计，从而提升流行病监测系统的有效性。交集相似性可以帮助用户评估其感染风险，这在新冠疫情期间对于降低感染传播具有重要价值 [24]。

<sup>①</sup> 为便于表示，我们以自连接（self-join）的形式定义 IS-Join。需要注意的是，本文提出的方法同样支持非自连接（non-self join）场景。

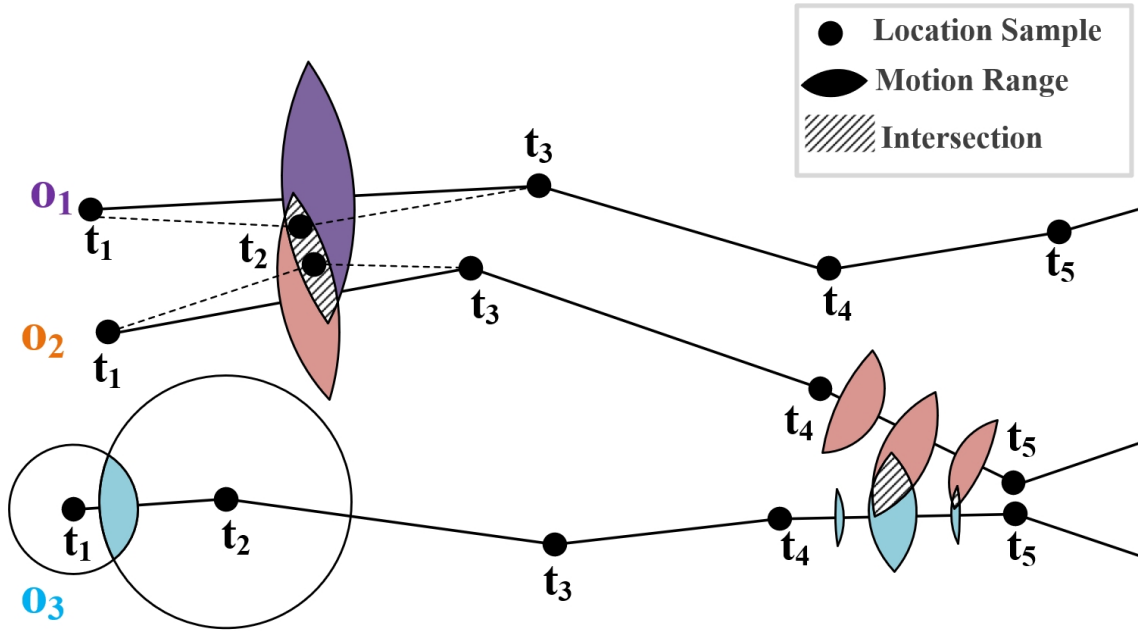


图 7-1 IS-Join 问题示例

示例。如图 ?? 所示，考虑三个移动对象  $o_1, o_2$  和  $o_3$ ，它们在时间戳  $t_1$  至  $t_5$  之间均具有位置采样点。为简化说明，假设这些位置采样在时间上是对齐的。对象  $o_1$  和  $o_2$  在  $t_2$  时刻的位置采样要么缺失，要么噪声过大而无法使用。如果仅通过判断位置采样之间的距离是否小于给定阈值来执行相似性连接，我们可能会错误地认为在时间区间  $[t_1, t_3]$  内  $(o_1, o_2)$  不是一个满足条件的结果对，尽管它们在  $t_2$  时刻的实际距离可能低于该阈值。

通过考虑对象的运动范围，可以避免这种错误排除。以  $o_3$  在  $t_1$  和  $t_2$  时刻的位置为例， $o_3$  在时间点  $t \in [t_1, t_2]$  内的运动范围为一个透镜形区域，其具体定义将在第 ?? 节中给出。对象  $o_1$  和  $o_2$  在时间点  $t \in (t_1, t_3)$  内的运动范围分别以紫色和棕色的透镜形区域表示。可以观察到， $o_1$  与  $o_2$  的运动范围在阴影区域内发生重叠，这表明它们可能存在潜在的位置交集。

接下来，我们关注时间区间  $[t_4, t_5]$ 。图中展示了  $o_2$  和  $o_3$  在区间  $[t_4, t_5]$  内若干离散时间点上的一系列运动范围。区间  $[t_4, t_5]$  的交集相似性被定义为该区间内各离散时间点交集相似性的平均值，具体定义将在后文的问题描述中进一步说明。给定一个时间区间和相似性阈值，IS-Join 问题旨在识别并返回在该时间区间内交集相似性大于或等于所给阈值的所有对象对。

据我们所知，这是首个在移动对象相似性连接研究中显式引入运动范围的工作。为了使 IS-Join 在实际中具有可行性，其算法必须在海量数据集上具备高效性和可扩展性。然而，IS-Join 问题的求解面临两个关键挑战。首先，在一个连续时间区间内计算精确的交集相似性在实际中是不可行的，因为这需要处理无限多个

时间点。其次，现有的连接算法主要针对其他类型的查询而设计，无法直接支持 IS-Join。实验结果表明，即使在小规模数据集上，简单地改造这些方法也会导致严重的效率问题。为此，我们提出了一种由时间区间运动范围引导的解决方案。具体而言，我们设计了一种基于 HBall-tree 的连接算法，结合有效的重划分与剪枝技术，相较于精心设计的基线方法，平均运行时间可降低约 3 倍。

本文的主要贡献总结如下。

- 我们提出了一种新的问题——交集相似性连接（Intersection Similarity Join, IS-Join），并研究了可用于求解该问题的现有技术。不同于以往基于离散位置采样点邻近性来识别相似对象对的方法，本文首次识别运动范围感知相似性超过给定阈值的对象对。
- 我们提出了一种由时间区间运动范围引导的高效 IS-Join 求解方案。为此，本文设计了一种混合 Ball-tree（Hybrid Ball-tree, HBall-tree）索引结构及其配套的剪枝增强技术。HBall-tree 能够高效地识别候选对象对，而剪枝增强技术则在处理早期阶段剔除不满足条件的对象对，从而显著提升查询性能。
- 我们在两个真实世界数据集上进行了大量实验，对所提出的解决方案进行了深入分析，并验证了该方法在性能上显著优于精心设计的基线方法。

## 7.1.2 相关工作

### 7.1.3 移动对象索引

移动对象通常被表示为带时间戳的位置采样点 [??]。R-tree [??] 和 KD-tree [??] 是支持高效移动对象查询的两类经典索引结构。这类索引中的每个内部节点对应一个矩形区域，其子树包含位于该区域内的数据点。此外，M-tree [??] 和 Ball-tree [?] 被设计用于高效处理圆形区域内的查询。Omohundro [?] 通过比较多种 Ball-tree 的构建方法，研究了构建时间与树结构质量之间的权衡关系。为提升索引结构的平衡性，Dolatshah 等人 [?] 利用主成分分析来改进空间划分方式，从而确定更优的划分轴。另一项相关研究 [?] 提出了面向移动对象的广义接触搜索查询。Zhang 等人 [?] 研究了接触相似性查询，并提出了用于索引的“UTM-tree”。然而，现有工作尚未基于运动范围来研究接触追踪问题，而这一点正是本文所关注的核心内容。

### 7.1.4 轨迹相似性连接

大量研究致力于轨迹相似性搜索与连接问题 [????]。在这些研究中，Shang 等人 [?] 采用线性组合的方式融合空间相似性与时间相似性。为降低系统



负载，已有工作提出了基于速度的表示方法 [22]。Ding 等人 [23] 提出了一种数据驱动的方法，利用真实历史轨迹数据挖掘实际可达区域来刻画问题。为了支持更灵活的轨迹相似性定义，Bakalov 等人 [24] 采用分段聚合近似 (Piecewise Aggregate Approximation, PAA) [25] 将轨迹表示为符号序列，并将 PAA 片段映射为来自预定义字母表的符号。他们构建了一种紧凑索引，用于高效评估近似连接，并通过后过滤步骤在加载最少轨迹数据的情况下对查询结果进行精化。Chen 等人 [26] 提出了一个通用的轨迹连接框架，涵盖距离连接和  $k$  最近邻连接。此外，Tampakis 等人 [27] 提出了一种子轨迹连接查询，用于检索所有“最大化”的相似轨迹片段，从而识别“最大匹配”的子轨迹。尽管上述研究将相似性概念扩展到了单纯位置采样邻近性之外，但据我们所知，尚无工作研究基于运动范围的交集相似性连接问题，因此其现有解决方案并不适用于本文所提出的 IS-Join 问题。

### 7.1.5 研究内容与挑战

### 7.1.6 研究成果与贡献

## 7.2 问题描述

在本节中，我们首先定义移动对象的运动范围以及移动对象之间的交集相似性。随后，引入放宽交集相似性的概念，并对交集相似性连接问题进行形式化定义。本文中使用的符号汇总于表 7.1。

**定义 7.1 (带时间戳的位置采样)** 移动对象的一个带时间戳的位置采样表示为  $s = ([x, y], t)$ ，其中  $[x, y]$  表示空间坐标， $t$  表示该位置被访问的时间戳。

**定义 7.2 (时间点运动范围)** 给定移动对象  $o$  的两个位置采样点  $s_i = ([x_i, y_i], t_i)$  和  $s_j = ([x_j, y_j], t_j)$ ，以及最大速度  $v_{max}$ ，对象  $o$  在时间  $t$  (其中  $t \in [t_i, t_j]$ ) 处的运动范围记为  $MR(o, t)$ ，其计算方式如公式 (7-1) 所示。函数  $dist(\cdot)$  表示两个位置之间的距离度量。该运动范围呈透镜形状，如图 7.1 所示，其两个半径  $r_i$  和  $r_j$  分别为  $v_{max} \cdot (t - t_i)$  和  $v_{max} \cdot (t_j - t)$ 。

$$MR(o, t) = \{(x, y) \mid dist((x, y), (x_i, y_i)) \leq v_{max} \cdot (t - t_i) \wedge dist((x, y), (x_j, y_j)) \leq v_{max} \cdot (t_j - t)\} \quad (7-1)$$

时间点运动范围的建模方式与已有研究保持一致 [22]。本文采用最大速度进行建模，以确保所估计的运动范围能够完全覆盖对象的真实运动轨迹。特别地，在采样时间戳处，对应的运动范围即退化为该时间戳的位置采样点本身。对于采样时间间隔异常较长（例如达到数小时）的对象，不在本文的分析范围之内。

**定义 7.3 (运动范围特征)** 设  $\mathcal{F}(o, t)$  表示位于运动范围  $MR(o, t)$  内的一组特

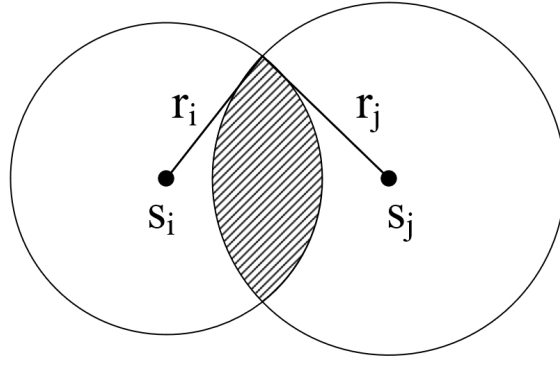


图 7-2 Illustration of time-point motion range

定特征。示例包括运动范围的面积、范围内的兴趣点、经过的道路路段，以及地理空间区域或环境因素等。

**定义 7.4（时间点交集相似性）** 对象  $o$  与  $o'$  在时间  $t$  处的时间点交集相似性记为  $IS(o, o', t)$ ，其定义如公式 ?? 所示，用于度量二者特征集合之间的重叠程度。其中， $|\mathcal{F}(\cdot)|$  表示位于  $MR(o, t)$  内的特征集合大小， $\cap$  表示特征集合之间的交集，该交集可根据具体应用场景表示共享的兴趣点、共同经过的道路路段，或重叠的运动范围区域等。相似性取值范围为  $[0, 1]$ ，其中 1 表示特征集合完全重叠，0 表示不存在重叠。

$$IS(o, o', t) = \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o, t)|} \times \frac{|\mathcal{F}(o, t) \cap \mathcal{F}(o', t)|}{|\mathcal{F}(o', t)|} \quad (7-2)$$

时间点交集相似性的定义基于如下考虑。首先， $MR(o, t)$  描述了对象  $o$  在时间  $t$  处可能占据的位置范围，而  $\mathcal{F}(\cdot)$  则表示该运动范围内的相关特征集合。采用乘积形式意味着两个因子都必须足够大，交集相似性才会取得较高值，从而保证只有在运动范围重叠程度高且特征相似性强这两个条件同时满足时，相似性得分才会较高。相比之下，经典的 Jaccard、取平均或取最小值等方法难以有效平衡运动范围重叠与特征相似性这两个方面。因此， $IS(o, o', t)$  通过评估二者特征集合的重叠情况，刻画了对象  $o$  与  $o'$  之间发生交互的可能性。

**定义 7.5（时间区间交集相似性）** 给定起始时间为  $t_s$ 、结束时间为  $t_e$  的时间区间  $I$ ，我们将对象  $o$  与  $o'$  在区间  $I$  上的交集相似性定义为该区间内时间点交集相似性的平均值，如公式 ?? 所示。

$$IS(o, o', I) = \frac{\int_{t_s}^{t_e} IS(o, o', t) dt}{t_e - t_s} \quad (7-3)$$

计算时间区间交集相似性并非易事，因为由于时间区间的连续性，需要在无限多个时间点上评估时间点交集相似性。为在实际应用中对大规模移动对象数据提供一种可扩展的近似计算方式，并在不显著损失一般性的前提下，本文提出了一种放宽的定义。

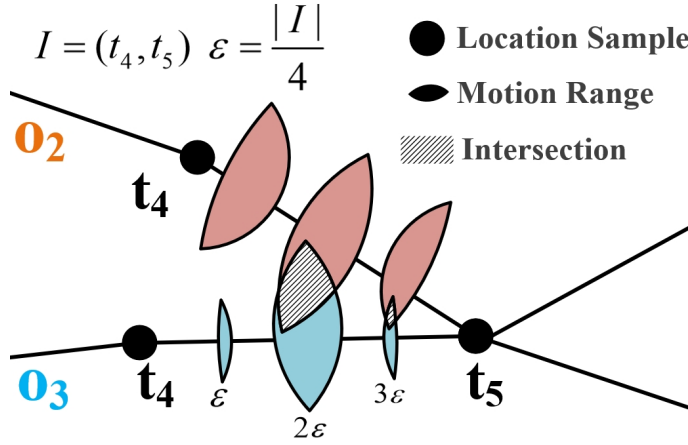


图 7-3 时间区间交集相似性的示意图

**定义 7.6** ( $\epsilon$ -放宽的时间区间交集相似性) 给定一个较小的常数  $\epsilon$ ，我们对时间区间  $I = [t_s, t_e]$  进行  $\epsilon$ -采样，通过均匀采样得到一组时间点  $T^\epsilon = \{t_1^\epsilon, \dots, t_m^\epsilon\}$ ，其中  $t_1^\epsilon = t_s$ ， $t_m^\epsilon = t_e$ ，且满足  $t_{i+1}^\epsilon - t_i^\epsilon = \epsilon$ 。随后，我们将  $\epsilon$ -放宽的时间区间交集相似性定义为对象  $o$  与  $o'$  在所有采样时间点  $t^\epsilon \in T^\epsilon$  上的时间点交集相似性的平均值，记为  $IS(o, o', I, \epsilon)$ ，其计算方式如公式 ?? 所示。

$$IS(o, o', I, \epsilon) = \frac{\sum_{i=1}^m IS(o, o', t_i^\epsilon)}{m} \quad (7-4)$$

从本质上看，时间区间相似性的计算需要对多个时间点上的时间点交集相似性进行评估。在下文中，如无特殊说明，为表述简洁起见，我们使用“交集相似性”来指代  $\epsilon$ -放宽的交集相似性。当  $\epsilon$  趋近于 0 时， $IS(o, o', I, \epsilon)$  将收敛到  $IS(o, o', I)$ 。

**示例。** 图 ?? 给出了一个时间区间交集相似性计算的示例。考虑时间区间  $I = (t_4, t_5)$ ，令  $\epsilon = |I|/4$ ，则采样得到的时间集合为  $T^\epsilon = t_4, t_4 + \epsilon, t_4 + 2\epsilon, t_4 + 3\epsilon, t_5$ 。两个移动对象  $o_2$  和  $o_3$  在  $T^\epsilon$  中各采样时间点处均具有对应的时间点运动范围。在时间点  $t_4$  和  $t_4 + \epsilon$  处，两者的运动范围不存在交集，因此  $IS(o_2, o_3, t_4) = 0$  且  $IS(o_2, o_3, t_4 + \epsilon) = 0$ 。在  $t_5$  时刻， $o_2$  与  $o_3$  的位置采样点相同，从而得到  $IS(o_2, o_3, t_5) = 1$ 。因此，区间  $I$  上的  $\epsilon$ -放宽时间区间交集相似性可计算为  $(0 + 0 + IS(o_2, o_3, t_4 + 2\epsilon) + IS(o_2, o_3, t_4 + 3\epsilon) + 1)/5$ 。其中， $IS(o_2, o_3, t_4 + 2\epsilon)$  和  $IS(o_2, o_3, t_4 + 3\epsilon)$  的取值依据公式 ?? 计算，该公式能够适应不同应用场景下的具体需求。

我们在定义 ?? 中对交集相似性连接问题进行形式化描述。

**定义 7.7 (IS-Join)** 给定对象集合  $\mathcal{A}$  和  $\mathcal{B}$ 、带有放宽划分参数  $\epsilon$  的查询时间区间  $I$ ，以及相似性阈值  $\theta$ ，交集相似性连接 (Intersection Similarity Join, IS-Join) 旨在从这两个集合中找出所有交集相似性不小于  $\theta$  的对象对所构成的集合  $\mathcal{P}$ ，即对于任意  $(o_i, o_j) \in (\mathcal{A} \times \mathcal{B}) \setminus \mathcal{P}$ ，均有  $(IS(o_i, o_j, I, \epsilon) < \theta)$ 。

### 7.3 运动范围引导的匹配方法

为高效地回答 IS-Join 查询，我们提出了一种由时间区间运动范围引导的匹配方法。在时间区间阶段，我们首先进行预检查，以判断两个对象的时间点运动范围之间是否存在交集。具体而言，我们定义时间区间运动范围  $MR(o, I)$ ，用于刻画对象  $o$  在区间  $I$  内可能到达的所有位置。如果  $MR(o, I) \cap MR(o', I) = \emptyset$ ，则可推出  $IS(o, o', I, \epsilon) = 0$ ，从而对对象对  $(o, o')$  进行剪枝。为高效判定  $MR(o, I) \cap MR(o', I) = \emptyset$ ，我们提出了一种新的混合 Ball-tree (Hybrid Ball-tree, HBall-tree) 结构。同时，我们引入了一种基于交集相似度上界的剪枝增强方法，以进一步剔除不满足条件的对象对，从而细化整体处理流程。在精化阶段，计算运动范围内特征的交集以度量交集相似度。我们首先介绍基于时间区间运动范围的预检查策略，随后给出基于混合 Ball-tree 的连接算法 (HBJ-Join)。

#### 7.3.1 时间区间预检查

本节介绍时间区间运动范围的概念，并给出具有理论保证的预检查策略。

**定义 7.8 (时间区间运动范围)** 给定移动对象  $o$  的两个带时间戳的位置样本  $s_i = ([x_i, y_i], t_i)$  和  $s_j = ([x_j, y_j], t_j)$ ，以及在  $t_i$  与  $t_j$  之间的最大速度  $v_{max}$ ，对象  $o$  在时间区间  $I = [t_i, t_j]$  内的运动范围，记为  $MR(o, I)$ ，被定义为对象  $o$  在区间  $I$  内可能到达的空间区域。

定理 ?? 给出了计算  $MR(o, I)$  的方法。

**定理 7.1** 对象  $o$  的运动范围  $MR(o, I)$  是由公式 ?? 表示的一个椭圆区域。椭圆的中心点记为  $(x_c, y_c)$ ，由公式 ?? 计算得到；椭圆的旋转角度记为  $\alpha$ ，由公式 ?? 计算，其中  $a$  和  $b$  分别表示半长轴和半短轴。

$$\left[ \frac{(x - x_c) \cos \alpha + (y - y_c) \sin \alpha}{a} \right]^2 + \left[ \frac{(x_c - x) \sin \alpha + (y - y_c) \cos \alpha}{b} \right]^2 = 1 \quad (7-5)$$

$$\begin{aligned} x_c &= \frac{x_i + x_j}{2}, \quad y_c = \frac{y_i + y_j}{2}, \\ a &= \frac{v_{max} \cdot (t_j - t_i)}{2}, \\ b &= \frac{\sqrt{v_{max}^2 \cdot (t_j - t_i)^2 - (x_j - x_i)^2 - (y_j - y_i)^2}}{2} \end{aligned} \quad (7-6)$$

$$\alpha = \arctan\left(\frac{y_j - y_i}{x_j - x_i}\right) \quad (7-7)$$

**证明：**对象  $o$  在时间区间  $I$  内能够行进的最大距离为  $(t_j - t_i) \cdot v_{max}$ ，该结论

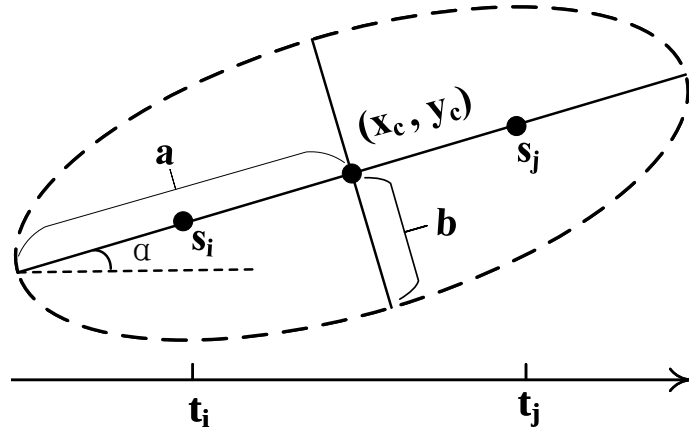


图 7-4 时间区间运动范围示例

同时适用于欧氏空间和路网空间。给定两个位置样本  $s_i$  和  $s_j$ ，对象  $o$  在某一具体时间点  $t$  的运动范围可由公式 ?? 定义。需要注意的是，椭圆上任意一点到其两个焦点的距离之和为常数  $2a$ 。当  $v_{max} \cdot (t_j - t_i) = 2a$  时，从  $t_i$  到  $t_j$  的所有可能到达位置构成一个以  $(x_i, y_i)$  和  $(x_j, y_j)$  为焦点的椭圆区域，如图 ?? 所示。我们对时间区间运动范围的建模方式与已有研究保持一致 [? ? ]。 ■

基于定理 ??，我们提出了一种预检查策略：若  $MR(o, I) \cap MR(o', I) = \emptyset$ ，则可以在早期阶段安全地剪枝对象对  $(o, o')$ ，从而避免对特征集合（例如 POI、运动范围面积等）进行不必要的交集计算。

**定理 7.2** 给定两个对象  $o$  和  $o'$ ，若  $MR(o, I) \cap MR(o', I) = \emptyset$ ，则  $IS(o, o', I, \epsilon) = 0$ 。

**证明：**对于对象  $o$  和  $o'$ ，如果它们在时间区间  $I$  内的时间区间运动范围不相交，则在  $I$  内任意时间点上， $o$  与  $o'$  的时间点运动范围都不可能发生重叠。由于与各自时间点运动范围相关的特征集合  $P(o, t)$  和  $P(o', t)$  在整个区间  $I$  内均不存在交集，因此在区间  $I$  上的交集相似度为 0。 ■

为了判断两个时间区间运动范围是否相交，一种直接的方法是计算它们的两两距离。具体而言，若距离  $dist(MR(o, I), MR(o', I))$  大于  $MR(o, I).a + MR(o', I).a$ ，则两个运动范围不相交。其中， $dist(MR(o, I), MR(o', I))$  表示  $MR(o, I)$  与  $MR(o', I)$  中心点之间的距离， $a$  表示运动范围的半长轴。这种直接方法在进行一对一比较时需要  $O(\mathcal{A} \times \mathcal{B})$  的时间复杂度，计算代价较高。

“latex

### 7.3.2 基于 HBall-tree 的交集预检查

我们提出了一种混合 Ball-tree (Hybrid Ball-tree, HBall-tree)，这是一种面向高效预检查的改进型索引结构，并展示了其如何提升“过滤-精化”过程的整体效

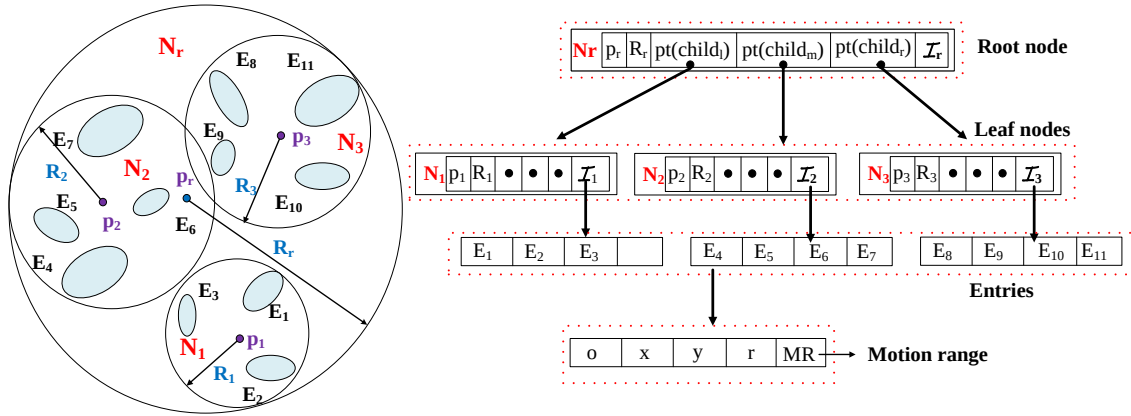


图 7-5 混合 Ball-tree 的示意图

率。

### 7.3.2.1 HBall-tree

我们选择 Ball-tree [?] 作为基础索引结构，这是因为相较于 R-tree [?] 和四叉树 [?] 等索引，Ball-tree 在处理类圆对象时表现更优 [?]。然而，当将 Ball-tree 应用于 IS-Join 查询时，仍然存在两个关键局限性。首先，Ball-tree 主要面向点或圆级别的最近邻搜索设计，无法直接支持时间区间运动范围的索引。其次，其划分过程未考虑数据点的分布情况，可能导致划分不均衡，从而形成高度不平衡的树结构 [?]。

为了解决第一个问题，我们将每个时间区间运动范围近似为一个圆 [?]。具体而言，该圆以时间区间运动范围的中心为圆心，其半径等于半长轴长度。该近似方式简化了时间区间运动范围的表示，使我们能够直接利用现有针对圆形对象设计的空间索引结构。其关键推论在于：如果两个近似圆不相交，则其对应的时间区间运动范围也必然不相交。该推论构成了我们过滤策略的基础，从而能够高效识别不可能发生交集的对象对。

为了解决第二个问题，我们提出了混合 Ball-tree (HBall-tree)，这是一种能够考虑数据分布特性的全新索引结构。图 ?? 给出了 HBall-tree 的示意图。每个时间区间运动范围被表示为  $E = \{o, x, y, r, MR(o, I)\}$ ，其中  $(x, y)$  和  $r$  分别表示近似圆的中心和半径， $o$  表示对象标识符。与原始 Ball-tree 每个非叶节点仅包含两个子节点不同，HBall-tree 通过一种重新划分 (repartition) 技术，使得每个父节点最多可以拥有三个子节点。每个节点的结构表示为  $N = \{p, R, pt(child_l), pt(child_m), pt(child_r), I\}$ ，其中  $p$  表示节点  $N$  所覆盖的时间区间运动范围的中心点， $R$  表示能够包围节点  $N$  中所有条目 (即圆) 的半径， $pt(\cdot)$  为指向子节点的指针， $I$  表示该节点中包含的时间区间运动范围集合。HBall-tree

中的插入策略、分裂策略以及枢轴点选择均遵循已有研究中的成熟方法 [?]。当某一子划分中的对象数量低于预定义阈值（即生成新子节点所需的最小对象数）时，子划分过程终止。

### 7.3.2.2 HBall-tree 的重新划分策略

直观上，更加平衡的树结构通常能够在搜索过程中减少节点访问次数。重新划分方法的整体思想如下：对于每一个初始划分，我们评估其两个子划分中的点数差异是否超过重新划分阈值  $\beta$  ( $\beta \in (0, 1)$ )。如果差异超过该阈值，则保留点数较少的子划分，并将点数较多的子划分进一步划分为两个新的子划分。考虑一个节点  $N$ ，初始时  $N.child_m = \emptyset$ 。当较大的子划分规模至少以比例  $\beta$  超过较小子划分时，即触发重新划分。例如，当  $|N.child_l.I| > |N.child_r.I|$ ，且  $\frac{|N.child_l.I| - |N.child_r.I|}{|N.child_r.I|} \geq \beta$ ，则触发重新划分。从理论上讲，较小的重新划分阈值会使初始子树更容易被重新划分，从而增加三叉节点的数量，使树结构更加平衡。然而，三叉节点数量的增加也可能导致节点访问次数上升，因此需要在树高与三叉节点数量之间进行权衡。

对于非叶节点，初始划分基于  $N.I$  中选取的两个最远点进行。具体而言，树构建过程中两个最远点的选取步骤如下：(i) 随机种子点：从  $N.I$  中随机选择一个点作为初始种子；(ii) 第一个最远点：在数据集中找到距离该种子点最远的点；(iii) 第二个最远点：在数据集中找到距离第一个最远点最远的点。设这两个最远点分别为  $N.child_l.p$  和  $N.child_r.p$ ，我们计算二者的中点，并在  $N.I$  中选取距离该中点最近的中心点，记为  $p_m$ 。点数较少的子划分被视为稳定划分，而我们对点数较多的子划分进行重新划分。具体地，若  $|N.child_l.I| > |N.child_r.I|$ ，则使用  $p_m$  和  $N.child_l.p$  将  $N.child_l.I$  中的运动范围重新划分为两个新的子划分，其中一个保留在  $N.child_l.I$ ，另一个分配给  $N.child_m.I$ ；反之，若  $|N.child_l.I| < |N.child_r.I|$ ，则使用  $p_m$  和  $N.child_r.p$  对  $N.child_r.I$  进行重新划分，一个子划分保留在  $N.child_r.I$ ，另一个分配给  $N.child_m.I$ 。最终，重新划分将原有的两个子划分扩展为三个子划分，新生成的子划分由  $N.pt(child_m)$  索引。该重新划分技术带来了两方面优势：(1) 降低树高与划分数目：通过提升平衡性，减少树的高度并最小化划分数目；(2) 提升搜索效率：更加平衡的树结构能够减少树节点访问次数，从而提高搜索效率。“

### 7.3.3 基于 HBall-tree 的 Join

我们提出了一种基于 HBall-tree 的 Join 算法。为了进一步提升效率，我们引入了一种剪枝增强方法，利用交集相似度的上界来进一步剔除不满足条件的对象对，从而优化整体处理流程。

### 7.3.3.1 剪枝增强策略

由于运动范围交集形状的不规则性，直接计算两个特征集合  $\mathcal{F}(o, t)$  与  $\mathcal{F}(o', t)$  的交集并非易事。给定时间点  $t$ ，若  $MR(o, t) \cap MR(o', t) \neq \emptyset$ ，当其中一个对象的运动范围完全被另一个对象覆盖时， $IS(o, o', t)$  取得其最大值。基于这一观察，公式 ?? 给出了  $IS(o, o', t)$  的一个上界。将公式 ?? 代入公式 ??，可得到公式 ??，用于估计  $IS(o, o', I, \epsilon)$  的上界。若  $IS(o, o', I, \epsilon)_{ub} < \theta$ ，则可以安全地剪枝对象对  $(o, o')$ 。

$$IS(o, o', t)_{ub} = \frac{\min(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)}{\max(|\mathcal{F}(o, t)|, |\mathcal{F}(o', t)|)} \geq IS(o, o', t) \quad (7-8)$$

$$IS(o, o', I, \epsilon)_{ub} = \frac{\sum_{i=1}^m IS(o, o', t_i^\epsilon)_{ub}}{m} \quad (7-9)$$

### 7.3.3.2 算法细节

算法 ?? 给出了基于 HBall-tree 的 Join 算法 (HBJ-Alg) 的伪代码。该算法的输入包括两个移动对象集合、HBall-tree 的根节点、重分区阈值以及交集相似度阈值；输出结果为交集对象对集合  $\mathcal{R}$ 。初始时， $\mathcal{R}$  为空（第 1 行）。对于每一个  $o \in \mathcal{A}$ ，我们使用列表  $\mathcal{L}$  存储可能与  $MR(o, I)$  发生交集的候选节点，初始时  $\mathcal{L}$  仅包含根节点  $N_r$ （第 3 行）。HBall-tree 通过插入对象集合  $\mathcal{B}$  的时间区间运动范围构建完成。

当  $\mathcal{L}$  非空时，算法调用  $\mathcal{L}.pop()$  取出一个节点  $N$ （第 5 行）。随后，在“圆级别”执行基于 HBall-tree 的过滤，以识别潜在的交集候选（第 6–9 行）。具体而言，我们检索 HBall-tree 中满足  $dist(o, N_c) \leq R + r_q$  的节点，从而判断  $MR(o, I)$  与  $N_c$  之间是否可能存在交集。若  $N$  为非叶节点，则检查其子节点。对于每个子节点  $N_c$ ，若对象  $o$  与  $N_c$  的距离不大于二者半径之和，则将  $N_c$  压入优先队列（第 8–9 行）。

若  $N$  为叶节点，则遍历  $N.\mathcal{I}$  中的所有条目。对于每个条目  $E$ ，计算  $(E.x, E.y)$  与  $(MR(o, I).x_c, MR(o, I).y_c)$  之间的欧氏距离  $dist(o, E)$ 。若  $dist(o, E) > E.r + o.r$ ，则可安全剪枝  $(o, E.o)$ （第 12–13 行）。否则，继续执行进一步的剪枝操作，即剪枝增强策略（第 14–15 行）。仅对保留下来的对象对，才进行精确的交集相似度计算，即利用公式 ?? 计算多个时间点的交集相似度（第 16–17 行）。最终返回结果集合  $\mathcal{R}$ （第 18 行）。

### 7.3.3.3 时间复杂度

HBall-tree 的构建时间复杂度为  $O(|\mathcal{B}| \times \log |\mathcal{B}|)$ 。预检查阶段的时间复杂度为  $O(|\mathcal{A}| \times \log |\mathcal{B}|)$ 。设  $\mathcal{C}'$  为候选集合，则候选精化阶段的时间复杂度为  $O(|\mathcal{C}'| \times |I|/\epsilon \times$



---

**算法 7-1:  $HBJoin(\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, \beta, \theta)$** 


---

**输入:** 两组移动对象,  $\mathcal{A}$  和  $\mathcal{B}$ ;  
 HBall-tree 的根节点  $N_r$ ;  
 时间间隔  $I$ ;  
 松弛参数  $\epsilon$ ;  
 重分区阈值  $\beta$ ;  
 交集相似度阈值  $\theta$ 。

**输出:** IS-Join 结果,  $\mathcal{R}$ 。

```

1  初始化:  $\mathcal{R} \leftarrow \emptyset$ ;
2  for  $o \in \mathcal{A}$  do                                     /* 遍历所有移动对象 */
3       $\mathcal{L} \leftarrow \{N_r\}$ ;
4      while  $\mathcal{L} \neq \emptyset$  do                       /* BFS 遍历 HBall-tree */
5           $N \leftarrow \mathcal{L}.pop()$ ;
6          if  $N$  是非叶子节点 then                     /* 非叶子节点分支 */
7              for  $N_c \in N.pt(child)$  do             /* 遍历子节点 */
8                  if  $dist(o, N_c) \leq N_c.R + o.r$  then
9                       $\mathcal{L}.push(N_c)$ ;
10         else                                         /* 叶子节点处理 */
11             for  $E \in N.I$  do                       /* 遍历叶子节点对象 */
12                 if  $dist(o, E) > E.r + o.r$  then
13                      $prune(o, E.o)$ ;                // 预检查策略
14                 if  $IS(o, E.o, I, \epsilon)_{ub} < \theta$  then
15                      $prune(o, E.o)$ ;                // 剪枝增强策略
16                 if  $IS(o, E.o, I, \epsilon) \geq \theta$  then
17                      $\mathcal{R} \leftarrow \{\mathcal{R}, (o, E.o)\}$ ; // 精炼步骤
18 return  $\mathcal{R}$ ;
    
```

---

$p$ )。因此, HBJ-Alg 的总体时间复杂度为

$$O((|\mathcal{A}| + |\mathcal{B}|) \times \log |\mathcal{B}| + |\mathcal{C}'| \times |I|/\epsilon \times p).$$

HBJoin-Alg 得益于增强的剪枝策略, 显著减少了需要进行精化计算的候选对象对数量。

## 7.4 实验研究

### 7.4.1 实验设置

#### 7.4.1.1 数据准备

实验研究<sup>②</sup>在两个真实世界的轨迹数据集上进行。第一个数据集 Geolife [??] 包含来自 182 名用户、历时四年的 17,621 条轨迹，涵盖了多种户外移动行为，包括日常通勤以及购物、骑行等休闲活动。第二个数据集 Porto [?] 包含在葡萄牙波尔图市 19 个月内记录的超过 170 万条出租车轨迹。Porto 数据集的轨迹以 15 秒为采样间隔记录，而 Geolife 的采样频率则不固定。时间戳被限定在 24 小时范围内，而不考虑具体日期，因为大多数城市交通出行行为具有日常重复性。我们将对象的最大速度  $v_{max}$  定义为其在给定时间区间内观测到的最高速度。确定  $v_{max}$  的真实取值或最优设置超出了本文的研究范围。为保证实际应用相关性，我们剔除了采样时间间隔过大（超过一小时）的极端轨迹，因为这类轨迹在现实应用中实用性有限，因此不纳入本研究。我们注意到，在隧道等环境中 GPS 信号可能较弱甚至完全丢失，从而导致异常位置或较低的采样率。尽管对这类异常值进行显式建模超出了本文的研究范围，但我们的数据预处理流程通过基于经度、纬度、轨迹长度和速度等约束过滤异常轨迹，从而在一定程度上缓解了这些影响。预处理后，Geolife 数据集中保留了 18,670 条轨迹中的 12,015 条有效轨迹，Porto 数据集中保留了 1,710,670 条轨迹中的 1,392,385 条。随后，从数据集中随机选取两个包含相同数量移动对象的对象组。

#### 7.4.1.2 对比算法

我们将所提出的方法与一种设计良好的基于 M-tree 的连接算法 [?]（见附录）进行对比，记为 MBJ-Alg，作为基线方法。为评估剪枝增强策略的影响，我们还测试了不包含剪枝增强策略的 HBJ-Alg，记为 HBJ-Alg#。在效率评估方面，我们同时衡量剪枝率和 CPU 时间。剪枝率定义为  $1 - |C|/|O|^2$ ，其中  $C$  表示候选集。CPU 时间表示算法每一轮的平均运行时间。此外，我们还评估了不采用重新划分策略的 HBJ-Alg，记为 HBJ-Alg\*。

#### 7.4.1.3 参数设置

参数设置如表 ?? 所示。M\*-tree 和 HBall-tree 的叶节点容量均设置为 10。默认情况下，重新划分阈值  $\alpha$  设为 0.5。在实验中，特征集合  $\mathcal{F}$  由随机生成的兴趣点 (POIs) 构成。具体而言，我们首先确定所有轨迹的最小和最大经度与纬度值，在

② 代码已开源：<https://github.com/likemelike/IS-Join>

该空间范围内使用均匀分布随机生成固定数量的 POIs。默认情况下，在 Porto 数据集的坐标范围内生成 100,000 个 POIs，在 Geolife 数据集中生成 600,000 个 POIs。默认距离度量采用欧氏距离。所有方法均使用 Java 实现，并在配备 AMD Ryzen 5 CPU (3.6GHz) 和 32GB 内存的 Ubuntu 系统上进行评测，采用单线程执行。除非另有说明，实验结果均为 20 次独立实验的平均值，每次实验使用不同的轨迹数据和特征集。

## 7.4.2 Performance Evaluation

### 7.4.2.1 剪枝有效性评估

首先，我们在默认参数设置下考察各算法的剪枝有效性。实验结果如表 ?? 所示。通过比较 MBJ-Alg 与 HBJ-Alg 的候选集规模和剪枝率可以发现，在 Geolife 和 Porto 数据集上，HBJ-Alg 的候选集规模分别仅为 MBJ-Alg 的 0.5% 和 2.3%。这些结果表明，所提出的方法能够显著压缩搜索空间。

接下来，我们在不同参数设置下进一步评估所提出算法的性能。需要说明的是，在改变某一个参数时，其余参数均保持为默认值。

### 7.4.2.2 移动对象基数的影响

我们研究了不同移动对象基数  $|O|$  对算法性能的影响。直观上，较大的  $|O|$  会导致需要处理更多的时间区间运动范围，因此所有算法的运行时间都将随之增加。首先，我们评估了 MBJ-Alg、HBJ-Alg 以及 HBJ-Alg\* 的索引构建时间，结果如图 ?? 所示。可以清楚地看到，MBJ-Alg 的构建开销明显高于 HBJ-Alg。与 HBJ-Alg\* 相比，HBJ-Alg 表现出相近且略有提升的性能。这一改进主要得益于 HBJ-Alg 中的再划分技术：尽管在触发再划分时会引入额外计算开销，但该技术能够减少总体分区数量，从而在整体上有助于降低构建时间。

接下来，我们分析了所提出算法在 IS-Join 查询中的总运行时间。随着移动对象数量的增加，由于交集候选对数量增多，计算负载显著上升，所有算法的时间复杂度均随之提高。从图 ?? 可以观察到，随着数据规模的扩大，MBJ-Alg 的 CPU 时间显著增加。例如，在 Geolife 数据集包含 12K 个移动对象、Porto 数据集包含 50K 个移动对象时，MBJ-Alg 的查询时间分别超过 3 秒和 30 秒，计算开销较大。相比之下，HBJ-Alg 和 HBJ-Alg# 始终优于 MBJ-Alg，尤其在大规模数据集上表现出更高的效率。值得注意的是，HBJ-Alg 的 CPU 时间几乎没有明显退化，充分体现了所提出方法的良好可扩展性。此外，与 HBJ-Alg# 相比，HBJ-Alg 的运行时间降低了约 30%，突出了剪枝增强策略所带来的性能提升。具体而言，在 Geolife 数

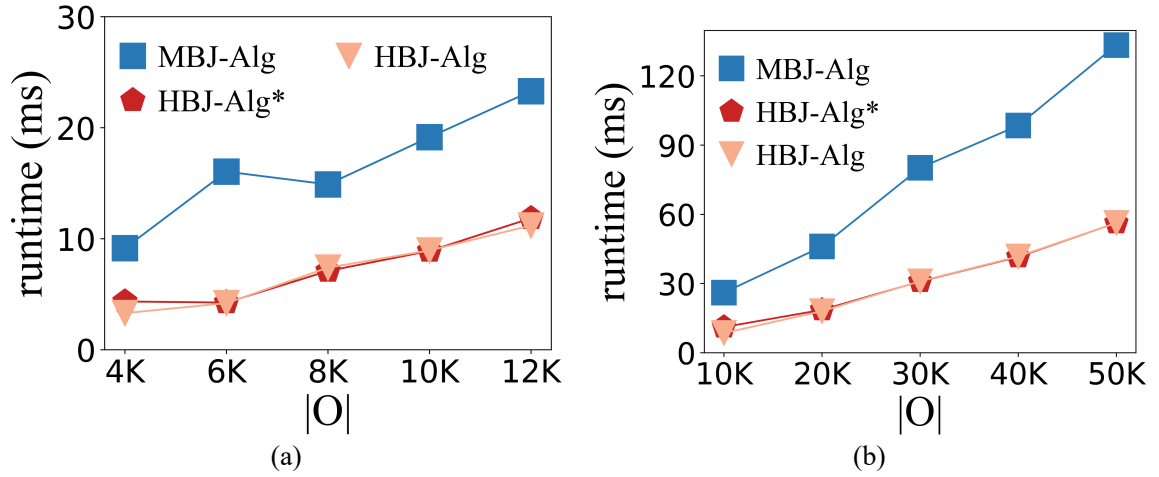


图 7-6 不同  $|O|$  下的构建时间影响 ?? :Geolife 数据集下的实验; ?? :Porto 数据集下的实验

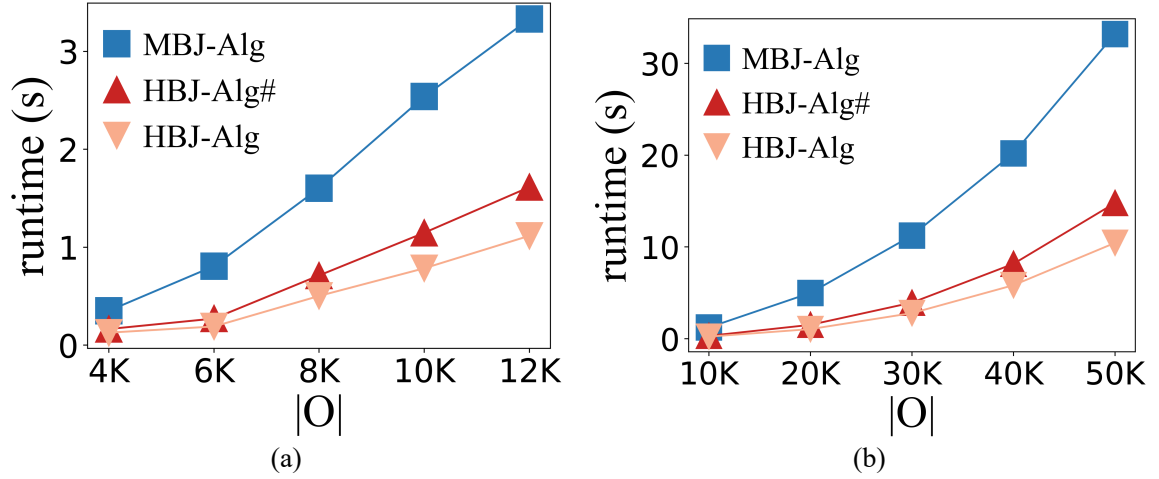


图 7-7 不同  $|O|$  下的查询时间影响 ?? :Geolife 数据集下的实验; ?? :Porto 数据集下的实验

数据集包含 12,000 个移动对象、Porto 数据集包含 50,000 个移动对象时, HBJ-Alg 相比 MBJ-Alg 在 Geolife 上实现了  $3.1x$  的加速, 在 Porto 上实现了  $3.3x$  的加速。

#### 7.4.2.3 特征数量的影响

在实验中, 我们将运动范围的特征集合设定为兴趣点 (POI)。直观而言, 在相同的时间区间运动范围内, 较大的  $|F|$  意味着两个对象之间存在更多潜在的交集位置, 这可能由于需要进行额外的位置检查而导致运行时间增加。如图 ?? 所示, 随着  $|F|$  的增大, 所有算法的运行时间均呈现出轻微的上升趋势。然而, 所提出的预检查策略和剪枝增强方法能够在早期阶段有效地剔除大量不可能满足条件的对象对, 从而显著减少需要进行 POI 交集计算的候选数量。因此, POI 集合规模对整

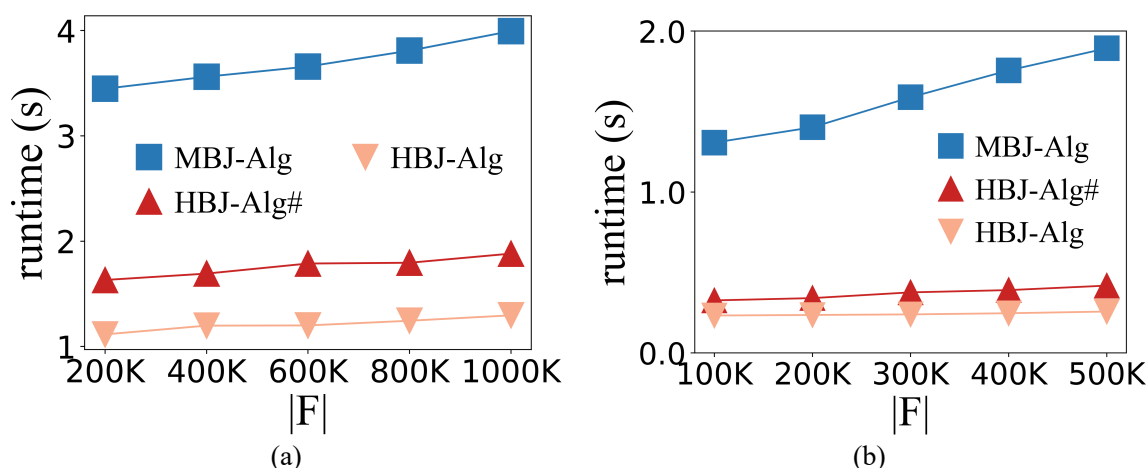


图 7-8 POI 数量对运行时间的影响 ?? : Geolife 数据集下的实验; ?? : Porto 数据集下的实验

体运行时间的影响较小，进一步验证了所提出方法的良好可扩展性。

#### 7.4.2.4 时间区间的影响

我们使用  $|I|$  表示给定时间区间的持续时长。具体而言，在 Geolife 数据集中， $|I|$  从 10 秒变化到 50 秒；在 Porto 数据集中， $|I|$  从 15 秒变化到 75 秒。较大的  $|I|$  会扩大潜在的时间区间运动范围，从而导致具有交集的对象对数量增加以及候选集规模增大。因此，所有算法在识别满足条件的对象对时都需要更多的计算开销。图 ?? 展示了随着  $|I|$  增大时的效率表现。正如预期的那样，随着  $|I|$  的增加，所有算法的运行时间均呈上升趋势，其中 HBJ-Alg 在所有参数设置下始终优于 MBJ-Alg 和 HBJ-Alg#。通过利用 M\*-tree，我们在一定程度上提升了过滤性能，并减少了后续需要进一步处理的候选对象数量。然而，由于 M-tree 的原始设计在构建阶段需要进行耗时的节点分裂操作，其并不适合直接用于 IS-Join 查询。此外，在大规模移动对象场景下，对仅覆盖单个时间区间运动范围的圆进行索引，可能会导致搜索空间膨胀，从而对查询性能产生不利影响。

#### 7.4.2.5 $\epsilon$ 的影响

较小的  $\epsilon$  值意味着需要探测更多的时间点运动范围，使得放松后的相似度更加接近真实的交集相似度，从而带来更高的计算开销，并导致运行时间的增加。图 ?? 展示了在变化放松参数  $\epsilon$  时的效率表现。在实验中，我们将时间点运动范围的数量从 2 变化到 10。随着  $\epsilon$  的变化，可以观察到运行时间呈现出轻微的上升趋势。同时也可以看到，在所有参数设置下，HBJ-Alg 始终表现出最优的性能。

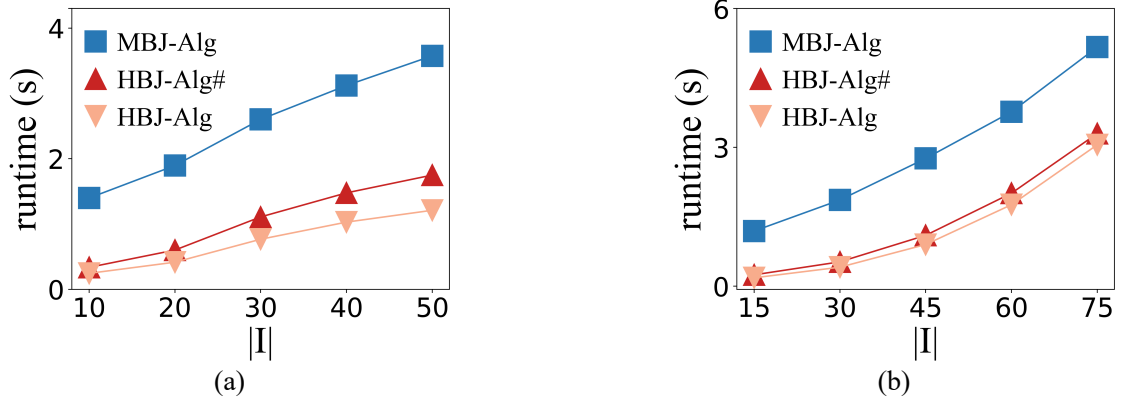


图 7-9  $|I|$  对运行时间的影响 ?? :Geolife 数据集下的实验; ?? :Porto 数据集下的实验

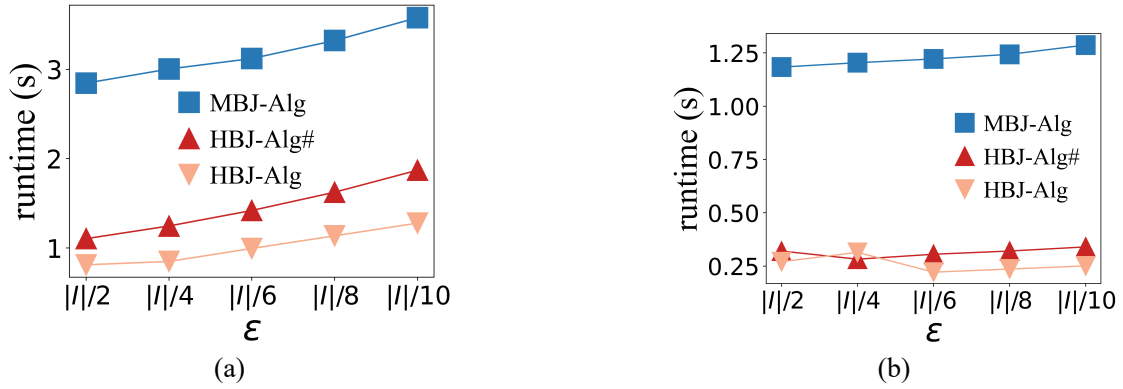


图 7-10 参数  $\epsilon$  的影响 ?? :Geolife 数据集下的实验; ?? :Porto 数据集下的实验

## 7.5 基于 $M^*$ -树的连接算法

首先, 我们介绍一种基于  $M^*$ -树的连接算法 (MBJ-Alg) 用于计算 IS-Join, 这作为基线方法。 $M^*$ -树是  $M$ -tree 的一种扩展 [??]。MBJ-Alg 是一个两阶段算法, 结合了预检查策略, 其中使用  $M$ -tree 的变体来增强候选对象的过滤能力。

在阶段 1 中,  $M^*$ -树使用近似圆来索引时间区间内的运动范围。这种圆形索引允许提前剪枝不合格对象, 从而集中于更可能发生交集的候选对象。在阶段 2 中, 对候选对象在时间点运动范围级别进行评估。下面将介绍  $M^*$ -树的结构及其在 IS-Join 查询中的应用。

### 7.5.1 $M^*$ -树

图 ?? 展示了  $M^*$ -树的结构, 其指定容量为  $M = 4$ , 表示每个叶节点的最大条目数。近似圆被视为单独的条目, 并独立存储在叶节点中。我们使用近似圆表

**算法 7-2: 基于基本过滤与精炼的算法**


---

**输入:** 两个移动对象集合,  $\mathcal{A}$  和  $\mathcal{B}$ ;  
 交集相似度阈值,  $\theta$ 。  
**输出:** 交集对,  $\mathcal{R}$ 。

```

1 初始化:  $\mathcal{L} \leftarrow \emptyset$ ;
2 for  $o \in \mathcal{A}$  do                                     /* 遍历集合  $\mathcal{A}$  */
3     for  $o' \in \mathcal{B}$  do                               /* 遍历集合  $\mathcal{B}$  */
4         if  $\text{dist}(MR(o), MR(o')) \leq MR(o).a + MR(o').a$  then
5              $\mathcal{L}.\text{add}((o, o'))$ ;
6 for  $(o, o') \in \mathcal{L}$  do                               /* 精炼步骤 */
7     if  $IS(o, o', I, \epsilon) \geq \theta$  then
8          $\mathcal{R} \leftarrow \{\mathcal{R}, (o, o')\}$ ;
9 return  $\mathcal{R}$ ;
    
```

---

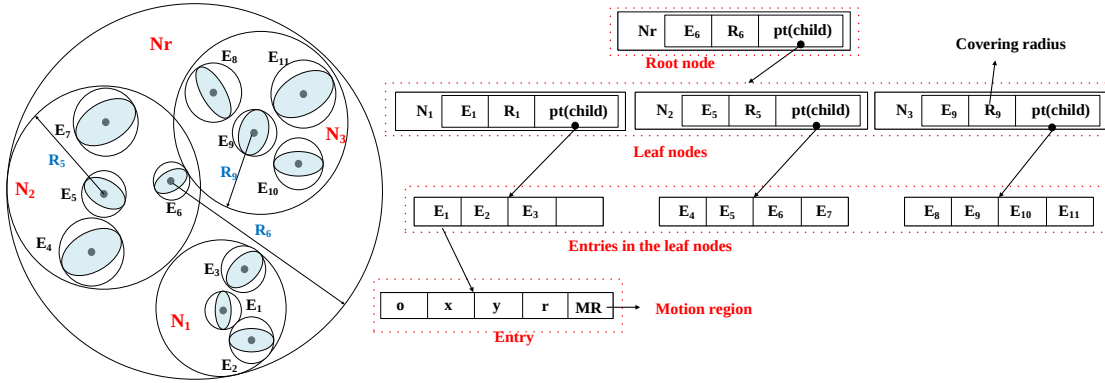


图 7-11 M\*-树的结构

示时间区间的运动范围, 并在 M\*-树中对其进行索引。每个圆存储为条目  $E = \{o, x, y, r, MR(o, I)\}$ , 其中  $o$  为对象标识,  $(x, y)$  表示圆心,  $r$  为半径,  $MR(o, I)$  包含时间区间的运动范围。M\*-树中的每个节点  $N$  选择叶节点中的一个条目  $E$  作为其路由对象, 格式为  $N = \{E, R, pt(child)\}$ 。这里,  $E$  是路由对象,  $R$  是能够覆盖  $N$  中所有条目 (即圆) 的半径,  $pt(child)$  是指向子节点的指针。需要注意的是, M\*-树在条目插入策略、节点分裂策略和路由对象选择策略上遵循了已有研究 [??] 的做法, 并将这些策略适配到 M\*-树结构中。

值得注意的是, 同一 M\*-树内的时间区间运动范围在时间上是对齐的。我们为特定时间区间索引所有运动范围, 当时间区间改变时, 需要重建 M\*-树索引。

### 7.5.2 交集检查

我们从查询条目  $E_q = (q, x_q, y_q, r_q, MR(q, I))$  和 M\*-树的根节点开始。目标是检索所有可能与  $E_q$  相交的条目  $E = \{o, x, y, r, MR(o, I)\}$ 。为了确定潜在交集, 我

们计算查询对象  $q$  与每个条目  $E$  之间的距离  $d(q, E) = \sqrt{(x_q - x)^2 + (y_q - y)^2}$ 。如果  $d(q, E) \leq r + r_q$ ，则时间区间运动范围  $MR(q, I)$  和  $MR(o, I)$  可能相交。其中， $r$  和  $r_q$  分别表示条目  $E$  和  $E_q$  的半径。对于每个查询对象  $q$ ，我们索引所有满足  $d(q, E) \leq r + r_q$  的条目，并将对应的  $(q, o)$  加入候选集合，其中  $o$  是与条目  $E$  关联的对象标识。

### 7.5.3 算法细节

---

**算法 7-3:**  $MBJ(\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, \theta)$ 


---

**输入:** 两组移动对象,  $\mathcal{A}$  和  $\mathcal{B}$ ;

$M^*$ -树的根节点  $N_r$ ;

时间间隔  $I$ ;

分区参数  $\epsilon$ ;

交集相似度阈值  $\theta$ 。

**输出:**  $\mathcal{R} = \{(o, o') \mid IS(o, o', I, \epsilon) \geq \theta, (o, o') \in \mathcal{A} \times \mathcal{B}\}$ 。

1 初始化:  $\mathcal{R} \leftarrow \emptyset$ ;

2 **for**  $o \in \mathcal{A}$  **do**

*/\* 遍历集合  $\mathcal{A}$  \*/*

3      $\mathcal{L} \leftarrow \{N_r\}$ ;

4     **while**  $\mathcal{L} \neq \emptyset$  **do**

*/\*  $M^*$ -树遍历 \*/*

5          $N \leftarrow \mathcal{L}.pop()$ ;

6         **if**  $N$  是非叶子节点 **then**

*/\* 非叶子节点 \*/*

7             **for**  $N_c \in N.pt(child)$  **do**

*/\* 遍历子节点 \*/*

8                 **if**  $d(o, N_c) \leq N_c.R + o.r$  **then**

9                      $\mathcal{L}.push(N_c)$ ;

10         **else**

*/\* 叶子节点处理 \*/*

11             **if**  $d(o, N) > N.R + o.r$  **then**

12                  $prune(o, N.o)$ ;

*// 预检查策略*

13             **if**  $IS(o, N.o, I, \epsilon) \geq \theta$  **then**

14                  $\mathcal{R} \leftarrow \{\mathcal{R}, (o, N.o)\}$ ;

*// 精炼步骤*

15 **return**  $\mathcal{R}$ ;

---

算法 ?? 给出了基于  $M^*$ -树的两阶段剪枝匹配算法 (MBJ-Alg) 的伪代码。算法的输入包括两个移动对象集合、 $M^*$ -树的根节点、带分区参数  $\epsilon$  的时间区间, 以及相似度阈值  $\theta$ 。我们使用列表  $\mathcal{R}$  存储所有满足条件的对象对, 初始时  $\mathcal{R}$  为空 (第 1 行)。

该算法由两个阶段组成: (1) 基于  $M^*$ -树的预检查 (第 3–12 行), 以及 (2) 精炼过程 (第 13–14 行)。在预检查阶段, 我们根据  $M^*$ -树获取候选对象集合。我们



使用列表  $\mathcal{L}$  存储可能与移动对象相交的节点，初始时  $\mathcal{L}$  仅包含根节点（第 3 行）。

当  $\mathcal{L}$  非空时，我们从中取出与对象  $o$  的距离  $d(o, N)$  最小的  $M^*$ -树节点  $N$ （第 5 行）。如果  $N$  不是叶节点，则考虑其子节点。对于  $N$  的每个子节点  $N_c$ ，我们检查  $o$  与  $N_c$  之间的距离。如果该距离不大于  $o$  与  $N_c$  半径之和，则将  $N_c$  压入优先队列（第 7-9 行）。

如果  $N$  是叶节点，且  $d(o, N) > N.R + o.r$ ，则可以安全地剪枝  $(o, N.o)$ （第 11-12 行）。当交集相似度不小于阈值  $\theta$  时，我们通过将  $(o, N.o)$  添加到结果集来更新  $\mathcal{R}$ （第 14 行）。

最后，返回  $\mathcal{R}$  作为得到的交集对象对（第 15 行）。

### 7.5.3.1 时间复杂度分析

一般来说，构建  $M^*$ -树需要  $O(|\mathcal{B}| \times n)$  的时间，其中  $\mathcal{B}$  是被  $M^*$ -树索引的对象集合， $n$  是节点分裂操作的次数。具体来说，当由于对象插入导致节点已满时，需要将其分裂为多个节点，这一操作的时间复杂度为  $O(|\mathcal{B}|)$ 。值得注意的是，实际构建时间可能会因数据分布等因素而有所不同。

设  $\mathcal{A}$  为查询对象集合。为了找到所有候选对象，过滤过程的时间复杂度为  $O(|\mathcal{A}| \times \log |\mathcal{B}|)$ 。若时间点运动范围内的 POI 平均数量为  $p$ ，则每次计算  $IS(o, o', I, \epsilon)$  的时间复杂度为  $O(|I|/\epsilon \times p)$ 。

设  $\mathcal{C}$  为候选对象集合。对候选对象进行精炼的时间复杂度为  $O(|\mathcal{C}| \times |I|/\epsilon \times p)$ 。

因此，MBJ-Alg 每轮执行的总时间复杂度为

$$O(|\mathcal{B}| \times n + |\mathcal{A}| \times \log |\mathcal{B}| + |\mathcal{C}| \times |I|/\epsilon \times p)。$$

## 7.6 Top- $k$ 交集相似度连接

我们的方法可以很容易扩展以支持 top- $k$  交集相似度连接 ( $k$ -IS-Join)，定义如下。与普通 IS-Join 不同， $k$ -IS-Join 不需要阈值参数，而是返回两个集合中前  $k$  个最相似的轨迹对。

**定义 7.9 ( $k$ -IS-Join)** Top- $k$  交集相似度连接 ( $k$ -IS-Join) 从两个对象集合中找到  $k$  个最相似的对象对集合  $\mathcal{R}$ ，即  $|\mathcal{R}| = k$ ，且满足  $\forall (o_i, o_j) \in \mathcal{R} (\forall (o'_i, o'_j) \in (\mathcal{A} \times \mathcal{B}) \setminus \mathcal{R}, \text{有 } (IS(o_i, o_j, I, \epsilon) > IS(o'_i, o'_j, I, \epsilon)))$ 。

算法 ?? 和算法 ?? 分别给出了基于  $M^*$ -树和 HBall-树的  $k$ -IS-Join 解法。

---

**算法 7-4:  $kMBJ(\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, k)$** 


---

**输入:** 两个移动对象集合,  $\mathcal{A}$  和  $\mathcal{B}$ ;  
 M\*-树的根节点  $N_r$ ;  
 时间区间  $I$ ;  
 分区参数  $\epsilon$ ;  
 参数  $k$ 。  
**输出:**  $\mathcal{R} = \text{top-}k$  对象对。

```

1  初始化:  $\mathcal{R} \leftarrow$  大小为  $k$  的优先队列  $\text{PriorityQueue}(\langle(o, o'), sim\rangle)$ ;
2  for  $o \in \mathcal{A}$  do                                     /* 遍历集合  $\mathcal{A}$  */
3       $\mathcal{L} = N_r.query(MR(o, I))$ ;
4      for  $o' \in \mathcal{L}$  do                                 /* 遍历候选对象 */
5           $sim_k = \mathcal{R}.top().sim$ ;
6          if  $|\mathcal{R}| < k$  then
7               $\mathcal{R}.push((o, o'), sim)$ ;
8          else if  $IS(o, o', I, \epsilon) \geq sim_k$  then
9               $\mathcal{R}.push((o, o'), sim)$ ;  $\mathcal{R}.poll()$ ;
10 return  $\mathcal{R}$ ;
    
```

---



---

**算法 7-5:  $kHBJ(\mathcal{A}, \mathcal{B}, N_r, I, \epsilon, k)$** 


---

**输入:** 两个移动对象集合,  $\mathcal{A}$  和  $\mathcal{B}$ ;  
 HBall-树的根节点  $N_r$ ;  
 时间区间  $I$ ;  
 分区参数  $\epsilon$ ;  
 参数  $k$ 。  
**输出:**  $\mathcal{R} = \text{top-}k$  对象对。

```

1  初始化:  $\mathcal{R} \leftarrow$  大小为  $k$  的优先队列  $\text{PriorityQueue}(\langle(o, o'), sim\rangle)$ ;
2  for  $o \in \mathcal{A}$  do                                     /* 遍历集合  $\mathcal{A}$  */
3       $\mathcal{L} = N_r.query(MR(o, I))$ ;
4      for  $o' \in \mathcal{L}$  do                                 /* 遍历候选对象 */
5           $sim_k = \mathcal{R}.top().sim$ ;
6          if  $|\mathcal{R}| < k$  then
7               $\mathcal{R}.push((o, o'), sim)$ ;
8          else
9              if  $IS(o, o', I, \epsilon)'_{ub} < \theta$  then
10                 continue;                             // 时间点剪枝
11             if  $IS(o, o', I, \epsilon) \geq sim_k$  then
12                  $\mathcal{R}.push((o, o'), sim)$ ;  $\mathcal{R}.poll()$ ;
13 return  $\mathcal{R}$ ;
    
```

---

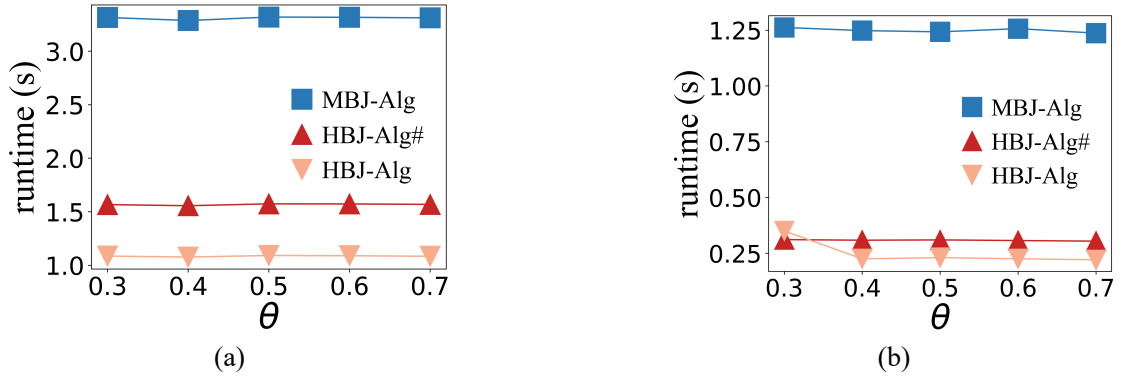


图 7-12 相似度阈值的影响 ?? :Geolife 数据集下的实验; ?? :Porto 数据集下的实验

## 7.7 IS-Join 的附加实验

### 7.7.0.1 相似度阈值的影响

对于 HBJ-Alg, 增加相似度阈值  $\theta$  可以提高剪枝效果。较大的  $\theta$  减少搜索空间, 从而降低 CPU 时间和访问的时间区间运动范围数量。图 ?? 展示了在改变相似度阈值时的效率表现。与 MBJ-Alg 对比, HBJ-Alg 在所有设置下均优于 MBJ-Alg, 运行时间最多可提升 3 倍。虽然 HBJ-Alg 在较高阈值下运行时间略有下降, 但 MBJ-Alg 和 HBJ-Alg# 的趋势不明显。

### 7.7.0.2 重新分区策略的评估

我们比较 HBJ-Alg 和 HBJ-Alg\* 的性能, 主要指标为节点访问总次数。图 ?? 显示了移动对象数量对重新分区策略性能的影响 (默认设置)。随着移动对象数量增加, 节点访问总数也增加。HBJ-Alg 与 HBJ-Alg\* 的性能差距随对象数量增长而扩大。HBJ-Alg 的重新分区策略可在不增加索引构建时间的情况下, 将节点访问数减少约 10%, 甚至可能降低索引构建时间 (见图 ??)。因此, 重新分区策略有助于提升整体效率, 处理大规模数据集时更有效。

## 7.7.1 Top- $k$ IS-Join 的实验研究

### 7.7.1.1 $k$ 的影响

我们评估算法在 top- $k$  交集相似度连接中的性能。图 ?? 展示了不同  $k$  下的效率表现。可以看出, HBJ-Alg 在 Geolife 和 Porto 数据集上的 CPU 时间分别比 MBJ-Alg 提升约 3 倍和 5 倍。这表明我们的方法在 top- $k$  连接中具有明显优势。

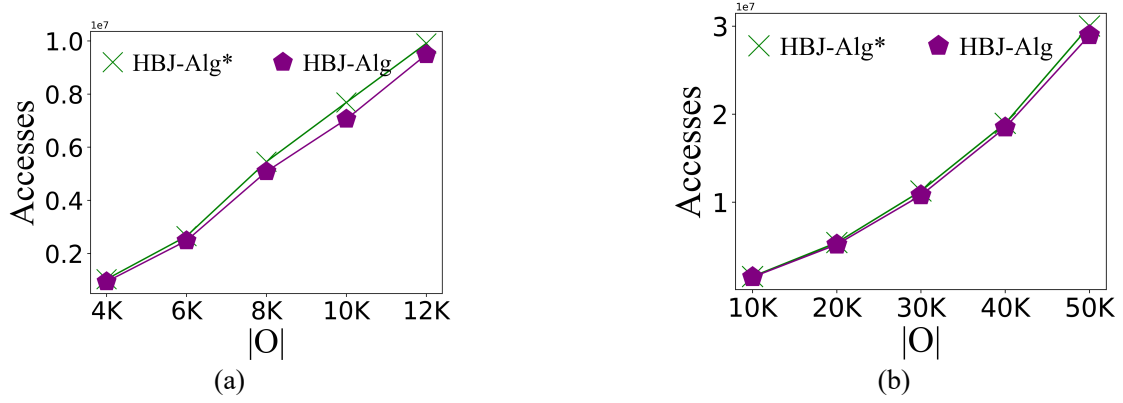


图 7-13 移动对象数量对节点访问次数的影响 ??:Geolife 数据集下的实验;  
 ??:Porto 数据集下的实验

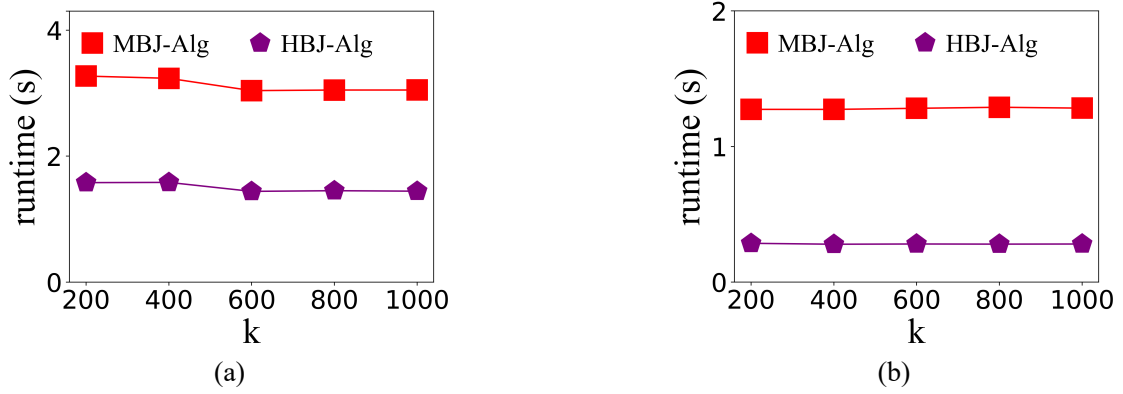


图 7-14  $k$  的影响 ??:Geolife 数据集下的实验; ??:Porto 数据集下的实验

## 7.8 特征精炼

本节中，我们将时间点运动范围的特征视为面积。第 ?? 节介绍了对应的候选对象精炼方法，用于近似交集相似度计算。

### 7.8.0.1 候选对象精炼

精确计算两个时间点运动范围的交集面积非常困难，计算代价高昂 [?]。为了近似计算时间点交集相似度，我们在时间点运动范围内进行均匀采样。具体方法为，将运动范围包裹在矩形内，在矩形内均匀采样，再用椭圆方程过滤掉矩形外的点。一个更简洁的方法是极坐标采样 [?]。

具体做法如下：在  $MR(o, t_1)$  上采样点，并计算落在  $MR(o', t_1)$  内的点数量  $n_1$ ；在  $MR(o', t_1)$  上采样点，并计算落在  $MR(o, t_1)$  内的点数量  $n_2$ 。基于此，时间点交集相似度近似计算公式如下（记为  $IS'(o, o', t)$ ）：

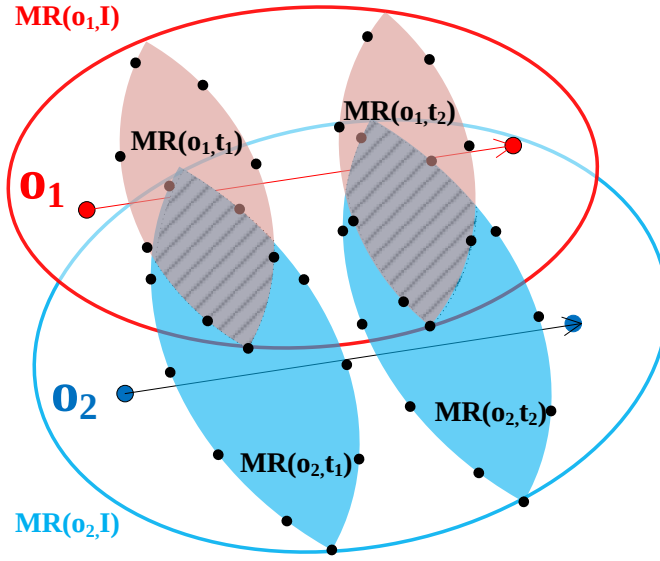


图 7-15 时间点运动范围内的采样示意

$$IS'(o, o', t) = \frac{n_1}{|S_o|} \times \frac{n_2}{|S_{o'}|} \quad (7-10)$$

其中， $|S_o|$  和  $|S_{o'}|$  分别表示对象  $o$  和  $o'$  的时间点运动范围的采样点数量。采样点数量与时间点运动范围面积成正比。采样点越多，近似结果越接近真实值，但计算开销也越大。

**示例.** 图 ?? 展示了候选对象精炼的示例。假设候选集合  $\mathcal{L} = \{(o_1, o_2)\}$ ，其时间区间运动范围有交集。将当前时间区间划分为两个时间片  $t_1^\epsilon$  和  $t_2^\epsilon$ 。在  $o_1$  的时间点运动范围上采样 8 个点，在  $o_2$  上采样 10 个点。

在  $t_1$  时， $n_1 = 3$ ， $n_2 = 2$ ，则根据公式 ??， $IS'(o, o', t_1) = \frac{3}{8} \times \frac{2}{10} = \frac{3}{40}$ 。在  $t_2$  时， $IS'(o, o', t_2) = \frac{4}{8} \times \frac{2}{10} = \frac{1}{10}$ 。因此， $IS'(o, o', I, \epsilon) = (\frac{3}{40} + \frac{1}{10})/2 = \frac{7}{80}$ 。

## 7.9 本章小结

本文提出了一种新的面向移动对象的运动范围感知相似连接框架 IS-Join。该框架通过引入对象的潜在运动区域，而非仅依赖离散的轨迹点，有效刻画了对象之间的时空交互关系。不同于传统的轨迹相似连接方法，IS-Join 以运动范围之间的交集相似度为建模对象，使其能够支持更加灵活的查询，适用于交通监控、野生动物追踪以及接触追踪等应用场景。为高效地在大规模数据上处理 IS-Join 查询，我们提出了一种融合重划分策略的 HBall-tree 索引结构，并设计了预检查与剪枝技术，以减少不必要的候选对象对评估，从而显著提升处理效率。基于两个真实世界数据集的大量实验结果表明，与精心设计的基线方法相比，我们的方法在性能

上最高可获得约 3 倍的加速效果。

| 符号                         | 含义            |
|----------------------------|---------------|
| $o$                        | 一个移动对象        |
| $s$                        | 一个带时间戳的位置采样点  |
| $[x, y], t$                | 坐标 / 时间戳      |
| $I$                        | 一个时间区间        |
| $MR(\cdot)$                | 对象的运动范围       |
| $dist(\cdot)$              | 距离度量函数        |
| $\mathcal{F}(\cdot)$       | 运动范围的特征集合     |
| $IS(\cdot)$                | 交集相似性度量       |
| $\theta$                   | 交集相似性阈值       |
| $\epsilon$                 | 放宽划分参数        |
| $T^\epsilon$               | 采样时间戳集合       |
| $\mathcal{A}, \mathcal{B}$ | 两个移动对象集合      |
| $\mathcal{R}$              | IS-Join 的结果集合 |

表 7-1 符号说明汇总

|                 | Geolife                                                                                                  | Porto                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| $ O $           | $\{4, 6, 8, 10, 12\} \times 10^3$<br>8K (默认)                                                             | $\{1, 2, 3, 4, 5\} \times 10^4$<br>10K (默认)                                                              |
| $ I $           | $\{10s, 20s, 30s, 40s, 50s\}$<br>30s (默认)                                                                | $\{15s, 30s, 45s, 60s, 75s\}$<br>45s (默认)                                                                |
| $ \mathcal{F} $ | $\{2, 4, 6, 8, 10\} \times 10^5$<br>600K (默认)                                                            | $\{1, 2, 3, 4, 5\} \times 10^5$<br>100K (默认)                                                             |
| $\epsilon$      | $\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$<br>$\frac{ I }{6}$ (默认) | $\{\frac{ I }{2}, \frac{ I }{4}, \frac{ I }{6}, \frac{ I }{8}, \frac{ I }{10}\}$<br>$\frac{ I }{6}$ (默认) |

表 7-2 评估参数设置

| 指标/数据集 \ 方法     | MBJ-Alg  | HBJ-Alg  |
|-----------------|----------|----------|
| 候选集规模 (Geolife) | 6540764  | 28862    |
| 剪枝率 (Geolife)   | 89.7800% | 99.9549% |
| 候选集规模 (Porto)   | 2142988  | 50072    |
| 剪枝率 (Porto)     | 97.8570% | 99.9499% |

表 7-3 剪枝性能

## 第八章 总结与展望



## 第九章 后截断章节

## 致 谢

## 参考文献

- [1] Dumitrescu I, Boland N. Improved preprocessing, labeling and scaling algorithms for the weight-constrained shortest path problem[J]. *Networks*, 2003, 42(3): 135–153.
- [2] Sharifzadeh M, Kolahdouzan M R, Shahabi C. The optimal sequenced route query[J]. *VLDB J.*, 2008, 17(4): 765–787.
- [3] Levin R, Kanza Y, Safra E, et al. Interactive route search in the presence of order constraints[J]. *PVLDB*, 2010, 3(1): 117–128.
- [4] Shang S, Lu H, Pedersen T B, et al. Modeling of traffic-aware travel time in spatial networks[C]. *MDM (1)*, IEEE Computer Society, 2013: 247–250.
- [5] Li J, Yang Y D, Mamoulis N. Optimal route queries with arbitrary order constraints[J]. *IEEE Trans. Knowl. Data Eng.*, 2013, 25(5): 1097–1110.
- [6] Zeng Y, Chen X, Cao X, et al. Optimal route search with the coverage of users' preferences[C]. *IJCAI*, AAAI Press, 2015: 2118–2124.
- [7] Xu Y, Chen L, Yao B, et al. Location-based top-k term querying over sliding window[C]. *WISE (1)*, Springer, vol. 10569 of *Lecture Notes in Computer Science*, 2017: 299–314.
- [8] Malviya N, Madden S, Bhattacharya A. A continuous query system for dynamic route planning[C]. *ICDE*, IEEE Computer Society, 2011: 792–803.
- [9] Xu J, Guo L, Ding Z, et al. Traffic aware route planning in dynamic road networks[C]. *DASFAA (1)*, Springer, vol. 7238 of *Lecture Notes in Computer Science*, 2012: 576–591.
- [10] Lim S, Rus D. Stochastic distributed multi-agent planning and applications to traffic[J]. *ICRA*, 2012: 2873-2879.
- [11] Babak J, Abbas B, (Avi) C A. Path-based capacity-restrained dynamic traffic assignment algorithm[J]. *Transportmetrica B: Transport Dynamics*, 2018: 1-24.
- [12] Li K, Chen L, Shang S. Towards alleviating traffic congestion: Optimal route planning for massive-scale trips[C]. *IJCAI*, *ijcai.org*, 2020: 3400–3406.
- [13] Dijkstra E W. A note on two problems in connexion with graphs[J]. *Numerische Mathematik*, 1959, 1: 269-271.
- [14] Xu J, Lu H, Bao Z. IMO: A toolbox for simulating and querying "infected" moving objects[J]. *Proc. VLDB Endow.*, 2020, 13(12): 2825–2828.
- [15] Alarabi L, Basalamah S M, Hendawi A M, et al. Traceall: A real-time processing for contact tracing using indoor trajectories[J]. *Inf.*, 2021, 12(5): 202.

- [16] Shang S, Chen L, Zheng K, et al. Parallel trajectory-to-location join[J]. TKDE, 2019, 31(6): 1194–1207.
- [17] Shang S, Chen L, Wei Z, et al. Parallel trajectory similarity joins in spatial networks[J]. VLDB J., 2018, 27(3): 395–420.
- [18] Ding Z, Li K, Chen L, et al. Parallel online similarity join over trajectory streams[C]. Proceedings of the ACM on Web Conference 2025, 2025: 3426–3437.
- [19] Kong X, Chen Q, Hou M, et al. Mobility trajectory generation: a survey[J]. Artif. Intell. Rev., 2023, 56(Supplement 3): 3057–3098.
- [20] Sistla A P, Wolfson O, Chamberlain S, et al. Modeling and querying moving objects[C]. ICDE, 1997: 422–432.
- [21] Patel J M, Chen Y, Chakka V P. STRIPES: an efficient index for predicted trajectories[C]. SIGMOD, 2004: 637–646.
- [22] Saltenis S, Jensen C S, Leutenegger S T, et al. Indexing the positions of continuously moving objects[C]. SIGMOD, 2000: 331–342.
- [23] Zhang R, Lin D, Ramamohanarao K, et al. Continuous intersection joins over moving objects[C]. ICDE, 2008: 863–872.
- [24] Zhang R, Qi J, Lin D, et al. A highly optimized algorithm for continuous intersection join queries over moving objects[J]. VLDB J., 2012, 21(4): 561–586.
- [25] Bakalov P, Hadjieleftheriou M, Keogh E J, et al. Efficient trajectory joins using symbolic representations[C]. Chrysanthos P K, Samaras G, (Eds.) MDM, ACM, 2005: 86–93.
- [26] Yi B, Faloutsos C. Fast time sequence indexing for arbitrary lp norms[C]. VLDB, Morgan Kaufmann, 2000: 385–394.
- [27] Vishnu C, Abhinav V, Roy D, et al. Improving multi-agent trajectory prediction using traffic states on interactive driving scenarios[J]. IEEE Robotics Autom. Lett., 2023, 8(5): 2708–2715.
- [28] Forrest S, Recio M, Seddon P. Moving wildlife tracking forward under forested conditions with the swift gps algorithm[J]. Animal Biotelemetry, 2022, 10(1): 19.
- [29] Li K, Chen L, Shang S, et al. Towards controlling the transmission of diseases: Continuous exposure discovery over massive-scale moving objects[C]. IJCAI, 2022: 3891–3897.
- [30] Zheng Y, Zhang L, Xie X, et al. Mining interesting locations and travel sequences from GPS trajectories[C]. WWW, 2009: 791–800.
- [31] Li J, Ni C, He D, et al. Efficient knn query for moving objects on time-dependent road networks[J]. VLDB J., 2023, 32(3): 575–594.
- [32] Zheng Y, Xie X, Ma W. Geolife: A collaborative social networking service among user, location and trajectory[J]. IEEE Data Eng. Bull., 2010, 33(2): 32–39.

- 
- [33] Guttman A. R-trees: A dynamic index structure for spatial searching[C]. SIGMOD, 1984: 47–57.
- [34] Brinkhoff T, Kriegel H, Seeger B. Efficient processing of spatial joins using r-trees[C]. SIGMOD, 1993: 237–246.
- [35] Bentley J L. Multidimensional binary search trees used for associative searching[J]. Commun. ACM, 1975, (9): 509–517.
- [36] Zhang M, Chen S, Jensen C S, et al. Effectively indexing uncertain moving objects for predictive queries[J]. Proc. VLDB Endow., 2009, 2(1): 1198–1209.
- [37] Ciaccia P, Patella M, Rabitti F, et al. Indexing metric spaces with m-tree[C]. SEBD, 1997: 67–86.
- [38] Ciaccia P, Patella M, Zezula P. M-tree: An efficient access method for similarity search in metric spaces[C]. VLDB, 1997: 426–435.
- [39] Liu T, Moore A W, Gray A G. New algorithms for efficient high-dimensional nonparametric classification[J]. J. Mach. Learn. Res., 2006, 7: 1135–1158.
- [40] Omohundro S M. Five balltree construction algorithms[M]. International Computer Science Institute Berkeley, 1989.
- [41] Dolatshah M, Hadian A, Minaei-Bidgoli B. Ball\*-tree: Efficient spatial indexing for constrained nearest-neighbor search in metric spaces[J]. CoRR, 2015.
- [42] Chao P, He D, Li L, et al. Efficient trajectory contact query processing[C]. DASFAA, 2021: 658–666.
- [43] Zhang X, Ray S, Shoeleh F, et al. Efficient contact similarity query over uncertain trajectories[C]. EDBT, 2021: 403–408.
- [44] Liang A, Yao B, Wang B, et al. Sub-trajectory clustering with deep reinforcement learning[J]. VLDB J., 2024, 33(3): 685–702.
- [45] Li K, Wang H, Chen Z, et al. Relaxed group pattern detection over massive-scale trajectories[J]. Future Generation Computer Systems, 2023, 144: 131–139.
- [46] Nutanong S, Zhang R, Tanin E, et al. Analysis and evaluation of v\*-knn: an efficient algorithm for moving knn queries[J]. VLDB J., 2010, 19(3): 307–332.
- [47] Ward P G D, He Z, Zhang R, et al. Real-time continuous intersection joins over large sets of moving objects using graphic processing units[J]. VLDB J., 2014, 23(6): 965–985.
- [48] Chang Y, Tanin E, Cong G, et al. Trajectory similarity measurement: An efficiency perspective[J]. Proceedings of the VLDB Endowment, 2024, 17(9): 2293–2306.
- [49] Han P, Wang J, Yao D, et al. A graph-based approach for trajectory similarity computation in spatial networks[C]. KDD, ACM, 2021: 556–564.
- [50] Ding Y, Zhou X, Wu G, et al. Mining spatio-temporal reachable regions with multiple sources over massive trajectory data[J]. IEEE Trans. Knowl. Data Eng., 2021, 33(7): 2930–2942.

- [51] Chen Y, Patel J M. Design and evaluation of trajectory join algorithms[C]. 17th ACM SIGSPATIAL International Symposium on Advances in Geographic Information Systems, Washington, USA: ACM, 2009: 266–275.
- [52] Tampakis P, Doukeridis C, Pelekis N, et al. Distributed subtrajectory join on massive datasets[J]. ACM Trans. Spatial Algorithms Syst., 2020, 6(2): 8:1–8:29.
- [53] Pfoser D, Jensen C S. Capturing the uncertainty of moving-object representations[C]. SSD, 1999: 111–132.
- [54] Trajcevski G, Choudhary A, Wolfson O, et al. Uncertain range queries for necklaces[C]. MDM, 2010: 199–208.
- [55] Moore A W. The anchors hierarchy: Using the triangle inequality to survive high dimensional data[C]. UAI, 2000: 397–405.
- [56] Samet H. The quadtree and related hierarchical data structures[J]. ACM Comput. Surv., 1984, 16(2): 187–260.
- [57] Zheng Y, Li Q, Chen Y, et al. Understanding mobility based on GPS data[C]. UbiComp, 2008: 312–321.
- [58] Moreira-Matias L, Gama J, Ferreira M, et al. Predicting taxi-passenger demand using streaming data[J]. IEEE Trans. Intell. Transp. Syst., 2013, 14(3): 1393–1402.
- [59] Lim S, Balakrishnan H, Gifford D, et al. Stochastic motion planning and applications to traffic[J]. I. J. Robotics Res., 2011, 30(6): 699–712.
- [60] Dotoli M, Hammadi S, Jeribi K, et al. A multi-agent decision support system for optimization of co-modal transportation route planning services[C]. CDC, IEEE, 2013: 911–916.
- [61] Ding B, Yu J X, Qin L. Finding time-dependent shortest paths over large graphs[C]. EDBT, ACM, vol. 261 of ACM International Conference Proceeding Series, 2008: 205–216.
- [62] Huang W, Yan C, Wang J, et al. A time-delay neural network for solving time-dependent shortest path problem[J]. Neural Networks, 2017, 90: 21–28.
- [63] Hua M, Pei J. Probabilistic path queries on road networks[J]. 2011.
- [64] Shang S, Lu H, Pedersen T B, et al. Finding traffic-aware fastest paths in spatial networks[C]. SSTD, Springer, vol. 8098 of Lecture Notes in Computer Science, 2013: 128–145.
- [65] Dafermos, C S. The traffic assignment problem for multiclass-user transportation networks[J]. Transportation Science, 1972, 6(1): 73–87.
- [66] Dafermos S C, Sparrow F T. The traffic assignment problem for a general network[J]. National Bureau of Standardsb-, 1969.
- [67] Peeta S, Mahmassani H S. System optimal and user equilibrium time-dependent traffic assignment in congested networks[J]. Ann. Oper. Res., 1995, 60(1): 81–113.

- 
- [68] Liebig T, Piatkowski N, Bockermann C, et al. Dynamic route planning with real-time traffic predictions[J]. *Inf. Syst.*, 2017, 64: 258–265.
- [69] Shang S, Liu J, Zheng K, et al. Planning unobstructed paths in traffic-aware spatial networks[J]. *GeoInformatica*, 2015, 19(4): 723–746.
- [70] Cao X, Chen L, Cong G, et al. Keyword-aware optimal route search[J]. *Proc. VLDB Endow.*, 2012, 5(11): 1136–1147.
- [71] Shang S, Guo D, Liu J, et al. Prediction-based unobstructed route planning[J]. *Neurocomputing*, 2016, 213: 147–154.
- [72] Cai X, Kloks T, Wong C K. Time-varying shortest path problems with constraints[J]. *Networks*, 1997, 29(3): 141–150.
- [73] Nikolova E, Brand M, Karger D R. Optimal route planning under uncertainty[C]. *ICAPS, AAAI*, 2006: 131–141.
- [74] Chen L, Shang S, Guo T. Real-time route search by locations[C]. *AAAI, AAAI Press*, 2020: 574–581.
- [75] Chen L, Shang S, Jensen C S, et al. Effective and efficient reuse of past travel behavior for route recommendation[C]. *KDD, ACM*, 2019: 488–498.
- [76] Greenshields B. A study of traffic capacity[J]. *A Study of Traffic Capacity*, 1934, 14.
- [77] Wilkie D, van den Berg J P, Lin M C, et al. Self-aware traffic route planning[C]. Burgard W, Roth D, (Eds.) *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2011, San Francisco, California, USA, August 7-11, 2011, AAAI Press*, 2011: 1521–1527.
- [78] Shang S, Ding R, Zheng K, et al. Personalized trajectory matching in spatial networks[J]. *VLDB J.*, 2014, 23(3): 449–468.
- [79] Chen L, Shang S, Yao B, et al. Pay your trip for traffic congestion: Dynamic pricing in traffic-aware road networks[C]. *AAAI, AAAI Press*, 2020: 582–589.
- [80] Hua M, Pei J. Probabilistic path queries in road networks: traffic uncertainty aware path selection[C]. *EDBT, ACM*, vol. 426 of *ACM International Conference Proceeding Series*, 2010: 347–358.
- [81] Özal A, Ranganathan A, Tatbul N. Real-time route planning with stream processing systems: a case study for the city of lucerne[C]. *GIS-IWGS, ACM*, 2011: 21–28.

## 攻读博士学位期间取得的成果

### 发表论文:

- [1] 李科, 陈力思, 商烁. Towards alleviating traffic congestion: Optimal route planning for massive-scale trips. *International Joint Conferences on Artificial Intelligence*, 2021, 3400-3406. (CCF A, 中科院分区, IF: 98.8)
- [2] 李科, 陈力思, 商烁, Panos Kalnis, Bin Yao. Traffic congestion alleviation over dynamic road networks: Continuous optimal route combination for trip query streams. *International Joint Conferences on Artificial Intelligence*, 2021, 62(10): 5344–5348. (CCF A, 中科院分区, IF: 98.8)
- [3] 李科, 饶漩, 庞小兵, 陈力思, 樊思琪. Route search and planning: A survey. *Big data research*, 2021, 26: 100246. (CCF A, 中科院分区, IF: 98.8)
- [4] 陈紫雯, 李科, 周偲琳, 陈力思, 商烁. Towards robust trajectory similarity computation: Representation-based spatio-temporal similarity quantification. *World Wide Web*, 26(4), 1271-1294. (CCF A, 中科院分区, IF: 98.8)
- [5] 李科, 陈力思, 商烁, 王海燕, 刘洋, Panos Kalnis, Bin Yao. Towards Controlling the Transmission of Diseases: Continuous Exposure Discovery over Massive-Scale Moving Objects. *International Joint Conferences on Artificial Intelligence*, 2022, 3891-3897. (CCF A, 中科院分区, IF: 98.8)
- [6] 李科, 王鸿宇, 陈紫雯, 陈力思. Relaxed group pattern detection over massive-scale trajectories. *Future Generation Computer Systems*, 2023, 144: 131-139. (CCF A, 中科院分区, IF: 98.8)
- [7] 王辰昊, 李科, 陈力思. Deep unified attention-based sequence modeling for online anomalous trajectory detection. *Future Generation Computer Systems*, 2023, 144: 1-11. (CCF A, 中科院分区, IF: 98.8)
- [8] 王海燕, 冯佳铭, 李科, 陈力思. Deep understanding of big geospatial data for self-driving: Data, technologies, and systems. *Future Generation Computer Systems*, 2022, 137:146-163. (CCF A, 中科院分区, IF: 98.8)
- [9] 丁重钧, 李科, 陈力思, 商烁. Parallel online similarity join over trajectory streams. *Proceedings of the ACM on Web Conference*, 2025, 3426-3437. (CCF A, 中科院分区, IF: 98.8)



- [10] 王鸿宇, 李科, 商烁. Parallel online similarity join over trajectory streams. *GeoInformatica*, 2024, 28 (2): 335-351. (CCF A, 中科院分区, IF: 98.8)
- [11] 陈紫雯, 李科, 陈力思, 胡楠, Yoshiharu Ishikawa. Co-movement aware trajectory generation via waypoint-guided generative adversarial networks. *GeoInformatica*, 2025, 29 (4):1093-1119. (CCF A, 中科院分区, IF: 98.8)
- [12] 陈紫雯, 李科, Yoshiharu Ishikawa. Ca-gen: Trajectory generation with co-movement awareness. *Proceedings of the 19th International Symposium on Spatial and Temporal Data*, 2025, 115-118. (CCF A, 中科院分区, IF: 98.8)
- [13] 李科, Leong Hou U, 商烁. App2Exa: accelerating exact kNN search via dynamic cache-guided approximation. *International Joint Conferences on Artificial Intelligence*, 2025. (CCF A, 中科院分区, IF: 98.8)
- [14] 李科, 陈力思, 商烁, Christian S Jensen, Panos Kalnis. Beyond Locations: A Motion Range-Aware Similarity Join. *In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, 1424-1434. (CCF A, 中科院分区, IF: 98.8)

**发明专利：**

[1] 作者 1, 作者 2\*, 作者 3, 作者 4。一种基于 xxxxx 的真气运转方法:  
ZL201120846830.0. 2023-02-20.

**参与项目：**

- 项目号. 项目名称. 项目级别, 2020.01–2022.12.